Affirmed Networks Packaging and Deployment Solutions Guide

Release: 5.2.0
Date: May 2025

Affirmed Networks Inc.

A Microsoft Company

Copyright© 2025 Affirmed Networks, Inc. All Rights Reserved.

AFFIRMED & DESIGN®, ACUITAS SERVICE MANAGEMENT SYSTEM®, AFFIRMED NETWORKS MOBILE CONTENT CLOUD™, AFFIRMED OPEN WORKFLOW™, AFFIRMED MOBILE CONTENT CLOUD™, UnityCloud™, UnityCloud PaaS™, UnityCloud Platform™, UnityCloud Operations™, UnityCloud Ops™, Operations and Policy Manager (OPM)™ and other trademarks and designs are the registered or unregistered trademarks of Affirmed Networks, Inc. and its subsidiaries in the United States and in foreign countries. All other trademarks are the property of their respective owners. The Affirmed Networks, Inc. trademarks may not be used in connection with any product or service that is not Affirmed Networks' in any manner that is likely to cause confusion among customers or in any manner that disparages or discredits Affirmed Networks, Inc.

This document contains information that is the property of Affirmed Networks, Inc. This document may not be copied, reproduced, reduced to any electronic medium or machine readable form, or otherwise duplicated, and the information herein may not be used, disseminated or otherwise disclosed, except with the prior written consent of Affirmed Networks, Inc.

Contents

List of Figures

List of Tables

Release Abstract

This document contains the software release notes for the Affirmed Networks UnityCloud current release.

Audience

This document is intended for network operators (sometimes referred to as Users or Operators in this document), installation staff, and other qualified service personnel who want to install, provision, monitor and learn about the Affirmed Networks system and associated features and functions. The audience should also be familiar with Long Term Evolution (LTE), and 3G or 4G networks. It may also be helpful to have a basic understanding of the HTTP protocol, TCP/IP stack, and the use of MMS services.

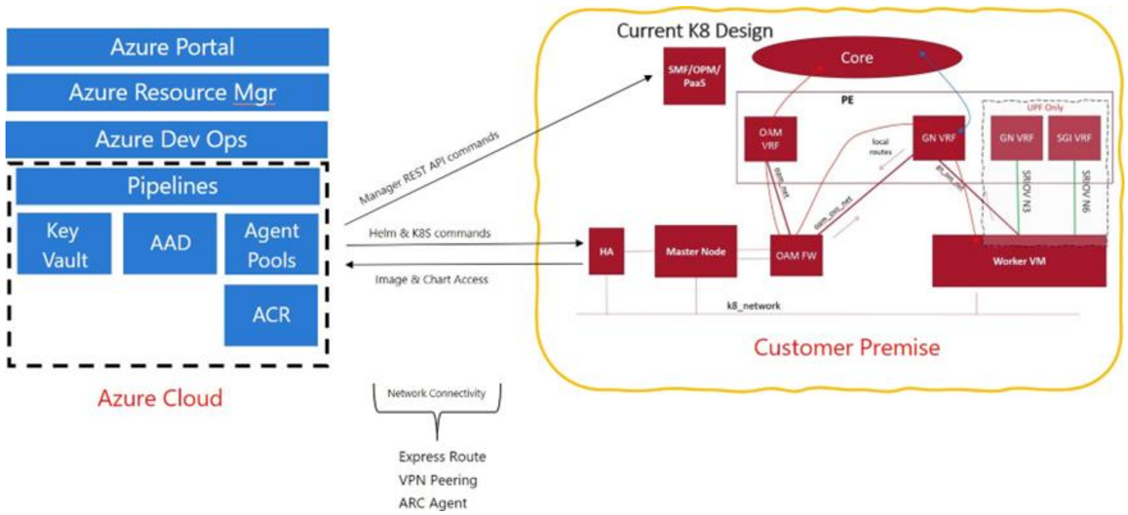
Release Information

See the Release Notes for important information, requirements, limitations, and restrictions that apply to this release.

Solution Design

Packaging and Deployment Solutions (PADS) is a method of using the Azure DevOps Service (ADO) as a deployment tool for UnityCloud. The ADO Pipeline is a continuous integration (CI) and continuous delivery (CD), testing, and deployment system. You can use the ADO release pipeline to automate the Kubernetes cluster finalization and deployment or upgrade the UnityCloud 5GC network-functions (NFs) – SMF (Session Management Function) and UPF (User Plane Function).

Figure 1. PADS High Level Architecture



Pipelines

PADS supports the following types of pipelines:

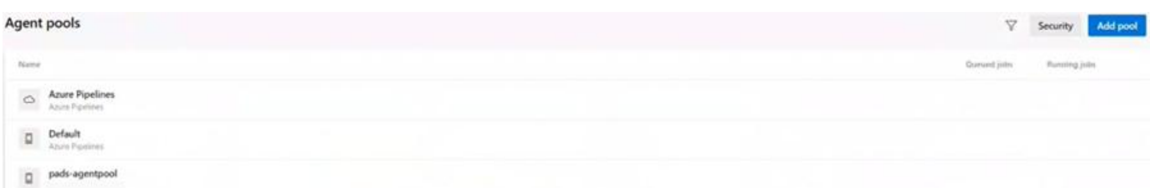
- Cluster building pipelines - used to finalize the cluster.
- CNF deploy/delete/upgrade pipelines - used for life-cycle management.
- Operation status pipeline - used to check whether all containers are up after Day N configuration.
- Pipeline locking pipeline - used to check and reset the current lock for the cluster.

ADO allows you to have a hosted agent and creates its own container on the agent using the specified image each time a pipeline is run. A self-hosted agent means that you provide a virtual machine (VM) or virtual machine scale set (VMSS) where the container is created at pipeline runtime. ADO enables the configured IP address and firewall rules to communicate to your on- premise resources.

Agent Pool

This section describes how to create and configure an agent pool in your ADO Project, as well as how to install and run a self-hosted ADO agent.

Figure 2. Agent Pools



Run a self-hosted ADO agent

1. Create a pool in your ADO Project.
    a. From the Project Settings tab, click **Add Pool** from the Agent Pools section.
    b. Select **Self Hosted** and click **OK**.
2. Install a self-hosted ADO agent by creating a VM in your Resource Group to make the VM act as the agent pool:

```
mkdir -p ~/Downloads; cd ~/Downloads; wget
https://vstsagentpackage.azureedge.net/agent/2.191.1/vsts- agent-linux-
x64-2.191.1.tar.gz
```

```
mkdir -p ~/myagent && cd ~/myagent; tar zxvf ~/Downloads/vsts- agent-linux-x64-2.191.1.tar.gz
```

3. Register the self-hosted ADO agent by generating personal access tokens on ADO.

```
adoToken=<token> adoOrganizationURL="https://dev.azure.com/A40-ADO-PADS-Dev" adoAgentName=$<hostname>
```

> ***Note:*** *You can regenerate the adoToken as needed.*

4. In the Agent Pools section, add the user who configures the permissions to set up an agent pool. The ***adoAgentPoolName*** is based on the pool name on ADO.

   a. From the Project Settings tab, click **Security** from the **Agent Pools** section.

   b. Select **Organization as administrator** and click **OK**.

```
adoAgentPoolName="pad-agentpool" cd ~/myagent

./config.sh remove --unattended --token $<adoToken>

./config.sh --unattended --url $<adoOrganizationURL> --auth pat

--token $<adoToken> --pool $<adoAgentPoolName> --agent

$<adoAgentName> --acceptTeeEula
```

5. Run the self-hosted ADO agent.

```
cd ~/myagent

./run.sh
```

Service Connection

A service connection is a connection between your ADO project and the resources, such as the Azure Container Registry (ACR), that are used by your pipeline. The service connection acts as a link between your ADO Project and your Azure Resource Group, so that ADO can recognize the location of your ACR and key vault.

You need two service connections for the PADS project: one to the Azure Resource Group and the other to the ACR—which is a Docker registry.

Figure 3. Edit Service Connection Window 1

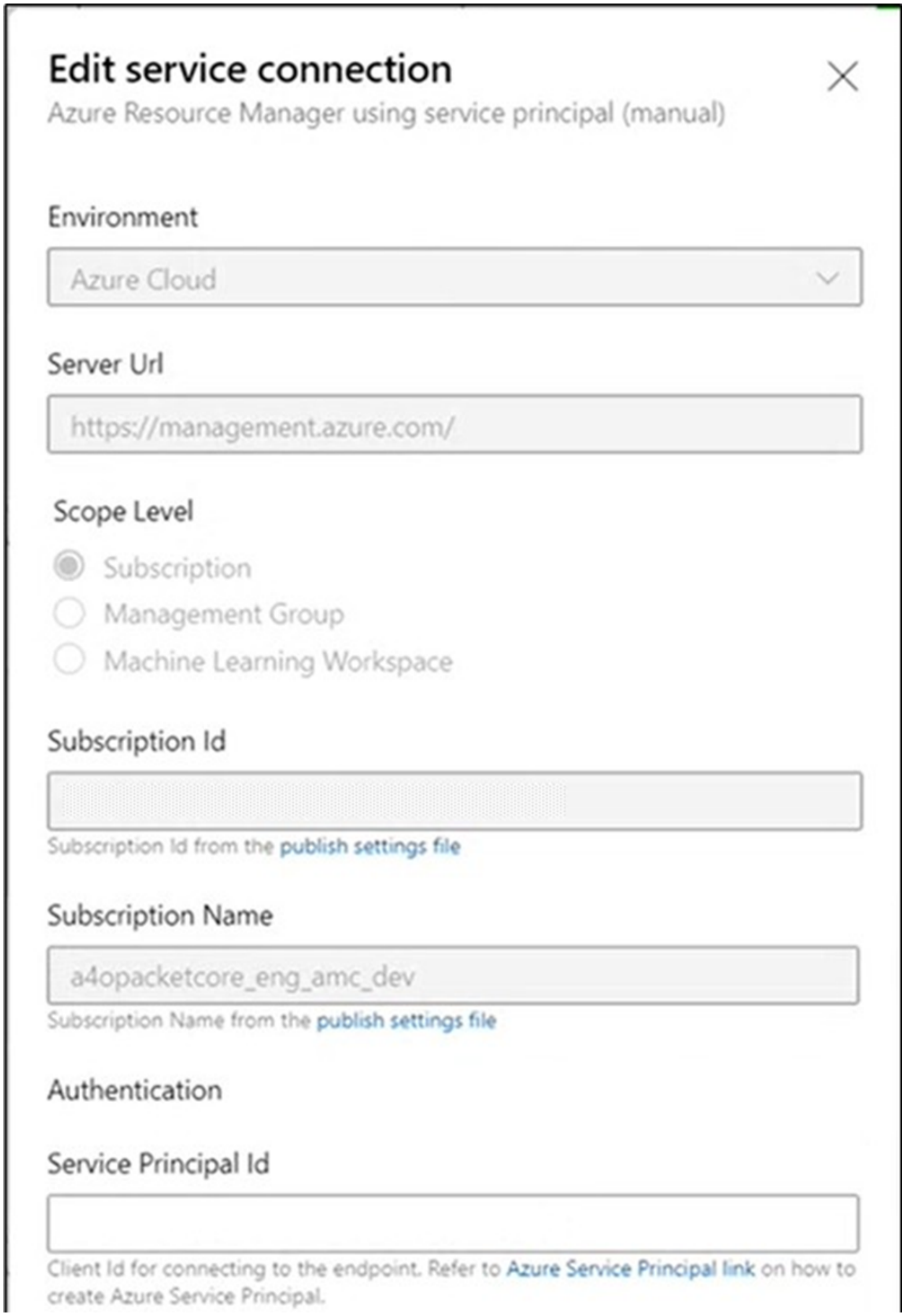
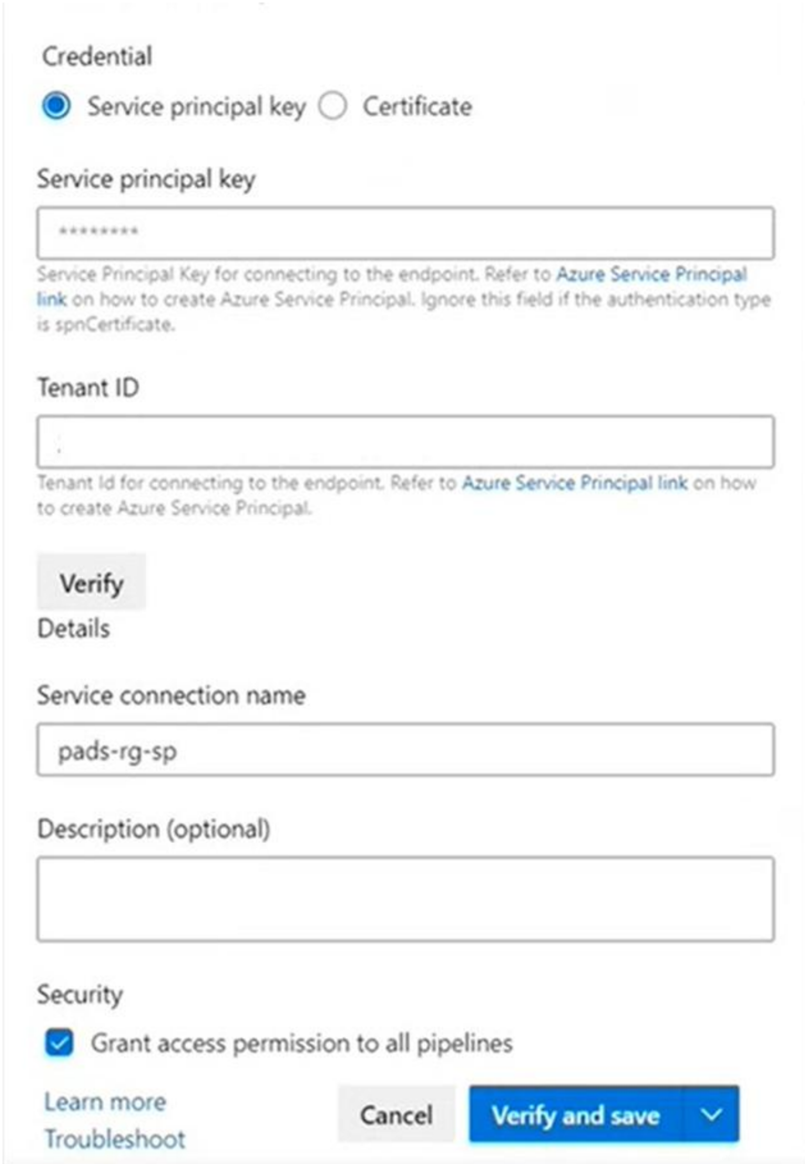
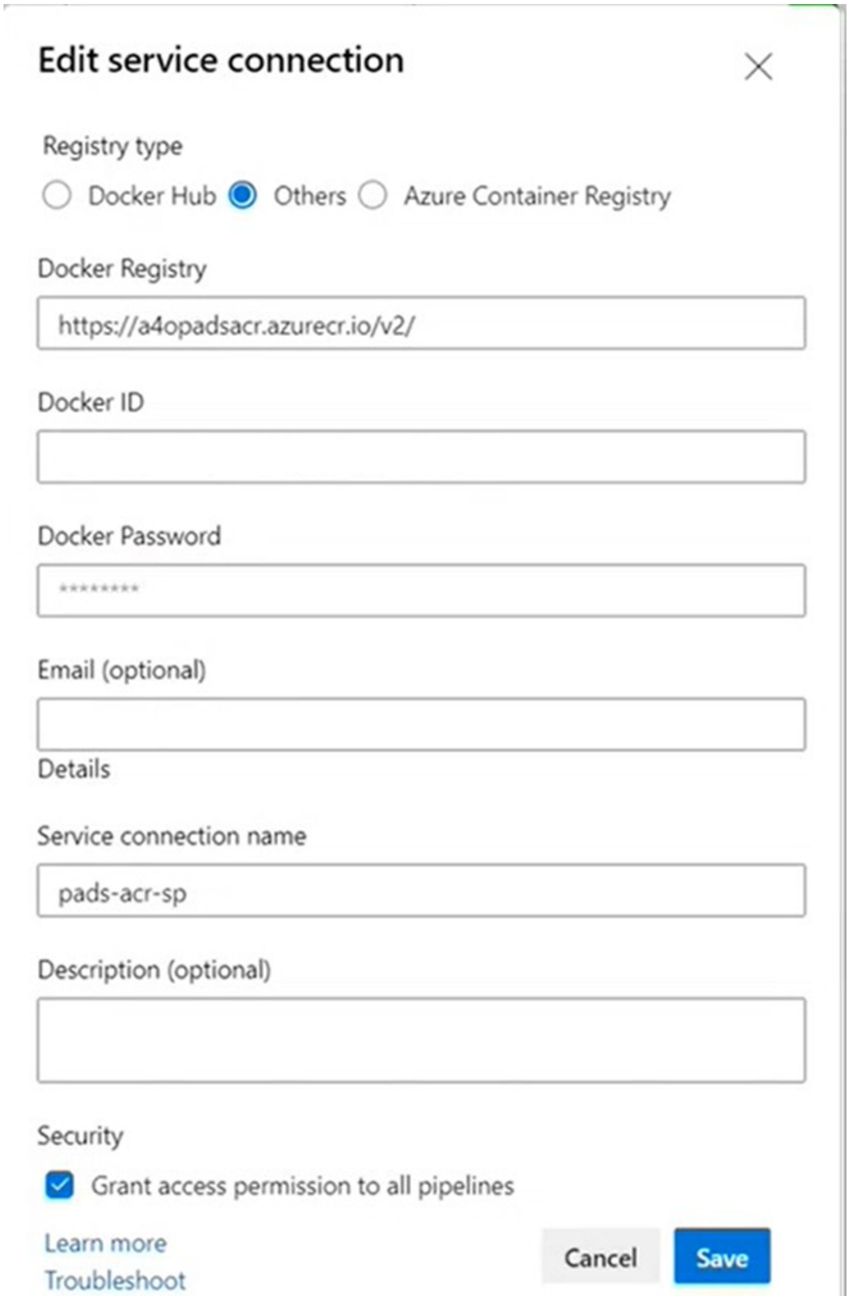


Figure 4. Edit Service Connection Window (Continued)



To connect your ADO project the Resource Group, you must provide an authentication ID and the necessary credentials using the Service Principal fields.

Figure 5. Edit Service Connection Window 2



Set Up Environments on ADO

You must add the following environments on ADO:

- CNF_Deploy

- CNF_Delete
- Finalize_Cluster

Create Environment

To create an environment in your ADO Project.

1. From the Pipelines tab, select **Environments**.
2. On the **Environments** section, click **New environment**.
3. Name the new environment.
4. (Optional) Provide a description and select a Resource.
5. Click **Create**.

Configure the Pipelines

After you install the PADS software (see Install PADS Software), you can add and configure the variable values for the following pipelines:

- ACR to OPM Image Copy Pipeline
- UCReg Image Copy Pipelines
- CNF Delete Pipeline
- CNF Deploy Pipeline
- CNF Upgrade Pipeline
- CNF Operation Status Pipeline
- Finalize K8 Cluster Pipeline
- OPM Automation Pipeline
- CSO Software Install Pipeline
- Pipeline Locking Pipeline
- Helm Deployment Pipeline

Note: *For information about the Nexus CNF Operator pipeline, see Using the Nexus Pipelines.*

affirmed-nr-macrovariables.yml File

This file is added to INFRA-DEV and can contain any macro variables that need to be loaded into the pipelines.

Table 1. affirmed-nr-macrovariables.yml (INFRA-DEV) Variables

Variables	Description
`azureServiceConnection`	Service Connection Name in ADO to Azure Resource Group
`spID`	Azure service principal ID
`spPassword`	Azure service principal password. This must be protected
`spTenant`	Azure tenant id

affirmed-nr-nf-variables-templates.yml File

This file is added to INFRA-DEV and can contain network function variables that nee to be loaded into the pipelines.

Table 2. affirmed-nr-nf-variables-templates.yml (INFRA-DEV) Variables

Variables	Description
`containerImageName`	Docker image name used in the pipeline
`containerImageTag`	Version of the image tag used in the pipeline
`libCodeRepositoryName`	Repository of the LIB-CODE
`infraDevRepositoryName`	Repository of the SVC-DEV
`sitespecific`	Site-specific file name
`siteSpecificSourceFolder`	Folder in INFRA-DEV where `sitespecific` is located.
`cnfType`	smf/upf
`versionspecific`	Name of the version-specific file
`versionSpecificSourceFolder`	Directory path to version-specific file
`macrovariablesSourceFolder`	Directory path to macro variable file
`macrovariables`	Name of the macro variable file

Configuring SUDO privileges

Several PADS pipelines allow SSH access to virtual machines and permit non-root users to execute commands. To ensure successful execution, perform the following actions prior to running PADS pipelines.

- Create a non-root user on the target VMs.
- Configure an SSH public key for this non-root user to enable SSH key-based authentication.
- Grant the user SUDO permissions to execute commands that require privileges.
- Configure the username parameters defined in the following pipelines for the non root user.
- Ensure that SSH private key secret names and key vault references are appropriately set for the non-root user.

ACR to OPM Image Copy Pipeline

This pipeline consumes the BOM files containing links to artifacts that point to a source ACR. It then maps the path from the BOM files to the applicable PRD files, and pushes the images and charts to the OPM software. It performs a pre-check to ensure that the images and charts to be uploaded have not already been added to OPM previously before copying the artifacts. Upon completion, it also verifies that the images and charts have been successfully downloaded to OPM.

> ***Note:*** *UnityCloud third party images, BOMs, and PRD files are delivered as part of the PADS TGZ file deliverable.*

In addition, Helm charts are copied from an ACR to the OPM chart museum as part of an ACR to OPM Image Copy pipeline. Links to helm charts in an ACR (OCI storage) are taken from the PRD files, which also contain the Docker Image links to an ACR. Confirm that the chart links in **`charts.<helm-chart>.registry`** have the OCI storage path. For example:

`oci://a4opadsacr.azurecr.io/artifactory/cnhelm/fed-opm/fed- opm-3.4.0-419-main-int.tgz`

Upgrade OPM’s Helm Version

If the OPM's helm version is less than 3.8.0, upgrade to the latest version using the steps below:

1. SSH to the OPM VM

> ***Note:*** *The OPM VM needs the helm version to be at least version 3.8.*

2. Download the 3.11 helm version using the following command:

```
wget https://get.helm.sh/helm-v3.11.0-linux-amd64.tar.gz
```

3. Extract the helm CLI and update the existing helm CLI.

   - `tar -xf helm-v3.11.0-linux-amd64.tar.gz`
   - `cd linux-amd64/`
   - `mv /usr/local/bin/helm /usr/local/bin/helm.old`
   - `mv /usr/local/bin/helm3 /usr/local/bin/helm3.old`
   - `cp helm /usr/local/bin/helm`
   - `cp helm /usr/local/bin/helm3`

4. Verify the helm version using the following command:

```
helm version
```

Adjust the Replica Count in OPM

Prior to running the pipeline, perform the following steps to adjust the replica count in OPM:

1. Scale down the chartm and dreg pods to 1 and wait for the other pods to terminate.

```
kc scale -n vnfmgr statefulset vnfmgr-chartm --replicas 1
```

```
kc scale -n vnfmgr statefulset vnfmgr-dreg --replicas 1
```

2. Run the ACRTOOPMCopy pipeline to upload the images and charts to OPM.

3. Copy the dreg data to the other worker nodes:

   a. SSH to the node where dreg is running. Run the below command to get the node details.

```
kc get po -n vnfmgr -l app=vnfmgr-dreg -o custom-
columns=NODE:spec.nodeName,IP:status.hostIP
```

   b. Get the node details to SCP the data:

```
kc get node --selector='!node-role.kubernetes.io/master' -o wide | grep -v
$(kc get po -n vnfmgr -l app=vnfmgr-dreg -o
jsonpath='{.items[0].spec.nodeName}')
```

c. SCP the data to all nodes (from the previous command) IPs using the following command:

```
scp -r /var/storage/vnfmgr/dreg root@<NODE- IP>:/var/storage/vnfmgr/
```

4. Copy the chartm data to the other worker nodes:

a. SSH to the node where chartm is running. Run the following command to get the node details:

```
kc get po -n vnfmgr -l app=vnfmgr-chartm -o custom-
columns=NODE:spec.nodeName,IP:status.hostIP
```

b. Get the node details to SCP:

```
kc get node --selector='!node-role.kubernetes.io/master' -o wide | grep -v
$(kc get po -n vnfmgr -l app=vnfmgr-chartm -o
jsonpath='{.items[0].spec.nodeName}')
```

c. SCP the data to all the node (from the above command) IPs using the below command

```
scp -r /var/storage/vnfmgr/chartm root@<NODE- IP>:/var/storage/vnfmgr/
```

5. Scale up the chartm and dreg pods to 3 and wait for the other pods to come up:

```
kc scale -n vnfmgr statefulset vnfmgr-chartm --replicas 3
```

```
kc scale -n vnfmgr statefulset vnfmgr-dreg --replicas 3
```

Configure the following files in the ACR to OPM Image Copy Pipeline and the corresponding repository (INFRA-DEV, LIB-CODE, or SVC-DEV):

- affirmed-nr-acr-to-opm-image-copy-macro-pipeline.yml (INFRA-DEV)
- code/acr_opm_image_copy/affirmed-nr-acrToOpmCopy.json (LIB-CODE)

affirmed-nr-acr-to-opm-image-copy-macro-pipeline.yml (INFRA-DEV)

The following table shows the variables used in the ACR to OPM Image Copy macro pipeline.

Table 3. affirmed-nr-acr-to-opm-image-copy-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool

Variables	Description
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- acrToOpmImageCopyTimeout`
`- template`
	The following are ACR to OPM variables:
	- Azure subscription name
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path
	- Timeout in minutes for the pipeline
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path.

`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter zoneName
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true.
`zoneName`	Parameter to specify zone name
`azureRegion`	Parameter for the Azure region the Pipeline runs on

code/acr_opm_image_copy/affirmed-nr-acrToOpmCopy.json (LIB-CODE)

The following table shows the variables used in the ACR to OPM Image Copy Input JSON pipeline.

Table 4. affirmed-nr-acrToOpmCopy.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with opm.unitycloud.com as the FQDN value and the Public certificate of multi-node OPM containing *.unitycloud.com, please refer to the UnityCloud documentation set.*

Variables	Description
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`opm.ipAddress`	IP address of OPM
`opm.sshUsername`	SSH username used to communicate with OPM
`opm.dockerRepo`	Docker repository on OPM. For example: `<ipaddress>:<port>`
`opm.helmUrl`	Destination of the helm url for `cna_ deploy_inventory` script. Enter the OPM IP address.
`opm.dockerYamlPath`	Docker yaml path for the `cna_deploy_ inventory` script. Enter the `<opm-ip- address>:<port>` information.
`opm.pathToCopyPrds`	File system path on OPM where PRD files will be copied to
`acrEndpoint`	ACR endpoint
`helmChartsIgnore`	Set this variable to **True** if helm charts don't need to be imported. Otherwise, set to **False**.
`prdLocation`	Directory Path (location) to PRD files in LIB- CODE repository. Note: Remove the / character at the end.
`prdFiles`	PRD files at "prdLocation" to be processed
`additionalImages`	List of additional images to be copied to OPM docker repository
`cna_deploy_inventoryLocation`	Directory Path where `cna_deploy_ inventory` is stored. **Note:** Remove the / character at the end.
`cna_deploy_inventoryPathToCopy`	File path on OPM to which cna_deploy_ inventory script must be copied. **Note:** Remove the / character at the end.
`opm.ipAddress`	OPM IP address
`opm.sshUsername`	SSH username used to communicate with OPM
`opm.dockerRepo`	Docker repository on OPM. For example: `<ipaddress>:<port>`
`opm.helmUrl`	Destination helm URL for cna_deploy_ inventory script. For now, just give OPM IP address
`opm.dockerYamlPath`	Docker YAML path for cna_deploy_ inventory script. For now, just give `<opm- ipaddress>:<port>`

Variables	Description
`opm.pathToCopyHelmCharts`	File path on OPM where helm charts are copied before they are pushed to the OPM chart museum ***Note:*** *Do not use the /filestore/chartm- storage directory as the* `opm.pathToCopyHelmCharts` *location.*
`opm.pathToCopyPrds`	File system path on OPM to where PRD files will be copied
`opm.username`	Web username for OPM login
`acrEndpoint`	ACR endpoint
`keyVault.name`	Azure Key Vault name
`keyVault.opmPasswordSecretName`	Azure Key-Vault secret name which contains the password for OPM login
`keyVault.opmPrivateKeysecretName`	OPM secret name on the Azure Key-Vault for the private SSH key
`keyVault.opmPublicServiceName`	OPM VM's SSHpublic key secret name from the Azure Key-Vault
`helmChartsIgnore`	True if helm charts don't need to be imported. False otherwise
`prdLocation`	Directory Path (location) to PRD files in LIB- CODE repository. Remove / at the end of the file path
`prdFiles`	PRD files at "prdLocation" that must be processed
`additionalImages`	List of additional images that must be copied to OPM docker repository
`cna_deploy_inventoryLocation`	Directory Path where cna_deploy_inventory is stored. Remove / at the end of the file path
`cna_deploy_inventoryPathToCopy`	File path on OPM to which cna_deploy_ inventory script must be copied. Remove / at the end of the file path
`bomLocation`	Directory path to the folder with the BOM files. Remove / at the end of the file path
`mappingBomPrdFiles`	Key value pairs of BOM file names and PRD file names. The Key is the BOM file name and the Value is the corresponding PRD File name
`mappingBomPrd`	Used to map BOM and PRD files. The default value is true. Set this to false is the PRD files are to be used directly.
`opmDockerPorts`	Multi-node OPM docker port to copy the images from ACR.
`opmDockerCharts`	Multi-node OPM chart path where the charts are copied from the ACR.

OPM Docker Ports field

The ACR to OPM Image Copy Input JSON pipeline uses the `opmDockerPorts` and `opmDockerCharts` values to create a symlink to the **opm.unitycloud.com:5005** folder by issuing the following command to copy the images and charts into the created folder, as shown here:

```
ln -s /etc/containerd/certs.d/<FQDN>:5005/
/etc/containerd/certs.d/<FQDN>:5006
```

```
ln -s /etc/containerd/certs.d/<FQDN>:5005/
/etc/containerd/certs.d/<FQDN>:5007

ln -s /etc/containerd/certs.d/<FQDN>:5005/
/etc/containerd/certs.d/<FQDN>:5008
```

> ***Note:*** *Prior to running the ACR to OPM Image pipeline, please refer to the section of the OPM Cluster Readiness MOP that refers to the ACR to OPM pipeline. In this step, the replica count for vnfmgr-chartm and vnfmgr-dreg is reduced to 1 prior to running the pipeline. Once the ACR to OPM pipeline is complete, files are manually copied from one worker node to other worker nodes and the replica count is reverted. This is a temporary fix on multi-node OPM.*

UCReg Image Copy Pipelines

These pipelines are responsible for populating Unitycloud Infra Docker Registry (Infra-Dreg) with images. There are two pipelines, one for ACR to MNOPM Infra-Dreg copy and the other for MNOPM Infra-Dreg to NF Infra-Dreg copy. The ACR-to-UCReg copy pipeline is similar to ACR to OPM copy except it copies to the new Infra-Dreg on the MNOPM. Note that the local chart museum on OPM still holds the Helm charts.

The pipelines to populate Infra-Dreg are run in phases with the first run done after the MN-OPM has been deployed. Once deployed, the images (and charts) are copied to MNOPM Infra-Dreg. The MNOPM Infra-Dreg to NF Infra-Dreg pipeline is run after the NF clusters (SMF or UPF) have been deployed (e.g., once the Infra-Dreg is running on the clusters).

The pipelines consume the BOM files containing links to artifacts that point to a source ACR as well as destination Infra-Dreg. It then maps the path from the BOM files to the applicable PRD files and pushes the images and charts to the MNOPM. It performs a pre-check to ensure that the images and charts to be uploaded have not already been added to OPM previously before copying the artifacts. Upon completion, it also verifies that the images and charts have been successfully downloaded to OPM. The MNOPM Infra-Dreg to NF Infra-Dreg copy pipelines use the same input BOM/PRD files and additionally include the source Infra-Dreg (MNOPM Infra-Dreg) and destination Infra-Dreg (NF Infra-Dreg).

affirmed-nr-acr-to-ucreg-image-copy-macro-pipeline.yml (INFRA-DEV)

The following table shows the variables used in the ACR to MNOPM-UCReg Image Copy macro pipeline.

Table 5. affirmed-nr-acr-to-ucreg-image-copy-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- acrToOpmImageCopyTimeout`
`- template`
	The following are ACR to OPM variables:
	- Azure subscription name
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path
	- Timeout in minutes for the pipeline
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path.

`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter zoneName
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true.
`zoneName`	Parameter to specify zone name
`azureRegion`	Parameter for the Azure region the Pipeline runs on

code/ucreg_image_copy/affirmed-nr-acrToMnOpmUcRegCopy.json (LIB-CODE)

The following table shows the variables used in the ACR to MNOPM Infra-Dreg Image Copy Input JSON pipeline.

Table 6. affirmed-nr-acrToMnOpmUcRegCopy.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type (only multiNode is supported): `multiNode`
`opmUcReg.svcFqdn`	Specifies the FQDN of the MNOP Infra-Dreg where ACR images will be copied.
`opmUcReg.svcIp`	Specifies the IP address of the MNOPM Infra-Dreg service.

Variables	Description
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. With this pipeline, this param is only a reference to the opm chart museum. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with opm.unitycloud.com as the FQDN value and the Public certificate of multi-node OPM containing *.unitycloud.com, please refer to the UnityCloud documentation set.*
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`opm.ipAddress`	IP address of fed-opm_multi ingress service
`opm.sshUsername`	SSH username used to communicate with OPM VM
`opm.helmUrl`	Destination of the helm url for `cna_ deploy_inventory` script. Enter opm.unitycloud.com/chartm/.
`opm.pathToCopyPrds`	File system path on OPM where PRD files will be copied to
`opm.pathToCopyHelmCharts`	File path on OPM where helm charts are copied before they are pushed to the OPM chart museum ***Note:*** *Do not use the /filestore/chartm- storage directory as the* `opm.pathToCopyHelmCharts` *location.*
`opm.username`	Web username for OPM login
`acrEndpoint`	ACR access URL (including optional image path)
`acrRepo`	ACR URL
`keyVault.name`	Azure Key Vault name
`keyVault.opmPrivateKeysecretName`	OPM secret name for the private SSH key on the Azure Key-Vault
`keyVault.opmPrivateKeysecretName`	OPM secret name on the Azure Key-Vault for the private SSH key.
`keyVault.opmPublicKeysecretName`	OPM secret name on the Azure Key-Vault for the public SSH key.
`helmChartsIgnore`	Set this variable to **True** if helm charts don’t need to be imported. Otherwise, set to **False**.
`prdLocation`	Directory Path (location) to PRD files in LIB- CODE repository. Note: Remove the / character at the end.
`prdFiles`	PRD files at “prdLocation” to be processed
`additionalImages`	List of additional images to be copied to OPM docker repository

Variables	Description
`cna_deploy_inventoryLocation`	Directory Path where `cna_deploy_ inventory` is stored. **Note:** Remove the / character at the end.
`cna_deploy_inventoryPathToCopy`	File path on OPM to which cna_deploy_ inventory script must be copied. **Note:** Remove the / character at the end.
`bomLocation`	Directory path to the folder with the BOM files. Remove / at the end of the file path
`mappingBomPrdFiles`	Key value pairs of BOM file names and PRD file names. The Key is the BOM file name and the Value is the corresponding PRD File name
`mappingBomPrd`	Used to map BOM and PRD files. The default value is true. Set this to false is the PRD files are to be used directly.
`opmDockerCharts`	Multi-node OPM chart path where the charts are copied from the ACR.

> ***Note:*** *Prior to running the ACR to UCReg Image pipeline, please refer to the section of the OPM Cluster Readiness MOP that refers to the ACR to OPM pipeline. In this step, the replica count for vnfmgr-chartm and vnfmgr-dreg is reduced to 1 prior to running the pipeline. Once the ACR to OPM pipeline is complete, files are manually copied from one worker node to other worker nodes and the replica count is reverted. This is a temporary fix on multi-node OPM.*

affirmed-nr-ucreg-to-ucreg-image-copy-macro-pipeline.yml (INFRA-DEV)

The following table shows the variables used in the MNOPM-UCReg to NF-UCReg Image Copy macro pipeline.

Table 7. affirmed-nr-ucreg-to-ucreg-image-copy-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

Variables	Description

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- acrToOpmImageCopyTimeout`
`- template`
	The following are ACR to OPM variables:
	- Azure subscription name
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path
	- Timeout in minutes for the pipeline
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path.

`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter zoneName
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true.
`zoneName`	Parameter to specify zone name
`azureRegion`	Parameter for the Azure region the Pipeline runs on

code/ucreg_image_copy/affirmed-nr-mnOpmUcRegToCnfUcRegCopy.json (LIB-CODE)

The following table shows the variables used in the MNOPM Infra-Dreg to NF Infra-Dreg Image Copy Input JSON pipeline.

Table 8. affirmed-nr-acrToMnOpmUcRegCopy.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type (only multiNode is supported): `multiNode`
`opmUcReg.svcFqdn`	Specifies the FQDN of the MNOP Infra-Dreg where ACR images will be copied.
`opmUcReg.svcIp`	Specifies the IP address of the MNOPM Infra-Dreg service.
`nfUcReg.svcFqdn`	Specifies the FQDN of the NF Infra-Dreg where MNOPM Infra-Dreg images will be copied.
`nfUcReg.svcIp`	Specifies the IP address of the NF Infra-Dreg service.

Variables	Description
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. With this pipeline, this param is only a reference to the opm chart museum. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with opm.unitycloud.com as the FQDN value and the Public certificate of multi-node OPM containing *.unitycloud.com, please refer to the UnityCloud documentation set.*
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`opm.ipAddress`	IP address of fed-opm_multi ingress service
`opm.sshUsername`	SSH username used to communicate with OPM VM
`opm.helmUrl`	Destination of the helm url for `cna_ deploy_inventory` script. Enter opm.unitycloud.com/chartm/.
`opm.pathToCopyPrds`	File system path on OPM where PRD files will be copied to
`keyVault.name`	Azure Key Vault name
`keyVault.opmPrivateKeysecretName`	OPM secret name for the private SSH key on the Azure Key-Vault
`keyVault.opmPublicKeysecretName`	OPM secret name on the Azure Key-Vault for the public SSH key.
`keyVault.opmPasswordSecretName`	Azure Key-Vault secret name which contains the password for OPM login
`helmChartsIgnore`	Set this variable to **True** if helm charts don't need to be imported. Otherwise, set to **False**.
`prdLocation`	Directory Path (location) to PRD files in LIB- CODE repository. Note: Remove the / character at the end.
`prdFiles`	PRD files at "prdLocation" to be processed
`additionalImages`	List of additional images to be copied to OPM docker repository
`cna_deploy_inventoryLocation`	Directory Path where `cna_deploy_ inventory` is stored. **Note:** Remove the / character at the end.
`cna_deploy_inventoryPathToCopy`	File path on OPM to which cna_deploy_ inventory script must be copied. **Note:** Remove the / character at the end.

Variables	Description
`opm.pathToCopyHelmCharts`	File path on OPM where helm charts are copied before they are pushed to the OPM chart museum ***Note:*** *Do not use the /filestore/chartm- storage directory as the* `opm.pathToCopyHelmCharts` *location.*
`opm.username`	Web username for OPM login
`opm.sshUsername`	SSH username to communicate to OPM
`bomLocation`	Directory path to the folder with the BOM files. Remove / at the end of the file path
`mappingBomPrdFiles`	Key value pairs of BOM file names and PRD file names. The Key is the BOM file name and the Value is the corresponding PRD File name
`mappingBomPrd`	Used to map BOM and PRD files. The default value is true. Set this to false is the PRD files are to be used directly.
`opmDockerCharts`	Multi-node OPM chart path where the charts are copied from the ACR.

CNF Delete Pipeline

This pipeline is used to delete the CNF (SMF/UPF) using the OPM. It deletes all the federations in the given K8s cluster and can also perform a cleanup process after the deletion of all the federations on the K8s cluster. The cleanup process is an automation of the cleanup MOP, which is done manually to clean `rook-ceph`.

The following sections show the configurable files in the CNF Delete Pipeline and their corresponding (repository):

- affirmed-nr-cnf-operations-delete-macro-pipeline.yml (SVC-DEV)
- affirmed-nr-cnf-operations-delete.json (LIB-CODE)

affirmed-nr-cnf-operations-delete-macro-pipeline.yml (SVC-DEV)

This pipeline calls OPM's NB API to delete all federations in the given cluster. The pipeline polls for all federations to be deleted until the timeout is reached. If all of the federations are deleted, the pipeline continues the `rook-ceph` cleanup process.

If all of the federations are not deleted, and some pods are stuck in the **Terminating** state, then the pipeline exits out after a timeout. You must then manually intervene to determine why the pods have not been deleted and clean them up using **`kubectl delete pod comma -n fed-upf --force --all`**.

The following table shows the variables used in the CNF Delete macro pipeline.

Table 9. affirmed-nr-cnf-operations-delete-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`zoneName`	Name of zone used to name the pipeline run
`cleanup`	Cleanup process after CNF Delete. Default is true

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- finalValidationDeleteTimeout`
`- finalValidationTimeout`
`- cnfDeleteCleanup`
`- cnfOperation`
`- template`
	The following are CNF variables:
	- Azure subscription name
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
	- Polling timeout in seconds for checking pods and services status for each fed. Values tested successfully with: 10.
	- Maximum timeout in minutes until final polling happens for all pods and services. After this, the pipeline will exit with and error code if all pods and services are not running or complete. Values tested successfully with: 10.
	- Clean up process after delete CNF, such as cleaning rook- ceph. Value is True or False
	- Operation of CNF. Leave this as 'delete'
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path.

`stages.template`	Path to micro pipeline with the corresponding ADO repository

affirmed-nr-cnf-operations-delete.json (LIB-CODE)

The following table shows the variables used in the CNF Delete JSON pipeline.

Table 10. affirmed-nr-cnf-operations-delete.json Variables

Variables	Description
opmMode	Specifies the OPM deployment type: singleNode or multiNode
multiNodeOPM.FQDN	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with **opm.unitycloud.com** as the FQDN value and the Public certificate of multi-node OPM containing ***.unitycloud.com**, please refer to the UnityCloud documentation set.*
multiNodeOPM.domainName	Domain name of the OPM FQDN
multiNodeOPM.opmMasterVMIP	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
multiNodeOPM.opmMasterVMPrivateKeySecretName	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
acrInfo.name	Name of ACR.
acrInfo.storageFormat	Storage format of ACR. If not oci , leave it as empty string.
prdLoctaion	Absolute path of directory in LIB-CODE where PRD files are in LIB-CODE repository. Remove / at the end.
helmValuesLocation	Absolute path of directory where Helm Values files are in LIB-CODE repository. Remove / at the end.
dockerRepo	Docker repository address to pull images from. <ip-address>:<port>
extensionPrdFolder	Directory Path(location) to extension PRD files in LIB-CODE repository. Remove / at the end. ***Note:*** *Recommended to keep this folder inside the appropriate UC version PRDs folder. This is because the start/stop order of the extension PRD file might change with UC versions.*
extensionPrdFile	Extension PRD file to use the start/stop order for CNF deployment.
nfType	Type of CNF being deployed (SMF/UPF)
kubeconfig.reference.keyVault.id	Name of the Azure Key-Vault
kubeconfig.reference.keyVault.secretName	Name of the secret in Azure Key-Vault with value as kubeconfig of the K8s Cluster <cluster-name>-kubeconfig
opm.ipAddress	OPM IP address

Variables	Description
`opm.username`	Username used to authenticate to OPM application. ***Note:*** *You can use this to SSH into the attaching K8 Cluster VMs to complete the attach cluster operation.*
`opm.keyVault.opmPasswordSecretName`	Secret name in Azure Key-Vault of opm.username
`snapshotFolder`	File path in LIB-CODE to snapshot folder
`snapshotFile`	Name of template snapshot file in LIB- CODE's `snapshotFolder`
`snapshotVariableValuesFile`	Name of snapshot variables/values file in snapshotFolder
`chartRepo`	Name of Chart Repository created on OPM. This is used by OPM to do Bulk Install using your snapshot
`dzName`	Name of Deployment Zone to which the K8s Cluster is attached
`clusterId`	ID of the attached K8s Cluster from OPM. Retrieve this ID from the FinalizeK8sCluster Pipeline, Attach K8s Cluster stage. User can get this from the FinalizeK8sCluster Pipeline Attach K8s Cluster stage
`cephDiskType`	Enter vdc for both SMF and UPF
`inputFinalizeK8ClusterFolder`	Folder in LIB-CODE that contains the finalizek8cluster.json file. Remove the / from the end of the file path.
`inputFinalizeK8ClusterFile`	Name of the finalizek8cluster.json file.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	MNOPM secret name on the Azure Key- Vault for the private SSH key.

> ***Note:*** *Scripts in the PADS docker image care used to clean up any rook-ceph related content once the CNF has been deleted.*

CNF Deploy Pipeline

The following sections show the configurable files in the CNF Deploy Pipeline and their corresponding repositories:

- affirmed-nr-cnf-operations-deploy-macro-pipeline.yml (SVC-DEV)
- code/cnf_operations/affirmed-nr-cnf-operations-deploy.json (LIB-CODE)
- code/cnf_operations/snapshots/snapshots_specific_values.json (LIB-CODE)

affirmed-nr-cnf-operations-deploy-macro-pipeline.yml (SVC-DEV)

The following table shows the variables used in the CNF Deploy macro pipeline.

Table 11. affirmed-nr-cnf-operations-deploy-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

Variables	Description
`variables` **`- azureSubscription`** **`- inputFileForCNFDeploy`** **`- inputSourceFolderForCNFDeploy`** **`- cnfDeployTimeout`** **`- finalValidationDeployTimeout`** **`- finalValidationTimeout`** ~~**`- zoneName`**~~ **`- template`** **`- fedDeployTimeout`** **`- inputFinalizeK8ClusterFolder`** **`- inputFinalizeK8ClusterFile`** **`- inputSourceFolderForFinalizeCluster`** **`- inputFileForFinalizeCluster`** **`- cnfDeployPreCheckTimeout`**	The following are CNF variables: - Azure subscription name - Input JSON file name - Folder in LIB-CODE where inputFile is. Remove / at the end of the file path. - Total timeout in minutes for the CNF pipeline to run. - Polling timeout in seconds for checking pods and services status for each fed. - Maximum timeout in minutes until final polling occurs for all pods and services. After this, the Pipeline will exit with an error code if all pods and services are not running or complete. - Name of Zone. Used to name the pipeline run. - The absolute path to the macrovariables.yml file in INFRA-DEV repository. When referring from another repository, add **@INFRA** at the end of the file path. - Polling interval (in seconds) for calling NB API to poll all federations status. Recommended value is 10 - Folder in LIB-CODE that contains the finalizek8cluster.json file. Remove the / from the end of the file path. - Name of the finalizek8cluster.json file. - Folder in LIB-CODE where inputFile for inputFileForFinalizeCluster - Input json file for finalizek8Cluster pipeline - Timeout for pre-check stage
`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter `zoneName`
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true
`zoneName`	Parameter to specify zone name
~~**`workerCount`**~~	Parameter for number of workers
`azureRegion`	Parameter for the Azure region on which the Pipeline runs
~~**`multiNodeOPM.opmMasterVMPrivateKeySecretName`**~~	MNOPM secret name on the Azure Key- Vault for the private SSH key.

code/cnf_operations/affirmed-nr-cnf-operations-deploy.json (LIB-CODE)

The following table shows the variables used in the CNF Deploy Input JSON pipeline.

Table 12. affirmed-nr-cnf-operation-deploy.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`useUcReg`	Specifies which Infra-Dreg to pull images from upon deployment “mnOpmUcReg” or “nfUcReg” or “none”
`opmUcReg.svcFqdn`	Specifies the FQDN of the MNOP Infra-Dreg where images will be pulled from on deploy.
`opmUcReg.svcIp`	Specifies the IP address of the MNOPM Infra-Dreg service.
`nfUcReg.svcFqdn`	Specifies the FQDN of the NF Infra-Dreg where images will be pulled from on deploy.
`nfUcReg.svcIp`	Specifies the IP address of the NF Infra-Dreg service.
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with* ***opm.unitycloud.com*** *as the FQDN value and the Public certificate of multi-node OPM containing* ****.unitycloud.com****, please refer to the UnityCloud documentation set.*
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.` `opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`nfType`	Type of CNF being deployed (smf/upf).
`kubeconfig.reference.keyVault.` `id`	Name of the Azure Key-Vault.
`kubeconfig.reference.keyVault.` `secretName`	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
`opm.ipAddress`	OPM IP address

Variables	Description
`opm.username`	Username used to authenticate to OPM application
`opm.sshUsername`	Username used to authenticate to OPM VMs via SSH
`opm.keyVault.name`	Key-Vault name
`opm.keyVault.opmPasswordSecretName`	Secret name in Azure Key-Vault of opm.username
`opm.keyVault.clusterIdSecretName`	Secret name in Azure Key-Vault of attached cluster ID
`opm.keyVault.opmPrivateKeysecretName`	Secret name in Azure Key-Vault of opm private key
`extensionPrdFolder`	LIB-CODE path to the extension pr file folder. Remove the "/" from the end.
`extensionPrdFile`	LIB-CODE path to the extension pr file.
`snapshotFile`	Name of template snapshot file in LIB- CODE's snapshotFolder
`snapshotVariableValuesFolder`	Folder where the snapshot specific file is stored.
`snapshotVariableValuesFile`	Name of snapshot variables/values file in snapshotFolder
`chartRepo`	Name of Chart Repository created on OPM. This is used by OPM to do Bulk Install using your snapshot
`dzName`	Name of Deployment Zone to which the K8s Cluster is attached
`clusterId`	ID of the attached K8s Cluster from OPM. Retrieve this ID from the FinalizeK8sCluster Pipeline, Attach K8s Cluster stage
`mechidPasswordSecretName`	Secret name in the AKV which contains the mechid password used to generate the cloud.conf
`secretValuesFile`	Name of file holding secret values - not used

code/cnf_operations/snapshots/snapshots_specific_values.json (LIB-CODE)

This file contains the values to the variables in the snapshot file used by the CNF Deploy Pipeline.

Use the following example to assign values to variables, such as `$ml_peer` and `$peer_as`:

```
{
"$ml_peer": "10.161.30.8",
"$peer_as": "64500"
}
```

Changes to be made in the SVC-DEV file to run Cinder CSI pipeline

Make changes in the SVC-DEV file in the location **SVC-DEV/pipeline/gmacro/ affirmed-nr-cnf-operations-deploy-macro-pipeline.yml**

Under the stages replace the file name in the template to point to the cinder csi micro pipeline

stages:

- template: pipeline/cnf_operations/affirmed-nr-cnf-operations-deploy-micro-pipeline-cinder-csi.yml@LIB-CODE

CNF Upgrade Pipeline

This pipeline is used to upgrade CNFs using the following high-level procedure:

- Save the Configuration
- Delete the Network Function
- Deploy the network function with the new version
- Restore the Configuration.

Note: *Both the Delete and Deploy pipelines are re-used.*

The following sections show the configurable files in the CNF Upgrade Pipeline and their corresponding repositories:

- affirmed-nr-cnf-operations-upgrade-macro-pipeline.yml (SVC-DEV)
- code/cnf_operations/affirmed-nr-cnf-operations-upgrade.json (LIB-CODE)
- CNF Upgrade Pipeline

affirmed-nr-cnf-operations-upgrade-macro-pipeline.yml (SVC-DEV)

The following table shows the variables used in the CNF Upgrade macro pipeline.

Table 13. affirmed-nr-cnf-operations-upgrade-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`zoneName`	Parameter to specify zone name
`cleanup`	Cleanup process after the CNF is deleted. The default is True.

`variables`
`- template`
`- inputFile`
`- inputSourceFolder`
`- cnfOperation`
`- cnfDeploymentTimeout`
`- fedDeployTimeout`
`- finalValidationDeleteTimeout`
`- deleteFinalValidationTimeout`
`- deployFinalValidationTimeout`
`- stages.template`
	The following are CNF variables:
	- Template file containing the macrovariables. Contains the `azureServiceConnection macrovariable`.
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
	- Operation of CNF. Leave this variable as upgrade.
	- Maximum timeout in minutes until which final polling happens for all pods and services. After this time, the pipeline will exit with an error code if all pods and services are not running or complete. This has been tested with 120 minutes as the timeout.
	- Polling timeout in seconds for checking pods and services status for each fed. Values tested successfully with: 10 seconds
	- Polling interval in seconds for polling federations as part of the CNF Delete Pipeline.
	- Total time in minutes to delete all federations as part of the CNF Delete Pipeline.
	- Total time in minutes to deploy all federations as part of the CNF Delete Pipeline
	- Path to micro pipeline with the corresponding ADO repository.

code/cnf_operations/affirmed-nr-cnf-operations-upgrade.json (LIB-CODE)

The following table shows the variables used in the CNF Upgrade Input JSON pipeline.

Table 14. affirmed-nr-cnf-operation-upgrade.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with* ***opm.unitycloud.com*** *as the FQDN value and the Public certificate of multi-node OPM containing* ****.unitycloud.com****, please refer to the UnityCloud documentation set.*
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`nfType`	Type of CNF being deployed (smf/upf).
`kubeconfig.reference.keyVault.id`	Name of the Azure Key-Vault.
`kubeconfig.reference.keyVault.secretName`	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
`opm.ipAddress`	OPM IP address.
`opm.username`	Username used to authenticate to OPM application.
`opm.keyVault.opmPasswordSecretName`	Secret name in Azure Key-Vault of `opm.username`.
`snapshotFolder`	File path in LIB-CODE to snapshot folder.
`snapshotFile`	Name of template snapshot file in LIB- CODE's `snapshotFolder`.
`snapshotVariableValuesFile`	Name of snapshot variables/values file in snapshotFolder.
`chartRepo`	Name of Chart Repository created on OPM. This is used by OPM to do Bulk Install using your snapshot.
`dzName`	Name of Deployment Zone to which the K8s Cluster is attached.
`cephDiskType`	Enter vdc for both SMF and UPF

Variables	Description
`extensionPrdFolder`	LIB-CODE path to the extension pr file folder. Remove the "/" from the end.
`extensionPrdFile`	LIB-CODE path to the extension pr file.
`masterVmIpAddress`	IP address of a Master VM.
`cfgmgrUsername`	User name to log in to the cfgmgr of the CNF.
`cfgmgrPasswordSecretName`	Password secrete name in the Azure Key- Vault for the cfgmgr user name.
`masterVmIpCopyCfgmgrPath`	Path to which to copy the cfgmgr back up file.
`cfgmgrBackupFilename`	Name of the file to download the cfgmgr backup.
`inputFinalizeK8ClusterFolder`	Folder in LIB-CODE that contains the `finalizek8cluster.json` file. Remove the / from the end of the file path.
`inputFinalizeK8ClusterFile`	Name of the finalizek8cluster.json file.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	MNOPM secret name on the Azure Key- Vault for the private SSH key.
`masterVmIpCopyCfgmgrPath`	File path in master VM to place the backup of CNF cfgmgr. This should be a home folder of non-root user or any folder which has writing permission for non-root user. (i.e., /home/<non-root_username>)

CNF Operation Status Pipeline

The CNF Operation Status pipeline is used to check whether all containers are up after Day N configuration.

The following sections show the configurable files in the CNF Operation Status Pipeline and their corresponding repositories:

- code/cnf_operations/affirmed-nr-cnf-operations-status.yml (INFRA-DEV)
- code/cnf_operations/affirmed-nr-cnf-operations-status.json (LIB-CODE)
- CNF Operation Status Pipeline

code/cnf_operations/affirmed-nr-cnf-operations-status.yml (INFRA-DEV)

The following table shows the variables used in the Operation Status Pipeline macro pipeline.

Table 15. affirmed-nr-cnf-operation-status.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- inputFile`
`- inputSourceFolder`
`- template`
	The following are CNF variables:
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path

`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter `zoneName`
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true
`fedDeployTimeout`	Polling interval (in seconds) for calling NB API to poll all federations status. Recommended value is 10
`zoneName`	Parameter to specify zone name
`workerCount`	Parameter for number of workers
`azureRegion`	Parameter for the Azure region on which the Pipeline runs
`versionSpecificSourceFolder`	Directory path to version specific file
`versionspecific`	Name of the version specific file

code/cnf_operations/affirmed-nr-cnf-operations-status.json (LIB-CODE)

The following table shows the variables used in the CNF Operations Status JSON pipeline.

Table 16. affirmed-nr-cnf-operation-status.json Variables

Variables	Description
opmMode	Specifies the OPM deployment type: `singleNode or multiNode`
multiNodeOPM.FQDN	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with **opm.unitycloud.com** as the FQDN value and the Public certificate of multi-node OPM containing ***.unitycloud.com**, please refer to the UnityCloud documentation set.*
multiNodeOPM.domainName	Domain name of the OPM FQDN
multiNodeOPM.opmMasterVMIP	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
multiNodeOPM.opmMasterVMPrivateKeySecretName	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
nfType	Type of CNF being deployed (smf/upf).
kubeconfig.reference.keyVault.id	Name of the Azure Key-Vault.
kubeconfig.reference.keyVault.secretName	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
opm.ipAddress	OPM IP address
opm.username	Username used to authenticate to OPM application
opm.keyVault.opmPasswordSecretName	Secret name in Azure Key-Vault of opm.username
snapshotFolder	File path in LIB-CODE to snapshot folder
snapshotFile	Name of template snapshot file in LIB- CODE's `snapshotFolder`
snapshotVariableValuesFile	Name of snapshot variables/values file in snapshotFolder
chartRepo	Name of Chart Repository created on OPM. This is used by OPM to do Bulk Install using your snapshot
dzName	Name of Deployment Zone to which the K8s Cluster is attached
clusterId	ID of the attached K8s Cluster from OPM. Retrieve this ID from the FinalizeK8sCluster Pipeline, Attach K8s Cluster stage
inputFinalizeK8ClusterFolder	Folder in LIB-CODE that contains the **finalizek8cluster.json** file. Remove the / from the end of the file path.
inputFinalizeK8ClusterFile	Name of the finalizek8cluster.json file.

CNF Upgrade ISSU Pipeline

The following sections show the configurable files in the CNF Upgrade ISSU Pipeline and their corresponding repositories:

- affirmed-nr-cnf-upgrade-issu-macro-pipeline.yml (SVC-DEV)
- code/cnf_upgrade_issu/affirmed-nr-cnfUpgradeIssu.json (LIB_CODE)
- code/cnf_upgrade_issu/snapshots_specific_values.json (LIB_CODE)

affirmed-nr-cnf-upgrade-issu-macro-pipeline.yml (SVC-DEV)

The following table shows the variables used in the CNF Upgrade ISSU macro pipeline.

Table 17. affirmed-nr-cnf-upgrade-issu-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- cnfISSUPipelineTimeout`
`- finalValidationUpgradeTimeout`
`- finalValidationTimeout`
~~`- zoneName`~~
`- template`
	The following are CNF variables:
	- Azure subscription name
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
	- Total timeout in minutes for the CNF pipeline to run.
	- Polling timeout in seconds for checking pods and services status for each fed.
	- Maximum timeout in minutes until final polling occurs for all pods and services. After this, the Pipeline will exit with an error code if all pods and services are not running or complete.
	- Name of Zone. Used to name the pipeline run.

- fedUpgradeTimeout **- inputFinalizeK8ClusterFolder** **- inputFinalizeK8ClusterFile** **- inputSourceFolderForFinalizeCluster** **- inputFileForFinalizeCluster** **- cnfISSUPipelineTimeout** **- cnfOperation**	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path. - Polling interval (in seconds) for calling NB API to poll all federations status. Recommended value is 10 - Folder in LIB-CODE that contains the finalizek8cluster.json file. Remove the / from the end of the file path. - Name of the finalizek8cluster.json file. - Folder in LIB-CODE where inputFile for inputFileForFinalizeCluster - Input json file for finalizek8Cluster pipeline - Timeout for pre-check stage
stages.template	Path to micro pipeline with the corresponding ADO repository
name	Name of the Pipeline when it is run. Includes parameter `zoneName`
appendCommitMessageToRunName	Append the commit message to the build number. The default is true
zoneName	Parameter to specify zone name
~~**workerCount**~~	Parameter for number of workers
azureRegion	Parameter for the Azure region on which the Pipeline runs
~~**multiNodeOPM.opmMasterVMPrivateKeySecretName**~~	MNOPM secret name on the Azure Key- Vault for the private SSH key.

code/cnf_upgrade_issu/affirmed-nr-cnfUpgradeIssu.json (LIB-CODE)

The following table shows the variables used in the CNF Upgrade ISSU Input JSON pipeline.

Table 18. affirmed-nr-cnflssuUpgrade.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`useUcReg`	Specifies which Infra-Dreg to pull images from upon deployment “mnOpmUcReg” or “nfUcReg” or “none”
`opmUcReg.svcFqdn`	Specifies the FQDN of the MNOP Infra-Dreg where images will be pulled from on deploy.
`opmUcReg.svcIp`	Specifies the IP address of the MNOPM Infra-Dreg service.
`nfUcReg.svcFqdn`	Specifies the FQDN of the NF Infra-Dreg where images will be pulled from on deploy.
`nfUcReg.svcIp`	Specifies the IP address of the NF Infra-Dreg service.
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with* ***opm.unitycloud.com*** *as the FQDN value and the Public certificate of multi-node OPM containing* ****.unitycloud.com****, please refer to the UnityCloud documentation set.*
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.` `opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`nfType`	Type of CNF being deployed (smf/upf).
`kubeconfig.reference.keyVault.` `id`	Name of the Azure Key-Vault.
`kubeconfig.reference.keyVault.` `secretName`	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
`opm.ipAddress`	OPM IP address
`opm.username`	Username used to authenticate to OPM application
`opm.sshUsername`	Username used to authenticate to OPM VMs via SSH
`opm.keyVault.name`	Key-Vault name

opm.keyVault.opmPassword SecretName	Secret name in Azure Key-Vault of opm.username
opm.keyVault.clusterIdSecretName	Secret name in Azure Key-Vault of attached cluster ID
opm.keyVault.opmPrivateKeysecretName	Secret name in Azure Key-Vault of opm private key
extensionPrdFolder	LIB-CODE path to the extension pr file folder. Remove the "/" from the end.
extensionPrdFile	LIB-CODE path to the extension pr file.
snapshotFile	Name of template snapshot file in LIB- CODE's snapshotFolder
snapshotVariableValuesFolder	Folder where the snapshot specific file is stored.
snapshotVariableValuesFile	Name of snapshot variables/values file in snapshotFolder
chartRepo	Name of Chart Repository created on OPM. This is used by OPM to do Bulk Install using your snapshot
issu	Set ISSU flag in OPM API call. Always set to "true"
rollbackPolicy	Upgrade failure policy for ISSU [rollbackOnFailure | pauseOnFailure]
dzName	Name of Deployment Zone to which the K8s Cluster is attached
clusterId	ID of the attached K8s Cluster from OPM. Retrieve this ID from the FinalizeK8sCluster Pipeline, Attach K8s Cluster stage
mechidPasswordSecretName	Secret name in the AKV which contains the mechid password used to generate the cloud.conf
secretValuesFile	Name of file holding secret values - not used

code/cnf_upgrade_issu/snapshots/snapshots_specific_values.json (LIB-CODE)

This file contains the values to the variables in the snapshot file used by the CNF Deploy Pipeline.

Use the following example to assign values to variables, such as `$ml_peer` and `$peer_as`:

```
{

"$ml_peer": "10.161.30.8",

"$peer_as": "64500"

}
```

Finalize K8 Cluster Pipeline

The following sections show the configurable files in the Finalize K8 Cluster Pipeline and their corresponding (repository):

- affirmed-nr-finalizeK8sCluster-macro-pipeline.yaml (INFRA-DEV)
- pipeline/finalize_cluster/affirmed-nr-finalizeK8sCluster-micro-pipeline.yml (LIB-CODE)
- code/finalize_cluster/affirmed-nr-finalizeK8sCluster.json (LIB-CODE)

affirmed-nr-finalizeK8sCluster-macro-pipeline.yaml (INFRA-DEV)

The following table shows the variables used in the FinalizeK8sCluster macro pipeline.

Table 19. affirmed-nr-finalizeK8sCluster-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- template`
	The following are CNF variables:
	- Azure subscription name
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path

`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter `zoneName`
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true
`zoneName`	Parameter to specify zone name
`workerCount`	Parameter for number of workers
`azureRegion`	Parameter for the Azure region on which the Pipeline runs

Variables	Description
`forceLocalDNS`	Parameter to fix local DNS issue with VM images
`forcelocalDnsPods`	True/False parameter if fix for force local DNS pods must be applied after K8s Cluster creation
`installEctdCerts`	Optional; installs the etcd-ca certificates necessary for fed- kube-prom
`disableSshAccessGnovs`	Optional; disables SSH access via the Gn-OVS network
`isMultiNodeOPMClusterDeployment`	Set this parameter to **True** for a multi-node OPM cluster or **False** for a CNF cluster.

pipeline/finalize_cluster/affirmed-nr-finalizeK8sCluster-micro-pipeline.yml (LIB-CODE)

This pipeline automatically generates M2M public and private keys and adds them to all the VMs in the cluster through the M2MKeySetup stage. It generates the M2M key- value pair and uploads the private key to the VMs only during MultiNode OPM cluster deployment. The public key is always uploaded.

code/finalize_cluster/affirmed-nr-finalizeK8sCluster.json (LIB-CODE)

The following table shows the variables used in the FinalizeK8sCluster Input JSON pipeline.

Table 20. affirmed-nr-finalizeK8sCluster.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with **opm.unitycloud.com** as the FQDN value and the Public certificate of multi-node OPM containing ***.unitycloud.com**, please refer to the UnityCloud documentation set.*
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.

Variables	Description
`attachCluster.clusterIP`	NF cluster's master node VM IP address, used to attach a cluster to OPM.
`attachCluster.m2mkeyKuberadminSecretName`	Name of the secret stored in the KeyVault secret name. PADS generates this name to store the NF cluster's M2M private key for the kuberadmin user. No input is required.
`attachCluster.attachType`	Specifies whether the cluster is openstack (`openstack_ attach`) or baremetal (`baremetal_ attach`). For non- Azure clusters, set this value to `nonazure_ attach`.
`attachCluster.username`	Username OPM uses to SSH into the attaching K8 Cluster VMS to complete the attach cluster operation. You should use the root user name for this value.
`useUcReg`	Specifies which Infra-Dreg to pull images from upon deployment "mnOpmUcReg" or "none"
`opmUcReg.svcIp`	Specifies the IP address of the MNOPM Infra-Dreg service.
`clusterName`	Name of the K8 cluster used for kubeconfig
`clusterParameters.ingressVM.adminAccess`	Communication type to Cluster. Accept the default value, SSH
`ClusterParameters.ingressVM.publicIpAddress`	IP-address to communicate with the jump VM
`ClusterParameters.ingressVM.port`	SSH port number to communicate with the jump VM
`ClusterParameters.ingressVM.adminUsername`	SSH username to communicate with the jump VM. Accept the default value, root
`ClusterParameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used to get to the ingress-VM
`clusterParameters.ingressVM.AdminPrivateKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key is used to get to the location of the VM
`clusterParameters.tuning`	True if Cluster tuning needs to be done. False otherwise
`ClusterParameters.vmM2MKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used from the ingress- VM to the individual VMS that forms the K8s cluster
`ClusterParameters.vmM2MKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used from the ingress-VM to the individual VMs that form the K8s cluster is stored
`clusterParameters.vms[].ipAddress`	VM IP address (for dual network environments, use the ens4 addresses)
`clusterParameters.vms[].port`	VM SSH Port
`clusterParameters.vms[].type`	Type of Node (Master/Worker)

Variables	Description
`clusterParameters.vms[].adminUsername`	SSH username to communicate with VMs
`clusterParameters.vms[].dataplane`	Set to ***true*** if the node (VM) must have a label `nodetype=dataplane`, otherwise set this value to **`false`**
`opm.ipAddress`	OPM IP address
`opm.sshUsername`	SSH username to communicate to OPM
`opm.keyVault.name`	Azure Key-Vault name with OPM information
`opm.keyVault.opmPrivateKeysecretName`	OPM secret name on the Azure Key-Vault for the private SSH key
`opm.keyVault.opmPasswordSecretName`	Azure Key-Vault secret name for OPM GUI user's password
`opm.certPath`	Certificate path on OPM which copies to all VMs on K8s Cluster
`certCopyPathToAllVMs`	Path on each VM in cluster to where the certificate copies
`opm.username`	OPM GUI username
`opm.dzName`	Deployment Zone name on OPM to which the create K8s Cluster needs to be attached
`clusterParameters.ha_proxy_vip`	The HA VIP address of the Kubernetes cluster post deployment
`clusterParameters.k8_primary_master_ip`	IP address of the primary master node of the CNF Cluster
`clusterParameters.vms[].racklabel`	Following the CNF cluster creation, this value is used to label all nodes (rack and zone).
`isMultiNodeOPMClusterDeployment`	Set this value before running the macro pipeline. Set to **`True`** for multi-node OPM cluster or **`False`** for a CNF cluster.
`ClusterParameters.ingressVM.adminUsername`	SSH username to communicate with the jump VM.
`ClusterParameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used to get to the ingress-VM.
`clusterParameters.ingressVM.AdminPrivateKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used to get to the ingress-VM is kept.
`ClusterParameters.vmM2MKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used from the ingress- VM to the individual VMS that forms the K8s cluster.
`ClusterParameters.vmM2MKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used from the ingress-VM to the individual VMS that forms the K8s cluster is kept.
`clusterParameters.vms[].adminUsername`	SSH username to communicate with VMs.

Variables	Description
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	MNOPM secret name on the Azure Key-Vault for the private SSH key.

VM Reboot of an NF Cluster

If a VM reboot of the NF cluster is necessary, complete the following steps as a workaround:

1. Using OPM GUI, de-attach the CNF Cluster from the Deployment Zone.

2. Manually SSH into each Master VM of the CNF Cluster and update the server IP in the */home/kuberadmin/admin.conf* file to the HA VIP (**`clusterParameters.ha_proxy_ vip`** in the *finalizeK8Cluster.json* input file).

3. Re-run the FinalizeK8Cluster Pipeline with the "Post Test-02 K8s cluster" and "Attach the K8 Cluster to the OPM" stages.

OPM Automation Pipeline

This pipeline automates the following items:

Automated Item	Description
OPM SSL Certificate Creation	This updates the OPM SSL certificate to include the OPM IP Address and enables use of the NB API with the HTTPS protocol. **`updateOpmSslCertificate`** is the associated variable from the macro pipeline.
Update Runtime Deploy Timeout	This updates the OPM SSL certificate to include the OPM IP Address and enables use of the NB API with the HTTPS protocol. **`updateOpmSslCertificate`** is the associated variable from the macro pipeline.
Create Chart Repository on OPM	This creates a Chart Repository entry on the OPM which refers to the OPM Chart Museum. The chart repository entry is referred to for CNF Operations and creating the Deployment Zone. **`createChartRepository`** is the associated variable from the macro pipeline.
Create Deployment Zone on OPM	This creates a Deployment Zone on the OPM to which the K8s cluster—created using the pipeline—is attached to. **`createDeploymentZone`** is the associated variable from the macro pipeline.

The following sections show the configurable files in the OPM Automation Pipeline and their corresponding repositories:

- affirmed-nr-opm-automation-macro-pipeline.yml (INFRA-DEV)

- affirmed-nr-opmAutomation.json (LIB-CODE)

affirmed-nr-opm-automation-macro-pipeline.yml (INFRA-DEV)

The following table shows the variables used in the OPM Automation macro pipeline.

Table 21. affirmed-nr-opm-automation-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- azureSubscription`
`- inputFile`
`- inputSourceFolder`
`- updateOpmSslCertificate`
`- createChartRepository`
`- createDeploymentZone`
`- updateRuntimeDeploytimeout`
`- template`
	The following are CNF variables:
	- Azure subscription name.
	- Input JSON file name.
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
	- True if the OPM SSL certificate needs to be updated with an IP address. Note that for multi-node OPM, this variable has no-op. The multi-node OPM updates the OPM SSL Certs, and no action is required by PADS.
	- True to create a chart repository on the OPM used for CNF Operations.
	- True to create a Deployment Zone used to attach the K8s cluster from the FinalizeK8sCluster Pipeline.
	- True to update runtime deploy timeout on the OPM.
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path

`stages.template`	Path to micro pipeline with the corresponding ADO repository

affirmed-nr-opmAutomation.json (LIB-CODE)

The following table shows the variables used in the OPM Automation Input JSON pipeline.

Table 22. affirmed-nr-finalizeK8sCluster.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with* ***opm.unitycloud.com*** *as the FQDN value and the Public certificate of multi-node OPM containing* ****.unitycloud.com****, please refer to the UnityCloud documentation set.*
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`opm.ipAddress`	OPM IP address
`opm.sshUsername`	SSH username to communicate to OPM
`opm.username`	OPM NB API username
`opm.keyVault.name`	Azure Key-Vault name with OPM information
`opm.keyVault.opmPrivateKeysecretName`	OPM secret name on the Azure Key-Vault for the private SSH key
`opm.keyVault.opmPasswordSecretName`	OPM user password secret name on the Azure Key-Vault
`opm.keyVault.clusterIdSecretName`	Cluster ID secret name on the Azure Key- Vault
`opm.sslCertSecretName`	Kubernetes secret name used on the OPM to store a new SSL certificate for an OPM with an IP address
`opm.chartRepository.name`	Name of chart repository to create on the OPM. This is used as reference in creating the Deployment Zone, as well
`opm.chartRepository.protocol`	Protocol for the Chart Repository
`opm.chartRepository.host`	OPM Chart Museum IP address
`opm.chartRepository.port`	OPM Chart Museum port number
`opm.chartRepository.type`	Type of Chart Repository. Please use `ChartMuseum` when referring to the OPM local chart museum
`opm.chartRepository.path`	Leave an empty string when using the local chart museum
`opm.chartRepository.username`	Leave an empty string when using the local chart museum

Variables	Description
`opm.chartRepository.password`	Leave an empty string when using the local chart museum
`opm.deploymentZone.name`	Name of Deployment Zone to be created on the OPM
`opm.deploymentZone.type`	Type of Deployment Zone on the OPM. For a K8s cluster created using the FinalizeK8Cluster Pipeline, use `attach` as the value
`opm.deploymentZone.location`	Location of the Deployment Zone. May contain spaces and/or special characters.
`opm.updateRuntimeDeployTimeout.value`	Runtime deploy timeout on the OPM. Its default value is 1,200 seconds
`locationDetails.name`	Name of the location to be created
`locationDetails.latitude`	Latitude of the location to be created
`locationDetails.longtitude`	Longitude of the location to be created
`locationDetails.tag`	User-defined details of the location to be created
`opm.sshUsername`	SSH username to communicate to OPM.
`opm.keyVault.name`	Azure Key-Vault name with OPM information.
`opm.keyVault.opmPrivateKey secretName`	OPM secret name on the Azure Key-Vault for the private SSH key.
`multiNodeOPM.opmMasterVMPrivate KeySecretName`	MNOPM secret name on the Azure Key- Vault for the private SSH key.

CSO Software Install Pipeline

This pipeline allows loading of third-party CSO-mandated agent software.

> ***Note:*** *Affirmed has verified the CSO Software Install pipeline syntax, but not CSO software installation.*

The following sections show the configurable files in the CSO Software Install pipeline and their corresponding repositories:

- affirmed-nr-csoSoftwareInstall-macro-pipeline.yml (INFRA-DEV)
- path/folder/affirmed-nr-csoSoftwareInstall.json (LIB-CODE)

affirmed-nr-csoSoftwareInstall-macro-pipeline.yml (INFRA-DEV)

The following table shows the variables used in the CSO Software Install macro pipeline.

Table 23. affirmed-nr-csoSoftwareInstall-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories. repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR

`variables`
`- inputFile`
`- inputSourceFolder`
`- sitespecific`
`- siteSpecificSourceFolder`
`- spID`
`- spPassword`
`- template`
	The following are CNF variables:
	- Input JSON file name
	- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path
	- Site specific file name
	- Folder in INFRA-DEV containing the `sitespecific` variable
	- Azure Service Principal ID
	- Protected Azure Service Principal Password
	- The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path

`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter `zoneName`.
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true
`zoneName`	Parameter to specify zone name
`workerCount`	Parameter for number of workers
`azureRegion`	Parameter for the Azure region on which the Pipeline runs
`forceLocalDNS`	Parameter to fix local DNS issue with VM images
`versionSpecificSourceFolder`	Directory path to version specific file
`versionspecific`	Name of the version specific file
`macrovariableSourceFolder`	Directory path to the macrovariable.yml file
`macrovariables`	File name of the macrovariable.yml file

path/folder/affirmed-nr-csoSoftwareInstall.json (LIB-CODE)

The following table shows the variables used in the CSO Software Install Input JSON pipeline.

Table 24. affirmed-nr-csoSoftwareInstall.json Variables

Variables	Description
`kubeconfig.reference.keyVault.id`	Name of the Azure Key-Vault
`kubeconfig.reference.keyVault.secretName`	Name of the secret in Azure Key-Vault with value as kubeconfig of the K8s Cluster <cluster-name>- kubeconfig
`repo`	Path to Chart
`version`	Version of the agent to fetch. Note that this may change per CSO needs.
`credentialUser`	The account UUID associated with the self- managed object
`imageScanEnabled`	Astra Enablement flag
`namespace`	Namespace to deploy agents/pods into
`proxy`	Instructs the agent to do all egress communication of findings via the Application Prox
`credentialSecrentName`	Name of the secret in Azure Key-Vault with value as the password of credential for CSO Software Install
`clusterIdName`	Name of the secret in Azure Key-Vault with value as the Cluster ID for CSO Software Install is stored

Pipeline Locking Pipeline

As part of ADO Pipeline locking, PADS creates a secret in key-vault with name as **`<clusterName>-lock`** and sets the pipeline name in it. For example:

```
{
"timestamp": "04/03/23" pipeline": ""
}
```

To create or update the lock for the cluster when used in a pipeline, you must provide the following values for the following variables in Pipeline Locking Pipeline.

- **`spID`**
- **`spPassword`**
- **`spTenant`**

To run the Pipeline Locking Check and Reset pipeline, `use pipelineLocking-macro-pipeline.yml` in the INFRA-DEV repository. PADS reads the Cluster Name from the `clusterName` field in `finalizeK8sCluster.json` in LIB-CODE

- The Check Current Lock stage tells the user which pipeline has a lock for the cluster.
- The Reset Current Lock stage can reset the lock for the cluster. You must enable this option (or set to **`true`**) when running the pipeline.

The following section shows the configurable files in the Pipeline Locking Pipeline and its corresponding repository:

affirmed-nr-pipelineLocking-macro-pipeline.yml (INFRA-DEV)

The following table shows the variables used in the Pipeline locking macro pipeline.

Table 25. affirmed-nr-pipelineLocking-macro-pipeline.yml (INFRA-DEV)

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`variables` **`- inputFile`** **`- inputSourceFolder`** **`- sitespecific`** **`- siteSpecificSourceFolder`** **`- spID`** **`- spPassword`** **`- spTenant`** **`- template`**	The following are CNF variables: - Input JSON file name - Folder in LIB-CODE where inputFile is. Remove / at the end of the file path - Site specific file name - Folder in INFRA-DEV containing the `sitespecific` variable - Azure Service Principal ID - Protected Azure Service Principal Password - Protected Azure Service Principal Tenant ID - The absolute path to the macrovariables.yml file in INFRA- DEV repository. When referring from another repository, add **@INFRA** at the end of the file path
`stages.template`	Path to micro pipeline with the corresponding ADO repository
`name`	Name of the Pipeline when it is run. Includes parameter `zoneName`.
`appendCommitMessageToRunName`	Append the commit message to the build number. The default is true
`zoneName`	Parameter to specify zone name
`workerCount`	Parameter for number of workers
`azureRegion`	Parameter for the Azure region on which the Pipeline runs

Variables	Description
`forceLocalDNS`	Parameter to fix local DNS issue with VM images
`versionSpecificSourceFolder`	Directory path to version specific file
`versionspecific`	Name of the version specific file
`macrovariableSourceFolder`	Directory path to the macrovariable.yml file
`macrovariables`	File name of the macrovariable.yml file

Helm Deployment Pipeline

The Helm Deployment pipeline deploys the multi-node OPM using helm commands to the specified cluster. HashiCorp Vault integration is supported from UC 5.2 release onwards, please refer to the section “**HashiCorp Vault Integration with MNOPM”** to enabled HashiCorp support.

The following tables show the variables used in the Helm Deployment pipeline.

pipeline/gmacro/affirmed-nr-helm-deploy-macro-pipeline.yml (SVC-DEV)

Table 26. affirmed-nr-helm-deploy-macro-pipeline.yml (SVC-DEV)

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`zoneName`	Name of zine used to name the pipeline run

Variables	Description

`variables`
`- template`
`- template`
`- inputFile`
`- inputSourceFolder`
`- deployTimeout`
`- perFedMaxPollTimeout`
`- finalValidationDeployTimeout`
`- finalValidationTimeout`
`- stages.template`
	The following are CNF variables:
	- Path to macrovariables.yml file in INFRA-DEV
	- Path to nf-variables-templates.yml file in INFRA-DEV
	- Input JSON file name
	- Folder in LIB-CODE where `inputFile` is. Remove / at the end of the file path.
	- Time for the pipeline to run.
	- Maximum time for polling the deployed fed.
	- Maximum time for validating the deployed fed.
	▪- Maximum timeout in minutes until final polling happens for all pods and services. After this, the pipeline will exit with and error code if all pods and services are not running or complete. Values tested successfully with: 10.
	- Path to the micro-pipeline with the corresponding ADO repository.

code/helm_deploy/affirmed-nr-helm_deploy.json (LIB-CODE)

Table 27. helm_deploy/affirmed-nr-helm_deploy.json (LIB-CODE)

Variables	Description
`kubeconfig.reference.keyVault.id`	Name of the Azure Key-Vault.
`kubeconfig.reference.keyVault.secretName`	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
`prdLocation`	The folder location where the PRD files to deploy multi-node OPM are present.
`helmValuesLocation`	The folder location where the helm values yaml file present for the feds to be deployed
`helmSpecificFolderLocation`	The location of the helm specific values file.
`helmSpecificFileName`	The file includes helm specific values.
`acrRepo`	The acr domain name where the images and charts are uploaded. (a4opadsacr.azurecr.io).
`ucregUrl`	The external load balancer IP address of the Infra-Dreg to be applied to the infra-dreg Kubernetes load balancer service.
`uregIp`	The Infra-Dreg URL applied to infra-dreg is at deployment.
`secretName`	The name used to pull the images and charts from the acr repository. (acr-secret)
`helmOperation`	“deploy” The helm operation to be executed [deploy|update|delete]

Variables	Description
`MongoInitUsername`	Secret name of mongo init username
`MongoInitPassword`	Secret name of mongo init password
`MongoUsername`	Secret name of mongo username
`MongoPassword`	Secret name of mongo password
`FedPaasHelpersTlskeySecretName`	Secret name of tls key certificate for the fed-pass_helpers.yml values file
`FedPaasHelpersTlsCrtSecretName`	Secret name of tls certificate for the fed-pass_helpers.yml values file
`clusterPrameters.ingressVM.publicIpAddress`	Load balancer IP address for MN-OPM k8s services
`clusterPrameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Key vault name where MN-OPM secrets are sored
`clusterPrameters.ingressVM.adminPrivateKey.reference.secretName`	MN-OPM M2M pivate key name
`clusterParameters.vms[].ipAddress`	MN-OPM VM IP address
`clusterParameters.vms[].port`	VM SSH Port
`clusterParameters.vms[].type`	Type of Node (Master/Worker)
`clusterParameters.vms[].adminUsername`	SSH username to communicate with MN-OPM VMs
`clusterParameters.vms[].dataplane`	Set to ***true*** if the node (VM) must have a label `nodetype=dataplane`, otherwise set this value to **`false`**

> ***Note:*** *The names of the values.yaml file, provided as part of the pipeline, should be named according to the start order. For example, if the value in the start order is fed-paas_helpers, the values.yaml file should also be fed-paas_helpers.yaml.*

> ***Note:*** *You must provide an image pull secret in each of the values.yaml file to pull images and charts. The path of the image pull secret is global.registry.docker.imagePullSecret.*

For fed-metallb and fed-opm_multi:

registry:

docker: <docker-repo-path>/<image-path>

imagePullSecret : <secretName>

For all other values.yaml files:

registry:

docker:<docker-repo-path>/<image-path> imagePullSecrets:

- name: <secretName>

where `<secretName>` is the name provided as part of the input.json file.

Fed-opm_multi must be deployed with infrDreg disabled to keep the legacy opm-dreg running during 5.1 to 5.2 NF upgrades. To enable opm-dreg, set the infrDreg.enabled flag to “False” in fed-opm_multi-values.yaml:

```
global:
  registry:
    docker:
      repoPath: tk5af.a4opacketcore.microsoft.com/rel_build_docker
    infraDreg:
      enabled: false
```

For additional information to include in your federation values.yaml file, please refer to the OPM documentation.

> ***Note:*** *You must provide the image path of the mongodb. This is uploaded ot the ACR in the fed-opm_multi values YAML file.*
>
> `pod-opm_master_rm: deployment:`
>
> `mongoServerImage: "<image path >".`

Helm Update Pipeline

The Helm Update pipeline updates the multi-node OPM to the current version using the helm update commands to the specified cluster. HashiCorp Vault integration is supported from UC 5.2 release onwards, please refer to the section “**HashiCorp Vault Integration with MNOPM”** to enabled HashiCorp support

The following tables show the variables used in the Helm Update pipeline.

pipeline/gmacro/affirmed-nr-helm-update-macro-pipeline.yml (SVC-DEV)

This input file contains variables used as input for the CNF ISSU Network Cloud macro pipeline. The variables are as follows.

Table 28. affirmed-nr-upgrade-issu-macro-pipeline.yml (SVC-DEV) Variables

Variables	Description
`cnfOperation`	Value set to upgrade for CNF ISSU.
`CnfISSUPipelineTimeout`	Micro-pipeline timeout .
`fedUpgradeTimeout`	Polling interval for the federations while the ISSU is running. This is used to retrieve the logs and status of the ISSU.
`finalValidationUpgradeTimeout`	Polling interval for the federation after the ISSU is completed; used to retrieve the status of the pods.

Table 29. affirmed-nr-helm-update-macro-pipeline.yml (SVC-DEV) Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`zoneName`	Name of zone used to name the pipeline run

Variables	Description
`variables` **`- template`** **`- template`** **`- inputFile`** **`- inputSourceFolder`** **`- deployTimeout`** **`- perFedMaxPollTimeout`** **`- finalValidationDeployTimeout`** **`- finalValidationTimeout`** **`- stages.template`**	The following are CNF variables: - Path to macrovariables.yml file in INFRA-DEV - Path to nf-variables-templates.yml file in INFRA-DEV - Input JSON file name ▪- Folder in LIB-CODE where inputFile is. Remove / at the end of the file path. - Time for the pipeline to run. - Maximum time for polling the deployed fed. - Maximum time for validating the deployed fed. ▪- Maximum timeout in minutes until final polling happens for all pods and services. After this, the pipeline will exit with and error code if all pods and services are not running or complete. - Path to the micro-pipeline with the corresponding ADO repository.

code/helm_operation/affirmed-nr-helm_update.json (LIB-CODE)

This input file contains variables used as input for the Helm Update Pipeline. The variables are as follows.

Table 30. affirmed-nr-helm_update.json (LIB-CODE) Variables

Variables	Description
`kubeconfig.reference.keyVault.id`	Name of the Azure Key-Vault.
`kubeconfig.reference.keyVault.secretName`	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
`prdLocation`	The folder location where the PRD files to deploy multi-node OPM are present.
`helmValuesLocation`	The folder location where the helm values yaml file present for the feds to be deployed
`helmSpecificFolderLocation`	The location of the helm specific values file.
`helmSpecificFileName`	The file includes helm specific values.
`acrRepo`	The acr domain name where the images and charts are uploaded. (a4opadsacr.azurecr.io).
`ucregUrl`	The external load balancer IP address of the Infra-Dreg to be applied to the infra-dreg Kubernetes load balancer service.
`uregIp`	The Infra-Dreg URL applied to infra-dreg is at deployment.
`secretName`	The name used to pull the images and charts from the acr repository. (acr-secret)
`helmOperation`	“upgrade” The helm operation to be executed [deploy|update|delete]

Variables	Description
`MongoInitUsername`	Secret name of mongo init username
`MongoInitPassword`	Secret name of mongo init password
`MongoUsername`	Secret name of mongo username
`MongoPassword`	Secret name of mongo password
`FedPaasHelpersTlskeySecretName`	Secret name of tls key certificate for the fed-pass_helpers.yml values file
`FedPaasHelpersTlsCrtSecretName`	Secret name of tls certificate for the fed-pass_helpers.yml values file
`clusterPrameters.ingressVM.publicIpAddress`	Load balancer IP address for MN-OPM k8s services
`clusterPrameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Key vault name where MN-OPM secrets are sored
`clusterPrameters.ingressVM.adminPrivateKey.reference.secretName`	MN-OPM M2M pivate key name
`clusterParameters.vms[].ipAddress`	MN-OPM VM IP address
`clusterParameters.vms[].port`	VM SSH Port
`clusterParameters.vms[].type`	Type of Node (Master/Worker)
`clusterParameters.vms[].adminUsername`	SSH username to communicate with MN-OPM VMs
`clusterParameters.vms[].dataplane`	Set to ***true*** if the node (VM) must have a label `nodetype=dataplane`, otherwise set this value to **`false`**

> ***Note:*** *The names of the values.yaml file, provided as part of the pipeline, should be named according to the start order. For example, if the value in the start order is fed-paas_helpers, the values.yaml file should also be fed-paas_helpers.yaml.*

> ***Note:*** *You must provide an image pull secret in each of the values.yaml file to pull images and charts. The path of the image pull secret is global.registry.docker.imagePullSecret.*

For fed-metallb and fed-opm_multi:

registry:

docker: **<docker-repo-path>/<image-path>**

imagePullSecret : <secretName>

For all other values.yaml files

registry:

docker: **<docker-repo-path>/<image-path>**

imagePullSecrets:

- name: **<secretName>**

where `<secretName>` is the name provided as part of the input.json file.

For additional information to include in your federation values.yaml file, please refer to the OPM documentation.

> ***Note:*** *You must provide the image path of the mongodb. This is uploaded to the ACR in the fed-opm_multi values YAML file.*
>
> `pod-opm_master_rm: deployment:`
>
> `mongoServerImage: "<image path >".`

Helm Delete Pipeline

The Helm Delete pipeline deletes the multi-node OPM using helm commands to the specified cluster. The following tables show the variables used in the Helm Delete pipeline.

pipeline/gmacro/affirmed-nr-helm-deploy-macro-pipeline.yml (SVC-DEV)

Table 31. affirmed-nr-helm-delete-macro-pipeline.yml (SVC-DEV)

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository

Variables	Description
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`zoneName`	Name of zone used to name the pipeline run

`variables`
`- template`
`- template`
`- inputFile`
`- inputSourceFolder`
`- deployTimeout`
`- perFedMaxPollTimeout`
`- finalValidationDeployTimeout`
`- finalValidationTimeout`
`- stages.template`
	The following are CNF variables:
	- Path to macrovariables.yml file in INFRA-DEV
	- Path to nf-variables-templates.yml file in INFRA-DEV
	- Input JSON file name
	- Folder in LIB-CODE where `inputFile` is. Remove / at the end of the file path.
	- Time for the pipeline to run.
	- Maximum time for polling the deployed fed.
	- Maximum time for validating the deployed fed.
	- Maximum timeout in minutes until final polling happens for all pods and services. After this, the pipeline will exit with and error code if all pods and services are not running or complete. Values tested successfully with: 10.
	- Path to the micro-pipeline with the corresponding ADO repository.

code/helm_delete/affirmed-nr-helm_deploy.json (LIB-CODE)

Table 32. helm_delete/helm_deploy.json

Variables	Description
`kubeconfig.reference.keyVault.id`	Name of the Azure Key-Vault.
`kubeconfig.reference.keyVault.secretName`	Name of the secret in the Azure Key-Vault with value as kubeconfig of the Kubernetes Cluster.
`prdLocation`	The folder location where the PRD files to deploy multi-node OPM are present.
`acrRepo`	The acr domain name where the images and charts are uploaded.(a4opadsacr.azurecr.io).
`secretName`	The name used to pull the images and charts from the acr repository. (acr-secret)
`chartpath`	The relative path where the charts are uploaded.
`helmOperation`	Operation to be performed. (<delete>)

CNF ISSU Network Cloud Pipeline

The CNF In Service Upgrade (ISSU) pipeline is an Azure DevOps pipeline that uses an OPM Northbound API to operate on the Network Cloud. The OPM ISSU support extends the existing snapshot-based NF upgrade by exposing a CNF Upgrade OPM NB API. This API allows you to specify a rollback policy and an ISSU variable (true/false) to trigger the upgrade of network functions. During and after the upgrade, the CNF status of each federation is monitored.

To use the pipeline, you must enter the ISSU value (**true/false**) and the rollback policy (**`PauseOnFailure/rollbackOnFailure`**) in the *input.json* file. Additionally, you need to have a snapshot file for the version to which the federations will be upgraded. A snapshot file is a JSON file that contains specifications for all federations, including the namespace, chartName, values, chartType, and groupName.

In the **values** field, you can specify IP addresses for fields such as "server" and "peerAddress." Instead of using IP addresses directly, replace them with variable names, then declare these variables in the `snapshot_specific_values.json` file. These variable names can then be reused in the snapshot file to replace the corresponding IP address. The `snapshot_specific_values.json` file is a JSON file that contains key-value pairs, where the key is the variable name and the value is the IP address.

Pipeline Limitations and Prerequisites

There are some limitations to using the OPM ISSU support, including:

- In version 5.1, OPM does not provide custom configurations for pre-fed timeout values.
- OPM does not support VM upgrades for ISSU.
- You must have a snapshot file for the version to which the CNF will be upgraded. Without the snapshot file, you cannot upgrade.

Before using the CNF In Service Upgrade (ISSU) pipeline, make sure that the federations are deployed successfully, and all federation pods are in the n/n state.

To use the CNF Upgrade OPM NB API, you need to provide the following input information:

- **Snapshot with Site-Specific Variables** - place the snapshot and site-specific JSON files in the input folder.
- **Rollback Policy (rollbackOnFailure/PauseOnFailure)** - specify the rollback policy in the input.json file. If you choose rollbackOnFailure, OPM rolls back all upgraded CNFs to their original version upon a failure. If you choose PauseOnFailure, OPM pauses the upgrade upon a failure and will not rollback.
- **ISSU (true/false)** - specify whether or not to use ISSU in the input.json file.

code/cnf_upgrade_issu/affirmed-nr-cnfUpgradeIssu.json (LIB-CODE)

The following table shows the variables used in the CNF In Service Upgrade (ISSU) JSON pipeline.

pipeline/gmacro/affirmed-nr-cnf-upgrade-issu-macro-pipeline.yml (SVC-DEV)

Table 33. affirmed-nr-cnf-upgrade-issu-macro-pipeline.yml

Variables	Description
`cnfOperation`	Value set to upgrade for CNF ISSU.
`CnfISSUPipelineTimeout`	Micro-pipeline timeout
`fedUpgradeTimeout`	Polling interval for getting the final status of all the federations after the ISSU is completed.
`finalValidationTimeout`	Polling interval for the federation after the ISSU is completed; used to retrieve the status of the pods.

Table 34. affirmed-nr-cnfUpgradeIssu.json (LIB-CODE) Variables

Variables	Description
`opm.ipAddress`	IP address of OPM
`opm.username`	SSH username used to communicate with OPM.
`opm.keyVault.name`	Azure Key-Vault name with OPM information.
`opm.keyvault.opmPasswordSecretName`	Azure key vault secret name for OPM GUI user's password.
`opm.keyVault.clusterIdSecretName`	Cluster ID secret name on the Azure Key- Vault
`useUcReg`	Specifies which Infra-Dreg to pull images from upon deployment "mnOpmUcReg" or "nfUcReg" or "none"
`opmUcReg.svcFqdn`	Specifies the FQDN of the MNOP Infra-Dreg where images will be pulled from on deploy.
`opmUcReg.svcIp`	Specifies the IP address of the MNOPM Infra-Dreg service.
`nfUcReg.svcFqdn`	Specifies the FQDN of the NF Infra-Dreg where images will be pulled from on deploy.
`nfUcReg.svcIp`	Specifies the IP address of the NF Infra-Dreg service.

Variables	Description
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with* ***opm.unitycloud.com*** *as the FQDN value and the Public certificate of multi-node OPM containing* ****.unitycloud.com****, please refer to the UnityCloud documentation set.*
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi- master's VIP IP address (used to ssh to multi- node OPM). Multi-master is where the multi- node OPM is deployed.
`multiNodeOPM.` `opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`multiNodeOPM.domainName`	Domain name of the OPM FQDN.
`chartRepo`	Name of Chart Repository created on OPM. This is used by OPM to do Bulk Install using your snapshot.
`dzName`	Name of Deployment Zone to which the K8s Cluster is attached.
`issu`	In Service Upgrade parameter. Set this value to **True** if enabled or **False** in disabled.
`snapshotFolder`	File path in LIB-CODE to snapshot folder.
`snapshotFile`	Name of template snapshot file in LIB- CODE’s snapshotFolder.
`snapshotVariableValuesFolder`	Folder where the snapshot specific file is stored.
`snapshotVariableValuesFile`	Name of snapshot variables/values file in snapshotFolder. Example: `snapshot_ specific_values.json`

VM Cleanup Cluster Pipeline

The following sections show the configurable files in the VM Cleanup Cluster Pipeline and their corresponding repositories.

Table 35. affirmed-nr-cleanup-cluster.json Variables

Variables	Description
`opmMode`	Specifies the OPM deployment type: `singleNode or multiNode`
`multiNodeOPM.FQDN`	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM.

	Set the description value for this to **opm.unitycloud.com**. *__Note:__ For additional information on how to deploy multi-node OPM with __opm.unitycloud.com__ as the FQDN value and the Public certificate of multi-node OPM containing __*.unitycloud.com__, please refer to the UnityCloud documentation set.*
`multiNodeOPM.domainName`	Domain name of the OPM FQDN
`multiNodeOPM.opmMasterVMIP`	For multi-node OPM deployment: multi-master's VIP IP address (used to ssh to multi-node OPM). Multi-master is where the multi-node OPM is deployed.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
`attachCluster.clusterIP`	NF cluster's master node VM IP address, used to attach a cluster to OPM.
`attachCluster.m2mkeyKuberadmin SecretName`	Name of the secret stored in the KeyVault secret name. PADS generates this name to store the NF cluster's M2M private key for the kuberadmin user. No input is required.
`attachCluster.attachType`	Specifies whether the cluster is openstack (`openstack_ attach`) or baremetal (`baremetal_ attach`). For non- Azure clusters, set this value to `nonazure_ attach`.
`attachCluster.username`	Username OPM uses to SSH into the attaching K8 Cluster VMS to complete the attach cluster operation. You should use the root user name for this value.
`clusterName`	Name of the K8 cluster used for kubeconfig
`clusterParameters.ingressVM. adminAccess`	Communication type to Cluster. Accept the default value, SSH
`ClusterParameters.ingressVM. publicIpAddress`	IP-address to communicate with the jump VM
`ClusterParameters.ingressVM. port`	SSH port number to communicate with the jump VM
`ClusterParameters.ingressVM.adminUsername`	SSH username to communicate with the jump VM. Accept the default value, root

`ClusterParameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used to get to the ingress-VM
`clusterParameters.ingressVM.AdminPrivateKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key is used to get to the location of the VM
`clusterParameters.tuning`	True if Cluster tuning needs to be done. False otherwise
`ClusterParameters.vmM2MKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used from the ingress- VM to the individual VMS that forms the K8s cluster
`ClusterParameters.vmM2MKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used from the ingress-VM to the individual VMs that form the K8s cluster is stored
`clusterParameters.vms[].ipAddress`	VM IP address
`clusterParameters.vms[].port`	VM SSH Port
`clusterParameters.vms[].type`	Type of Node (Master/Worker)
`clusterParameters.vms[].adminUsername`	SSH username to communicate with VMs
`clusterParameters.vms[].dataplane`	Set to ***true*** if the node (VM) must have a label `nodetype=dataplane`, otherwise set this value to **`false`**
`opm.ipAddress`	OPM IP address
`opm.sshUsername`	SSH username to communicate to OPM
`opm.keyVault.name`	Azure Key-Vault name with OPM information
`opm.keyVault.opmPrivateKey secretName`	OPM secret name on the Azure Key-Vault for the private SSH key
`opm.certPath`	Certificate path on OPM which copies to all VMs on K8s Cluster
`certCopyPathToAllVMs`	Path on each VM in cluster to where the certificate copies
`opm.username`	OPM GUI username
`opm.opmPasswordSecretName`	Azure Key-Vault secret name for OPM GUI user's password
`opm.dzName`	Deployment Zone name on OPM to which the create K8s Cluster needs to be attached
`clusterParameters.ha_proxy_vip`	The HA VIP address of the Kubernetes cluster post deployment

`clusterParameters.k8_primary_master_ip`	IP address of the primary master node of the CNF Cluster
`clusterParameters.vms[].racklabel`	Following the CNF cluster creation, this value is used to label all nodes (rack and zone).
`isMultiNodeOPMClusterDeployment`	Set this value before running the macro pipeline. Set to **`True`** for multi-node OPM cluster or **`False`** for a CNF cluster.
`ClusterParameters.ingressVM.adminUsername`	SSH username to communicate with the jump VM.
`ClusterParameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used to get to the ingress-VM.
`clusterParameters.ingressVM.AdminPrivateKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used to get to the ingress-VM is kept.
`ClusterParameters.vmM2MKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used from the ingress- VM to the individual VMS that forms the K8s cluster.
`ClusterParameters.vmM2MKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used from the ingress-VM to the individual VMS that forms the K8s cluster is kept.
`clusterParameters.vms[].adminUsername`	SSH username to communicate with VMs.
`opm.sshUsername`	SSH username to communicate to OPM.
`opm.keyVault.name`	Azure Key-Vault name with OPM information.
`opm.keyVault.opmPrivateKey secretName`	OPM secret name on the Azure Key-Vault for the private SSH key.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	MNOPM secret name on the Azure Key-Vault for the private SSH key.

VM ISSU Pipeline

The Virtual Machine In Service Upgrade (VM ISSU) pipeline upgrades the VMs used by the Kubernetes cluster to a specific version provided by UnityCloud.

> ***Note:*** *Upgrade the Kubernetes version for each node individually.*

Before you run this pipeline, ensure you have the following information:

- The URL of the UnityCloud upgrader tar file. This file contains the both the seed file.tgz and the ucUpgrader.py script used to upgrade the VMs. For reference, the URL appears similar to:
  - → https://a4oprodaf.a4opacketcore.microsoft.com/artifactory/rel_build_artifacts/vms/vm-platform/ucUpgrader-vm-platform-<`version`>-main-int.tar
  - → This URL should appear in the *inServiceUpgrade.json* input file, and can be downloaded from the `Upgrade.sh` (located inside the scripts folder) using the `curl -o $tarFileDestination $ucUpgraderTarFileURL curl` command.
- The IP address and respective type (Master/Worker) of each VM in the cluster to be upgraded.

The following sections show the configurable files in the VM ISSU Pipeline and their corresponding repositories.

code/in_service_upgrade/affirmed-nr-inServiceUpgrade.json (LIB-CODE)

The following table shows the variables used in the VM ISSU Pipeline.

Table 36. affirmed-nr-inServiceUpgrade.json (LIB-CODE) Variables

Variables	Description
`opmMode`	The type of OPM (single node/multi-node).
`clusterName`	The name of the Kubernetes cluster used for kubeconfig.
`ClusterParameters.ingressVM.adminAccess`	The communication type to cluster. Accept the default valueSSH.
`ClusterParameters.ingressVM.publicIpAddress`	The IP address used to communicate with the jump VM.
`ClusterParameters.ingressVM.port`	The SSH port number to communicate.
`ClusterParameters.ingressVM.adminUsername`	The SSH user name to communicate with the jump VM. Accept the default value root.
`ClusterParameters.ingressVM.adminPrivateKey.reference.keyvault.name`	The Azure key vault secret name that stores SSH private key used to get to the location of the VM.
`ClusterParameters.ingressVM.adminPrivateKey.reference.secretName`	The Azure key vault secret name that stores the SSH private key used from the ingress VM to the individual VMs from the Kubernetes cluster.

Variables	Description
`ClusterParameters.vmM2MKey.reference.keyvault.name`	Azure key vault name that stores the SSH private key used from the ingress VM to the individual VMs that forms the Kubernetes cluster.
`ClusterParameters.vmM2MKey.reference.secretName`	Azure key vault secret name that stores the SSH private key used from the ingress VM to the individual VMs that the Kubernetes cluster.
`ClusterParameters.vms[].ipAddress`	VM IP address.
`ClusterParameters.vms[].type`	VM type. (**worker/master**)
`ucUpgraderConfig`	Used to set the configuration parameters for ucUpgrader.py.
`ucUpgraderConfig.vmType`	The VM you are upgrading. (**vm-opm** or **vm- platform**)
`ucUpgraderConfig.vmVersion`	The VM version to which you are upgrading.
`ucUpgraderConfig.seedFile`	The seedFile used to upgrade the VM.
`ucUpgraderConfig.upgradeVM`	Indicates if the VM is a new install (**False**) or is being upgraded (**True**).
`ucUpgraderConfig.ignoreConfigparserOverride`	Set this value to **True** when the VM reboots after upgrade to cause the **configParser** to run as normal and ensure the upgrade will not stop after the IP address is configured.
`ucUpgraderConfig.k8sVersionChange`	The Kubernetes version to which you are upgrading. If you enter the same version as what is currently configured, the Kubernetes version does not upgrade. Instead, the OS and kernel upgrade provided that the new seed file has the matching Kubernetes version.
`ucUpgraderConfig.multiBootDebug`	Set this variable to **True** so the multi-boot menu stays up for 4 minutes, then go to default partition. Set this variable to **False** to have the multi-boot menu time out after 30 seconds.
`tarFileDestination`	The path to the tar file present which is to be at the root folder location
`delayInUpgrade`	Delay in minutes to upgrade each VM. Ideal time would be 10min. Value to be provided is 10m.
`storageAccountResourceGroup`	Resource group name where the tar file is uploaded
`storageAccountName`	Storage account name where the tar file is uploaded
`containerName`	Container name of the storage account
`blobName`	File name of the tar file
`clusterParameters.vms`	Input all VMs that must be upgraded inside the array.
`clusterParameters.ingressVM.adminPrivateKey.reference.keyVault.name`	The name of the Azure Key-Vault.
`clusterParameters.ingressVM.adminPrivateKey.reference.secretName`	The name of the secret in the Azure Key-Vault corresponding to the public key used while deployment.

CNF Deploy full build pipeline

The CNF Deploy full build pipeline is a streamlined, one-touch process that creates a cluster, copies charts and images to the registry, deploys the network function in the cluster through a single pipeline When CNF version has NF Infra-Dreg, this pipeline should be used so that it will retag the image source to NF Infra-Dreg. Essentially, it is a macro pipeline that sequentially invokes the following micro pipelines:

- OPM Automation
- ACR to MNOPM UcReg Image copy
- Finalize Kubernetes cluster
- CNF deploy
- MNOPM UcReg to NF UcReg Image copy
- Retag image source to NF Infra-Dreg

Since it reuses existing pipelines, the prerequisites and input configuration for this pipeline are a consolidation of those for all these pipelines.

Prerequisites

1. Cluster VMs should be created with the latest image and have cinder-csi enabled.
2. All charts and images for the CNF release should be loaded into ACR.
3. MNOPM used for the CNF deployment should be the latest version, and fed Infra-Dreg should be enabled during its deployment.

The following sections show the configurable files in the CNF Deploy full build Pipeline and their corresponding repositories:

pipeline/gmacro/affirmed-nr-cnf-operations-fullbuild-macro-pipeline.yml

Table 37. affirmed-nr-cnf-operations-fullbuild-macro-pipeline.yml @SVC-DEV

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository

`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`parameters :`	
`template`	Path to macrovariables.yml file in INFRA-DEV
`template`	Path to nf-variables-templates.yml file in INFRA-DEV
`deployTimeout`	Time for the pipeline to run.
`perFedMaxPollTimeout`	Max time for polling the fed which has been deployed.
`finalValidationDeployTimeout`	Max time for validating the deployed fed.
`finalValidationTimeout`	Maximum timeout in minutes until final polling occurs for all pods and services. After this, the Pipeline will exit with an error code if all pods and services are not running or complete.
`inputFileForOPMAutomation`	Input json file for the OPM Automation pipeline
`inputSourceFolderForOPMAutomation`	Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
`opmPrereqAutomationTimeout`	Timeout of the OPM Automation pipeline
`acrToOpmImageCopyTimeout`	Timeout in minutes for the pipeline "ACR to MNOPM UcReg Image copy" and "MNOPM UcReg to NF UcReg Image copy".
`inputFileForACRToUcReg`	Input json file for "ACR to MNOPM UcReg Image copy" pipeline.
`inputSourceFolderForACRToUcReg`	Folder in LIB-CODE where inputFile for inputFileForACRToUcReg
`inputFileForUcRegToUcReg`	Input json file for "MNOPM UcReg to NF UcReg Image copy" pipeline.
`inputSourceFolderForUcRegToUcReg`	Folder in LIB-CODE where inputFile for inputFileForUcRegToUcReg
`finalizeK8ClusterTimeout`	Timeout in minutes for the pipeline finalizek8cluster
`inputSourceFolderForFinalizeCluster`	Folder in LIB-CODE where inputFile for inputFileForFinalizeCluster
`inputFileForFinalizeCluster`	Input json file for finalizek8Cluster pipeline.
`fedDeployTimeout`	Polling interval (in seconds) for calling NB API to poll all federations status.Recommended value is 10
`cnfDeployTimeout`	Total timeout in minutes for the CNF pipeline to run.

finalValidationDeployTimeout	Polling timeout in seconds for checking pods and services status for each fed
inputSourceFolderForCNFDeploy	Folder in LIB-CODE where inputFile for inputFileForCNFDeploy is. Remove / at the end of the file path.
inputFileForCNFDeploy	Input json file for cnfdeploy pipeline.
inputSourceFolderForFinalizeCluster	Folder in LIB-CODE where inputFile for inputFileForFinalizeCluster
inputFileForFinalizeCluster	Input json file for finalizek8Cluster pipeline.
inputFileForCNFUpgrade	Input json file for cnf deploy pipeline.
inputSourceFolderForCNFUpgrade	- Folder in LIB-CODE where inputFile for cnf deploy pipeline is. Remove / at the end of the file path
cnfOperation	The default value is “deploy”.
stages.template	Path to micro pipeline with the corresponding ADO repository

CNF Deploy full build pipeline using MNOPM Infra-Dreg

The CNF Deploy full build pipeline is a streamlined, one-touch process that creates a cluster, copies charts and images to the MNOPM Infra-Dreg registry, deploys the network function in the cluster through a single pipeline. When CNF version does not have NF Infra-Dreg, this pipeline should be used so that it will point the image source to MNOPM Infra-Dreg. Essentially, it is a macro pipeline that sequentially invokes the following micro pipelines:

- OPM Automation
- ACR to MNOPM UcReg Image copy
- Finalize Kubernetes cluster
- CNF deploy

Since it reuses existing pipelines, the prerequisites and input configuration for this pipeline are a consolidation of those for all these pipelines.

Prerequisites

1. All charts and images for the CNF release should be loaded into ACR.
2. MNOPM used for the CNF deployment should be the latest version, and fed-infra_dreg should be enabled during its deployment.

The following sections show the configurable files in the CNF Deploy full build Pipeline and their corresponding repositories:

pipeline/gmacro/affirmed-nr-cnf-operations-fullbuild-macro-pipeline-mnopm-ucreg.yml

Table 38. affirmed-nr-cnf-operations-fullbuild-macro-pipeline-mnopm-ucreg.yml @SVC-DEV

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`parameters :`	
`template`	Path to macrovariables.yml file in INFRA-DEV
`template`	Path to nf-variables-templates.yml file in INFRA-DEV
`deployTimeout`	Time for the pipeline to run.
`perFedMaxPollTimeout`	Max time for polling the fed which has been deployed.
`finalValidationDeployTimeout`	Max time for validating the deployed fed.
`finalValidationTimeout`	Maximum timeout in minutes until final polling occurs for all pods and services. After this, the Pipeline will exit with an error code if all pods and services are not running or complete.
`inputFileForOPMAutomation`	Input json file for the OPM Automation pipeline

`inputSourceFolderForOPMAutomation`	Folder in LIB-CODE where inputFile is. Remove / at the end of the file path.
`opmPrereqAutomationTimeout`	Timeout of the OPM Automation pipeline
`acrToOpmImageCopyTimeout`	Timeout in minutes for the pipeline "ACR to MNOPM UcReg Image copy" and "MNOPM UcReg to NF UcReg Image copy".
`inputFileForACRToUcReg`	Input json file for "ACR to MNOPM UcReg Image copy" pipeline.
`inputSourceFolderForACRToUcReg`	Folder in LIB-CODE where inputFile for inputFileForACRToUcReg
`finalizeK8ClusterTimeout`	Timeout in minutes for the pipeline finalizek8cluster
`inputSourceFolderForFinalizeCluster`	Folder in LIB-CODE where inputFile for inputFileForFinalizeCluster
`inputFileForFinalizeCluster`	Input json file for finalizek8Cluster pipeline.
`fedDeployTimeout`	Polling interval (in seconds) for calling NB API to poll all federations status.Recommended value is 10
`cnfDeployTimeout`	Total timeout in minutes for the CNF pipeline to run.
`finalValidationDeployTimeout`	Polling timeout in seconds for checking pods and services status for each fed
`inputSourceFolderForCNFDeploy`	Folder in LIB-CODE where inputFile for inputFileForCNFDeploy is. Remove / at the end of the file path.
`inputFileForCNFDeploy`	Input json file for cnfdeploy pipeline.
`inputSourceFolderForFinalizeCluster`	Folder in LIB-CODE where inputFile for inputFileForFinalizeCluster
`inputFileForFinalizeCluster`	Input json file for finalizek8Cluster pipeline.
`inputFileForCNFUpgrade`	Input json file for cnf deploy pipeline.
`inputSourceFolderForCNFUpgrade`	- Folder in LIB-CODE where inputFile for cnf deploy pipeline is. Remove / at the end of the file path
`cnfOperation`	The default value is “deploy”.

`stages.template`	Path to micro pipeline with the corresponding ADO repository

MNOPM Finalizek8 Cluster Pipeline

As part of these changes a new pipeline has been added to run the finalizek8cluster for MNOPM deployment.

The following sections show the configurable files in the MNOPM Finalize K8 Cluster Pipeline and their corresponding (repository):

- affirmed-nr-mnopm_finalizeK8sCluster-macro-pipeline.yml (INFRA-DEV)
- pipeline/mnopm_finalize_cluster/affirmed-nr-mnopm-finalizeK8sCluster-micro-pipeline.yml (LIB-CODE)
- code/mnopm_finalize_cluster/affirmed-nr-finalizeK8sCluster-multi-opm.json (LIB-CODE

affirmed-nr-mnopm_finalizeK8sCluster-macro-pipeline.yml

The following table shows the variables used in the FinalizeK8sCluster macro pipeline.

Table 39. affirmed-nr-mnopm_finalizeK8sCluster-macro-pipeline.yml Variables

Variables	Description
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`variables` `- azureSubscription`	The following are CNF variables: - Azure subscription name

- inputFile **- inputSourceFolder** **- template**	- Input JSON file name - Folder in LIB-CODE where inputFile is. Remove / at the end of the file path. - The absolute path to the macrovariables.yml file in INFRA-DEV repository. When referring from another repository, add **@INFRA** at the end of the file path
stages.template	Path to micro pipeline with the corresponding ADO repository
name	Name of the Pipeline when it is run. Includes parameter `zoneName`
appendCommitMessageToRunName	Append the commit message to the build number. The default is true
zoneName	Parameter to specify zone name
workerCount	Parameter for number of workers
azureRegion	Parameter for the Azure region on which the Pipeline runs
forceLocalDNS	Parameter to fix local DNS issue with VM images

code/mnopm_finalize_cluster/affirmed-nr-finalizeK8sCluster-multi-opm.json (LIB-CODE

The following shows the variables used in the MNOPM FinalizeK8sCluster Input JSON pipeline.

Variables	Description
opmMode	Specifies the OPM deployment type: `singleNode or multiNode`
multiNodeOPM.FQDN	For multi-node OPM deployment: FQDN of OPM configured when deploying multi-node OPM. Set the description value for this to **opm.unitycloud.com**. ***Note:*** *For additional information on how to deploy multi-node OPM with* ***opm.unitycloud.com*** *as the FQDN value and the Public certificate of multi-node OPM containing* ****.unitycloud.com****, please refer to the UnityCloud documentation set.*

Variables	Description
multiNodeOPM.domainName	Domain name of the OPM FQDN
multiNodeOPM.opmMasterVMIP	For multi-node OPM deployment: multi-master's VIP IP address (used to ssh to multi-node OPM). Multi-master is where the multi-node OPM is deployed.
multiNodeOPM.opmMasterVMPrivateKeySecretName	For multi-node OPM deployment: secret name in vault which contains the key that is used to ssh to master VM node without password.
attachCluster.clusterIP	NF cluster's master node VM IP address, used to attach a cluster to OPM.
attachCluster.m2mkeyKuberadmin SecretName	Name of the secret stored in the KeyVault secret name. PADS generates this name to store the NF cluster's M2M private key for the kuberadmin user. No input is required.
attachCluster.attachType	Specifies whether the cluster is openstack (`openstack_ attach`) or baremetal (`baremetal_ attach`). For non- Azure clusters, set this value to `nonazure_ attach`.
attachCluster.username	Username OPM uses to SSH into the attaching K8 Cluster VMS to complete the attach cluster operation. You should use the root user name for this value.
clusterName	Name of the K8 cluster used for kubeconfig
clusterParameters.ingressVM. adminAccess	Communication type to Cluster. Accept the default value, SSH
ClusterParameters.ingressVM. publicIpAddress	IP-address to communicate with the jump VM
ClusterParameters.ingressVM. port	SSH port number to communicate with the jump VM
ClusterParameters.ingressVM.adminUsername	SSH username to communicate with the jump VM. Accept the default value, root
ClusterParameters.ingressVM. adminPrivateKey.reference.key Vault.name	Azure Key-Vault name with SSH private-key to be used to get to the ingress-VM
clusterParameters.ingressVM. AdminPrivateKey.reference. secretName	Azure Key-Vault secret- name where the SSH private-key is used to get to the location of the VM
clusterParameters.tuning	True if Cluster tuning needs to be done. False otherwise

Variables	Description
ClusterParameters.vmM2MKey. reference.keyVault.name	Azure Key-Vault name with SSH private-key to be used from the ingress- VM to the individual VMS that forms the K8s cluster
ClusterParameters.vmM2MKey. reference.secretName	Azure Key-Vault secret- name where the SSH private-key to be used from the ingress-VM to the individual VMs that form the K8s cluster is stored
clusterParameters.vms[]. ipAddress	VM IP address
clusterParameters.vms[].port	VM SSH Port
clusterParameters.vms[].type	Type of Node (Master/Worker)
clusterParameters.vms[]. adminUsername	SSH username to communicate with VMs
clusterParameters.vms[]. dataplane	Set to ***true*** if the node (VM) must have a label `nodetype=dataplane`, otherwise set this value to **`false`**
opm.ipAddress	OPM IP address
opm.sshUsername	SSH username to communicate to OPM
opm.keyVault.name	Azure Key-Vault name with OPM information
opm.keyVault.opmPrivateKey secretName	OPM secret name on the Azure Key-Vault for the private SSH key
opm.certPath	Certificate path on OPM which copies to all VMs on K8s Cluster
certCopyPathToAllVMs	Path on each VM in cluster to where the certificate copies
opm.username	OPM GUI username
opm.opmPasswordSecretName	Azure Key-Vault secret name for OPM GUI user's password
opm.dzName	Deployment Zone name on OPM to which the create K8s Cluster needs to be attached
clusterParameters.ha_proxy_vip	The HA VIP address of the Kubernetes cluster post deployment
clusterParameters.k8_primary_master_ip	IP address of the primary master node of the CNF Cluster
clusterParameters.vms[]. racklabel	Following the CNF cluster creation, this value is used to label all nodes (rack and zone).
isMultiNodeOPMClusterDeployment	Set this value before running the macro pipeline. Set to **`True`** for multi-node OPM cluster or **`False`** for a CNF cluster.

Variables	Description
`ClusterParameters.ingressVM.adminUsername`	SSH username to communicate with the jump VM.
`ClusterParameters.ingressVM.adminPrivateKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used to get to the ingress-VM.
`clusterParameters.ingressVM.AdminPrivateKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used to get to the ingress-VM is kept.
`ClusterParameters.vmM2MKey.reference.keyVault.name`	Azure Key-Vault name with SSH private-key to be used from the ingress- VM to the individual VMS that forms the K8s cluster.
`ClusterParameters.vmM2MKey.reference.secretName`	Azure Key-Vault secret- name where the SSH private-key to be used from the ingress-VM to the individual VMS that forms the K8s cluster is kept.
`clusterParameters.vms[].adminUsername`	SSH username to communicate with VMs.
`opm.sshUsername`	SSH username to communicate to OPM.
`opm.keyVault.name`	Azure Key-Vault name with OPM information.
`opm.keyVault.opmPrivateKey secretName`	OPM secret name on the Azure Key-Vault for the private SSH key.
`multiNodeOPM.opmMasterVMPrivateKeySecretName`	MNOPM secret name on the Azure Key-Vault for the private SSH key.

Running Pipelines

You must meet the following prerequisites in order to properly run the configured pipelines, including FinalizeK8sCluster, CNF Delete, and CNF Deploy Pipelines.

Prerequisites to Initially Run Pipelines

Uncomment the code in each of the pipelines

The code below has been commented which is not run in the MSFT site. This need to be uncommented to make it run in the ATT Site

File : affirmed-nr-acr-to-opm-image-copy-micro-pipeline.yml

```
Under job:
- job: "PreChecks"
- bash: |
              # echo az account set --subscription $(JVS-AZ-NC-SUB) ATT
code is commented out as it is not required
              # az account set --subscription $(JVS-AZ-NC-SUB)
              # az account list -o table
            displayName: Setting account to ADO subscription

- job: "ACR_OPM"

           - bash: |
              # echo az account set --subscription $(JVS-AZ-NC-SUB)
              # az account set --subscription $(JVS-AZ-NC-SUB)
              # az account list -o table
            displayName: Setting account to ADO subscription


affirmed-nr-cnf-operations-deploy-micro-pipeline.yml

 - job : 'cnfDeploy'

# Commented out the existing code as it is for ATT
              # - bash: |
              #       set -e;
              #       echo "##[debug] Checking Keyvault Secrets..."
              #
secretFilePath=$(_artifactsLocation)/input/$(json.secretValuesFile);
              #       echo "secretFilePath = $secretFilePath"
              #       echo -e "\n"

              #       echo "##[debug] iterate over the secret names(keys
in secretValues json object) in secret values json file"
              #       for secretName in `jq -r '. | keys[] as $k |
"\($k)"' $secretFilePath`; do
              #         echo "secretName = $secretName"
              #         keyvaultSecretName=`jq --raw-output --arg tmp
"$secretName" '.[$tmp]' $secretFilePath`;
              #         output=$(az keyvault secret show --name
"$keyvaultSecretName" --vault-name
"$(json.kubeconfig.reference.keyVault.id)" -o tsv)
              #         if [[ -z $output ]]; then
              #           echo "##[error]Keyvault secret
$keyvaultSecretName not found. Exiting..."
              #           exit 1
              #         else
              #           echo "##[section]$keyvaultSecretName secret
found."
              #         fi
              #         echo -e "\n"
              #       done
              #   displayName: "Check if secrets are present in Azure Key-
Vault"
```

```
              # - bash: |
              #       set -e;
              #       echo "##[debug] Updating secret values file
$(_artifactsLocation)/input/$(json.secretValuesFile)"
              #
secretFilePath=$(_artifactsLocation)/input/$(json.secretValuesFile);
              #       echo "secretFilePath = $secretFilePath"
              #       echo -e "\n"

              #       # iterate over the secret names(keys in secretValues
json object) in secret values json file
              #       for secretName in `jq -r '. | keys[] as $k |
"\($k)"' $secretFilePath`; do
              #       echo "secretName = $secretName"
              #         keyvaultSecretName=`jq --raw-output --arg tmp
"$secretName" '.[$tmp]' $secretFilePath`;
              #         echo -e "\n"

              #         echo "##[debug] Fetching secret value from Azure
Key-Vault..."
              #         keyvaultSecretValue=`az keyvault secret show --
name "$keyvaultSecretName" --vault-name
"$(json.kubeconfig.reference.keyVault.id)" | jq -r '.value'`;
              #         echo "##[section] Successfully retrived value for
$keyvaultSecretName"
              #         echo -e "\n"

              #         echo "##[debug] Updating secret value in secret
values file..."
              #         output=`jq --arg secretName "$secretName" --arg
keyvaultSecretValue "$keyvaultSecretValue"
".[\"$secretName\"]=\"$keyvaultSecretValue\"" $secretFilePath`;
              #         echo $output > $secretFilePath;
              #         echo -e "\n"
              #       done
              #   displayName: "Replace secret names with values from
Azure Key-Vault"

              # - bash: |
              #       set -e;
              #       echo "##[debug] Checking Snapshot file..."
              #       export
secretSnapshotValuesFilePath="$(_artifactsLocation)/input/$(json.secretVal
uesFile)"
              #       echo "secretSnapshotValuesFilePath =
$secretSnapshotValuesFilePath"
              #       echo -e "\n"

              #       echo "##[debug] Get snapshot varibles from
$secretSnapshotValuesFilePath"
              #       export snapshot_variables=( $(echo `jq -r '. |= keys
' $secretSnapshotValuesFilePath` | jq --raw-output '.[]') );
              #       for snapshot_variable in "${snapshot_variables[@]}";
do
```

```
              #         echo " variables: $snapshot_variable"
              #       done
              #       echo -e "\n"

              #       echo "##[debug] Get snapshot varibles values from
$secretSnapshotValuesFilePath"
              #       for snapshot_variable in "${snapshot_variables[@]}";
do
              #           export snapshot_variable_value=$(jq --raw-output
--arg tmp "$snapshot_variable" '.[$tmp]' $secretSnapshotValuesFilePath |
sed 's/\\/\\\\\\\\/g');
              #           sed -i
"s#$snapshot_variable#$snapshot_variable_value#g"
$(_artifactsLocation)/input/$(json.snapshotFile);
              #       done
              #       echo -e "\n"

              #       echo "##[debug] Print updated snapshot file"
              #       cat $(_artifactsLocation)/input/$(json.snapshotFile)
              #   displayName: "Update Snapshot file with secret values"
              #   timeoutInMinutes: 5
```

PADS Release 5.1.0.3 uses the Operations and Policy Manager (OPM) for CNF Operations such as deploy, delete, update.

Because PADS uses NB API to communicate to the OPM, you must update the OPM's default SSL Certificate to include the OPM's IP Address. Complete the following steps to update the SSL Certificate:

> ***Note:*** *The following prerequisites to initially run pipelines only need to be completed once. Once these are completed, they are applied to all pipeline runs subsequently created.*

> ***Tip:*** *The prerequisites in the following sections can be completed using the OPM Automation Pipeline, including [Prerequisites to Run a FinalizeK8sCluster Pipeline](#) and steps 1-3 in [Prerequisites to Run a CNF Deploy Pipeline](#). The steps detailed in these sections are left for reference. For more information, see [Run an OPM Automation Pipeline](#).*

1. SSH into the OPM VM.
2. Create a new certificate for the OPM, which contains the OPM IP address.

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -subj
'/CN=ambassador-cert' -nodes -addext "subjectAltName = IP:<opm-ip-
address>"
```

3. Print the certificate and verify an IP address is provided.

```
openssl x509 -in cert.pem -text
```

4. Create a Kubernetes secret with cert.pem as its value.

```
kubectl create secret tls tls-cert --cert=cert.pem -- key=key.pem -n
vnfmgr
```

5. Verify the secret is created.

```
kubectl get secret -n vnfmgr
```

6. Edit the wildcard-host of the OPM and change the `spec.tlsSecret.name` value to `tls-cert`.

```
kubectl edit host wildcard-host -n vnfmgr
```

7. Log in to the OPM GUI and verify the new certificate works correctly.

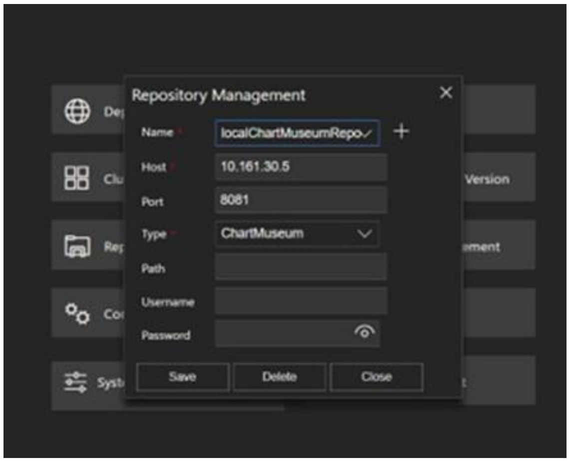
Prerequisites to Run a FinalizeK8sCluster Pipeline

Complete the following prerequisites prior to running a FinalizeK8sCluster Pipeline.

> ***Tip:*** *The prerequisites in this section can be completed using the OPM Automation Pipeline. The steps detailed here are left for reference. For more information, see* *[Run an OPM Automation Pipeline](#)*.

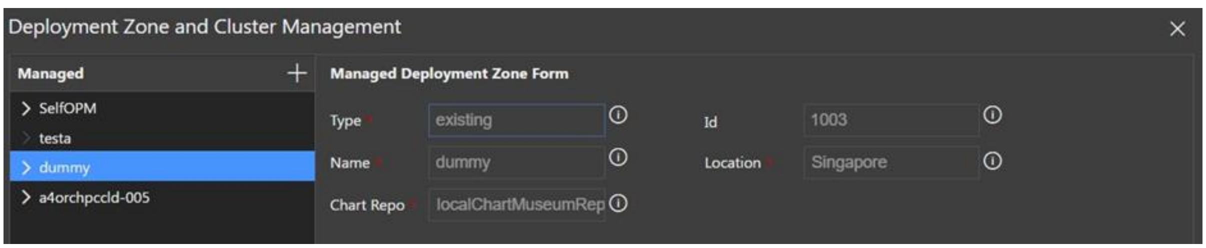
1. Create Chart Repository on OPM (Repository Management) using the details of OPM Chart Museum.

Figure 6. OPM – Create Chart Repository



2. Create an existing Deployment Zone by providing the Deployment Zone name in the finalsizeK8sClutser.json file and the Chart Repository you created in the previous step.

Figure 7. OPM – Create Deployment Zone



Prerequisites to Run a CNF Deploy Pipeline

Complete the following prerequisite steps prior to running a CNF Deploy Pipeline.

> ***Tip:*** *The prerequisites detailed in steps 1-3 can be completed using the OPM Automation Pipeline and are detailed in the following section for reference. Steps 4-6 are not automated and require manual completion. For more information, see [Run an OPM Automation Pipeline](#).*

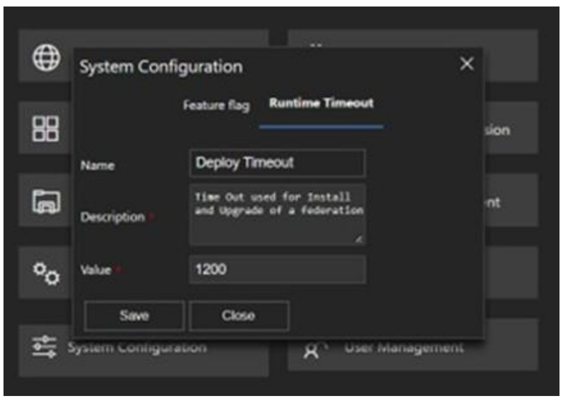
1. Load all of the Helm Charts to OPM's Chart Museum Repository using the cna_deploy_ inventory script.

```
./cna_deploy_inventory --inventory *.json --dest_docker_url
https://$<OPM>:5005 --dest_helm_url https://$<OPM>/chartm/ --
docker_yaml_path ${OPM}:5005 --login <user>:<password> --use_ nerdctl
```

> ***Note:*** *You must run the CNF deploy Pipeline on a K8s Cluster with 0 federations installed.*

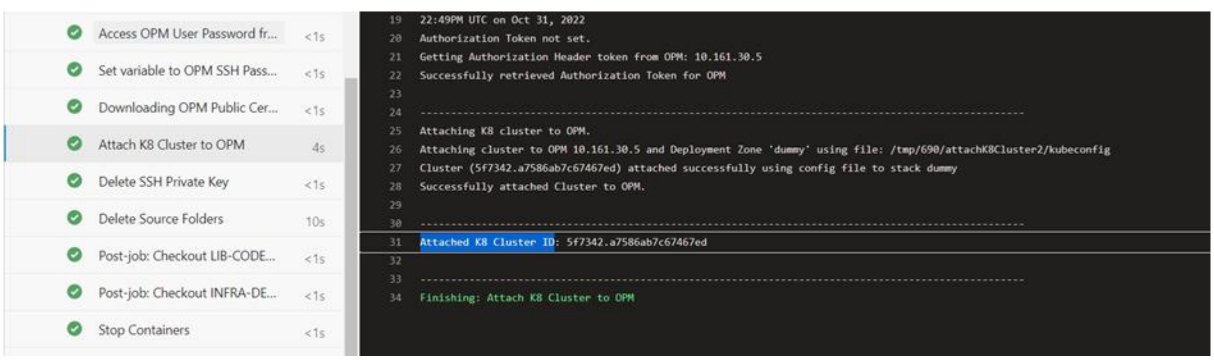
2. Update the **`Runtime Deploy Timeout`** value in the OPM GUI. The recommended value is ***1200***.

Figure 8. OPM – Update Runtime Deploy Timeout



3. Get the Cluster ID from the FinalizeK8sCluster Pipeline, attach the K8s Cluster Stage log and update it in *cnf-operations-deploy.json*.

Figure 9. OPM – FinalizeK8sCluster ClusterID



4. Get the Templatized Snapshot with variables, copy it to LIB-CODE and update the snapshot file path in *cnf-operations-deploy.json*.

5. Add the list of variables as keys and corresponding values in the snapshot_specific_ values.json file.

6. Create a secret in Azure Key-Vault with an **`<OPM NB API user and password>`** value.

Run an OPM Automation Pipeline

In the FinalizeK8sCluster Pipeline, after a K8s Cluster is created and attached to the OPM, a Cluster ID is generated on the OPM. To automate using the cluster ID generated from the FinalizeK8Cluster Pipeline in CNF Operations Pipelines, it is stored in the Azure Key-Vault and the secret name is used as reference in the CNF Operations related pipeline.

- In the FinalizeK8sCluster Pipeline, `clusterName` is used to create/update secret name in the Azure Key-Vault and store the Cluster ID there, as well. Secret name is **`<clusterName>-opm-cluster-id`**. A similar pattern is used for storing kubeconfig in the Key-Vault.
- In both the CNF Deploy and CNF Delete Pipelines, `clusterId` is removed from the input JSON file. **`opm.keyVault.clusterIdSecretName`** is added and its value is **`<clusterName>-opm-cluster-id`**, similar to the kubeconfig secret name. The pipeline picks this value from the Key-Vault.

Running an OPM Automation Pipeline

1. Access the **Pipelines** page.
2. On the list of available pipelines, select the **[your org] OPMAutomation** pipeline.
3. Click the **Run pipeline** button.
4. On the **Run pipeline** window:
   a. Verify the correct branch is chosen or select it from the drop-down.
   b. Verify the advanced options are configured correctly.
   c. (Optional) Select to enable system diagnostics.
   d. Click **Run**.

Run a Kubernetes Cluster Building Pipeline

Prior to running a Kubernetes Cluster Building pipeline, verify that the prerequisite tasks are complete. See Prerequisites to Run a FinalizeK8sCluster Pipeline.

Running a Kubernetes Cluster Building Pipeline

1. Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] finalize-K8s** pipeline.

3. Click the **Run pipeline** button.

4. On the **Run pipeline** window:

   a. Verify the correct branch is displayed or select it from the drop-down.

   b. Verify the advanced options are configured correctly.

   c. (Optional) Select to enable system diagnostics.

   d. Click **Run**.

Run a CNF Deploy Pipeline

Prior to running a CNF Deploy pipeline, verify that the prerequisite tasks are complete. See Prerequisites to Run a CNF Deploy Pipeline.

Running a CNF Deploy Pipeline

Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] CNFDeploy** pipeline.

3. Click the **Run pipeline** button.

4. On the **Run pipeline** window:

   a. Verify the correct branch is displayed or select it from the drop-down.

   b. Verify the advanced options are configured correctly.

   c. (Optional) Select to enable system diagnostics.

   d. Click **Run**.

Run a CNF Delete Pipeline

Prior to running a CNF Delete pipeline, verify that the prerequisite tasks are complete.

Running a CNF Delete Pipeline

Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] CNFDelete** pipeline.

3. Click the **Run pipeline** button.

4. On the **Run pipeline** window:

    a. Verify the correct branch is displayed or select it from the drop-down.

    b. Verify the advanced options are configured correctly.

    c. (Optional) Select to enable system diagnostics.

    d. Click **Run**.

Run a CNF Upgrade Pipeline

Prior to running a CNF Upgrade pipeline, verify that the prerequisite tasks are complete.

Running a CNF Upgrade Pipeline

Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] CNFUpgrade** pipeline.
3. Click the **Run pipeline** button.
4. On the **Run pipeline** window:

    a. Verify the correct branch is displayed or select it from the drop-down.

    b. Verify the advanced options are configured correctly.

    c. (Optional) Select to enable system diagnostics.

    d. Click **Run**.

Run a CNF Operation Status Pipeline

Prior to running a CNF Operation Status pipeline, verify that the prerequisite tasks are complete.

Running CNF Operation Status Pipeline

Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] CNFOperationStatus** pipeline.
3. Click the **Run pipeline** button.
4. On the **Run pipeline** window:

    a. Verify the correct branch is displayed or select it from the drop-down.

b. Verify the advanced options are configured correctly.

c. (Optional) Select to enable system diagnostics.

d. Click **Run**.

Run a CSO Software Install Pipeline

Prior to running a CSO Software Install pipeline, verify that the prerequisite tasks are complete.

Running a CSO Software Install Pipeline

Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] CSOSoftwareInstall** pipeline.
3. Click the **Run pipeline** button.
4. On the **Run pipeline** window:
    a. Verify the correct branch is displayed or select it from the drop-down.
    b. Verify the advanced options are configured correctly.
    c. (Optional) Select to enable system diagnostics.
    d. Click **Run**.

Run a Pipeline Locking Pipeline

Prior to running a Pipeline Locking pipeline, verify that the prerequisite tasks are complete. See Pipelines.

Running a Pipeline Locking Pipeline

Access the **Pipelines** page.

2. On the list of pipelines, select the **[your org] PipelineLockingPipeline** pipeline.
3. Click the **Run pipeline** button.
4. On the **Run pipeline** window:
    a. Verify the correct branch is displayed or select it from the drop-down.
    b. Verify the advanced options are configured correctly.
    c. (Optional) Select to enable system diagnostics.

d. Click **Run**.

Nexus Pipelines

PADS automates the deployment, upgrading, and deletion of CNFs on the Azure Operator Nexus (AON) platform through various pipelines.

To deploy CNF, execute the following pipelines:

1. Publisher flow pipeline

   This pipeline creates AOSM resources such as:

   - Publisher
   - Artifact store
   - Network Function Definition Group (NFDG)
   - Network Service Design Group (NSDG)

2. Designer flow pipeline

   This pipeline creates Artifact Manifests within the Publisher Artifact store and uploads artifacts into the Artifact store's Azure Container Registry (ACR). It also creates AOSM resources such as:

   - Network Function Definition Version (NFDV)
   - Network Service Design Version (NSDV)
   - CNF specific Global Configuration Group Schema (Global CGS)
   - Site-specific Instance Configuration Group Schema (Instance CGS)
   - Secret Configuration Group Schema (Secret CGS)

3. Operator Deploy pipeline

   This pipeline creates the following AOSM resources and deploys the CNF via creating SNS:

   - CNF specific Global Configuration Group Value (Global CGV)
   - Site-specific Instance Configuration Group Value (Instance CGV)
   - Secret Configuration Group Value (Secret CGV)
   - Site
   - Site Network Service (SNS)

To delete CNF, execute the Operator Delete pipeline.

1. Operator Delete pipeline

   This pipeline deletes the following AOSM resources and deletes the CNF deployment by deleting the SNS:

   - CNF specific Global Configuration Group Value (Global CGV)
   - Site-specific Instance Configuration Group Value (Instance CGV)
   - Secret Configuration Group Value (Secret CGV)
   - Site
   - Site Network Service (SNS)

> ***Note:*** *The Azure Operator Nexus platform is a pre-GA release.*

> ***Warning:*** *Ensure that any Service Principal used for executing Nexus Pipelines to create AOSM artifacts in a given Resource Group is granted Contributor level access to that Resource Group.*

> ***Warning:*** *If a Nexus-related PRD must be modified, complete this before running them with Nexus pipelines. Managing PRD files is beyond the scope of the PADS software application.*

Prerequisites

Use the procedure below to complete the required prerequisites to run the pipelines to deploy CNF.

1. Install or upgrade PADS software. Refer sections Install PADS Software and Upgrade PADS Software
2. Configure ADO variable group
3. Configure pipeline template variables
4. Configure Azure resource name template variables
5. Configure Publisher flow pipeline
6. Run Publisher flow pipeline

7. Configure Designer flow pipeline
8. Run Designer flow pipeline
9. Configure the N-AKS cluster as follows using the Affirmed Networks MOP provided:
    a. Arc-enabled
    b. Associated custom location
10. Configure Operator Deploy pipeline
11. Run Operator Deploy pipeline

Use the procedure below to complete the required prerequisites to run the pipelines to delete CNF.

1. Ensure that CNF is already deployed and running successfully using CNF Operator Deploy or CNF Operator ISSU pipeline
2. Configure Operator Delete pipeline
3. Run Operator Delete pipeline

Note: While configuring macro pipelines, COMMON repo can be commented out if “connectit-send-notification-template.yaml” and “connectit-az-resource-status-template.yaml” are not called.

> ***Note:** The Correlation ID displays in the ADO pipeline box.*

Configure ADO variable group

Ensure that the following variables are defined in a ADO variable group.

- SMF-KV-NAME or UPF-KV-NAME – Key vault name for getting secret values to populate secret CGV and for getting credentials to access source Registry during ACR copy pipeline.
- SP-APP-ID – Service Principal ID
- SP-SECRET – Service Principal Client secret
- SP-TENANT-ID – Service Principal Tenant ID
- POOL-NAME – Agent Pool Name
- SMF-MOTS-ID or UPF-MOTS-ID – Correlation ID to tag in AOSM resources
- SMF-SP-NAME or UPF-SP-NAME – Automated SP Name to tag in AOSM resources

Configure pipeline template variables

Before running any pipelines, make sure to configure the pipeline template variables YAML file. This file gathers all the pipeline variables used across different pipelines. The JSON configuration files within each individual pipeline contain placeholders for some variables, which will be referenced from this YAML file.

By default, there are two copies of pipeline template variables files in the SVC-DEV repository: one for SMF and another for UPF.

- SVC-DEV/config/variables/smf-ds-pipeline-template-variables.yml
- SVC-DEV/config/variables/upf-ds-pipeline-template-variables.yml

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed.

The table below outlines the variables defined in the pipeline template variables file. Some of the variables are configured by default. Make sure to confirm them and update the ‘<CHANGEME>’ placeholders with appropriate values.

Table 40. pipeline-template-variables.yml Variables

Variables	Description
`containerImageName`	Docker image name used in the pipeline.
`containerImageTag`	Version of the image tag used in the pipeline
`paasPrdFile`	Name of PAAS PRD file under LIB-CODE/prd folder
`cnfPrdFile`	Name of CNF PRD file under LIB-CODE/prd folder
`artifactVersionRange`	The version range of the artifacts. Edit this to match the UC version currently running. Example: `>=4.0.0-0`
`UCMajorReleaseNumber`	UC Major Release number. Example: `5.1.0`
`nfdvMinorVersion`	NFDV Minor Version. Example: `2`
`removeSourceFolder`	Flag to indicate whether to remove the source folder. By default, it is `false`.
`enableRollbackOnISSUFailure`	Flag to indicate whether to enable rollback on CNF ISSU failure or not. By default, it is `false`. Allowed values – `true`, `false` When this flag is enabled, ensure that NF ARM template has the changes to enable rollback on failure at both the NF App level and the entire NF level. Refer Sample CNF ARM template.
`keyvaultName`	Name of the Azure Key-Vault that stores secret values. Use variable $(SMF-KV-NAME) or $(UPF-KV-NAME) based on CNF be deployed.
`registrySecretName`	Secret name in Key-Vault that contains the password for UnityCloud Artifactory. Applies when srcDest Designer pipeline parameter is set to “UC_to_ACR”.

Variables	Description
`registryUserName`	Secret name in Key-Vault that contains the username for UnityCloud Artifactory. Applies when srcDest Designer pipeline parameter is set to "UC_to_ACR".
`libCodeRepositoryName`	Name of the LIB-CODE repository.
`infraDevRepositoryName`	Name of the INFRA-DEV repository.
`program`	Program name.
`progRelease`	Program release.
`lcAppName`	Lowercase application name.
`nfCode`	Network function code.
`contact_dl`	Contact distribution list.
`vendor`	Vendor name.
`spID`	Service principal ID.
`spPassword`	Service principal password.
`spTenant`	Service principal tenant ID.
`siteSpecificSourceFolder`	Path for site-specific source folder in INFRA-DEV
`versionSpecificSourceFolder`	Path for version-specific source folder in INFRA-DEV
`macrovariablesSourceFolder`	Path for macro variables source folder in INFRA-DEV
`sitespecific`	Site-specific JSON file under siteSpecificSourceFolder in INFRA-DEV
`lcadoEnv`	ADO environment in lower case. Production or non-production
`group`	Name of ADO variable group. Multiple variable groups can be configured.
`versionspecific`	Name of Version-specific global JSON file under `versionSpecificSourceFolder`
`macrovariables`	Name of Macro variables YAML file under `macrovariablesSourceFolder`
`resultJson`	Path to the site-specific result JSON file

Configure Azure resource name template variables

Before running any pipelines, make sure to configure the Azure resource name template variables YAML file. This file gathers all the Azure resource name variables used across different pipelines. The JSON configuration files within each individual pipeline contain placeholders for some variables, which will be referenced from this YAML file.

By default, there are two copies of Azure resource name template variables files in the SVC-DEV repository: one for SMF and another for UPF.

- SVC-DEV/config/variables/smf-ds-az-resourcename-template-variables.yml
- SVC-DEV/config/variables/upf-ds- az-resourcename -template-variables.yml

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed.

The table below outlines the variables defined in the Azure resource name template variables file. Some of the variables are configured by default. Make sure to confirm them and update the ‘<CHANGEME>’ placeholders with appropriate values.

Table 41. az-resourcename-template-variables.yml Variables

Variables	Description
`publisherSubscriptionID`	Subscription ID of Publisher resources
`publisherRGName`	Azure Resource Group name for the publisher resources. In case of Artifacts ACR copy, it will be used as Destination publisher Resource group name
`publisherName`	Name of the publisher. In case of Artifacts ACR copy, it will be used as Destination publisher name
`sourcePublisherRGName`	Azure Resource Group name for the source publisher in case of Artifacts ACR copy.
`sourcePublisherName`	Name of the source publisher. (in case of Artifacts ACR copy)
`sourcepublisherSubscriptionId`	Subscription ID for the source publisher. (in case of Artifacts ACR copy)
`sourceartifactStoreName`	Name of the source artifact store. (in case of Artifacts ACR copy)
`sourceartifactManifestName`	Name of the source artifact manifest. (in case of Artifacts ACR copy)
`artifactStoreName`	Name of the artifact store. In the case of Artifacts ACR copy, it will be used as Destination artifact store name.
`artifactManifestName`	Name of the artifact manifest. In the case of Artifacts ACR copy, it will be used as Destination artifact manifest name.
`nfdgName`	Name of the NFDG (Network Function Definition Group).
`nsdgName`	Name of the NSDG (Network Service Design Group).
`nsdVersion`	Network Service Design Version
`nfdVersion`	Network Function Definition Version
`cnfCGSchemaName`	Name of the CNF global Configuration Group Schema
`siteSpecificCGSchemaName`	Name of the site-specific CGS.
`secretCGSchemaName`	Name of the secret CGS.
`nfTemplateVersion`	Version of the NF (Network Function) template.
`armTemplateVersion`	Version of the CNF ARM (Azure Resource Manager) template.
`armTemplateName`	Name of the CNF template within the artifact manifest

storageAccountRG	Resource Group for the Azure storage account.
azureJVSSubscription	Subscription ID for the Azure JVS storage account
azureStorageAccount	Name of the Azure storage account. Storage account (in the destination Resource Group) to which generated destination location based PRD files are to be copied to.
snsName	Name of the SNS (Site Network Service) to be created/reused/deleted by CNF operator deploy/ISSU/delete pipeline
instanceCGVName	Name of the instance CGV (Configuration Group Value) to be created/reused/deleted by CNF operator deploy/ISSU/delete pipeline.
globalCGVName	Name of the global CGV to be created/reused/deleted by CNF operator deploy/ISSU/delete pipeline.
secretCGVName	Name of the secret CGV to be created/reused/deleted by CNF operator deploy/ISSU/delete pipeline.
siteName	Name of the site to be created/reused/deleted by CNF operator deploy/ISSU/delete pipeline.

Configure the Publisher flow Pipeline

The following files need to be configured to run the Publisher flow pipeline.

- pipeline/nexus-publisher/nexus-publisher-flow-macro-pipeline.yml (SVC-DEV)
- code/nexus_publisher/nexusPublisher.json (LIB-CODE)

pipeline/nexus-publisher/nexus-publisher-flow-macro-pipeline.yml (SVC-DEV)

The table below outlines the variables used in the publisher macro pipeline file. Some of the variables are configured by default. Make sure to confirm them and update the ‘<CHANGEME>’ placeholders with appropriate values.

Table 42. nexus-publisher-macro-pipeline.yml Variables

Variables	Description
name	Name of the pipeline. Replace ‘CHANGEME’ with expected values
pool.name	Name of the Agent Pool
resources.repositories.repository	Name of the repository used across the pipeline
resources.repositories.type	Type of repository

Variables	Description
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline
`resources.containers.image`	Image name with reference to ACR
`resources.containers.endpoint`	Service connection name
`parameters.azureBranch`	LIB-CODE and SVC-DEV branch
`parameters.UCMajorRelease`	UC Major Release (Use dash instead of dot; example 4-3)
`parameters.PublisherRGAzureRegion`	Destination Publisher Azure Region (Lowercase Only)
`variables.template`	Configure two templates. One is the absolute path to the pipeline-template-variables.yml and the other is that to the az-resource-name-template-variables.yml file under config/variables folder in SVC-DEV repository. Configure the files with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed
`variables.PublisherRGAzureRegion`	Publisher Resource Group Azure Region
`variables.publisherRGAzureRegionCode`	Publisher Resource Group Azure Region code
`variables.UCMajorRelease`	UC Major Release
`variables.inputFilenexusPublisher`	Input JSON file under `inputSourceFoldernexusPublisher`. Configure the nexusPublisher.json file with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed
`variables.inputSourceFoldernexusPublisher`	Folder in LIB-CODE containing the inputFile. Remove the "/" at the end.
`variables.deployTimeout`	Time allocated for the pipeline to run
`stages.template`	Path to micro pipeline with the corresponding ADO repository

code/nexus_publisher/nexusPublisher.json (LIB-CODE)

This input file contains variables used as input for the Nexus Publisher Pipeline. By default, there are two copies of nexusPublisher.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_publisher/affirmed-ds-smf-nexusPublisher.json
- LIB-CODE/code/nexus_publisher/affirmed-ds-upf-nexusPublisher.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

The table below outlines the variables used in the nexusPublisher.json file. These variables are configured with either default values or placeholders that reference pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 43. nexusPublisher.json (LIB-CODE) Variables

Variables	Description
`publisherFolder`	Path to publisher folder relative to the `inputSourceFolder`. Do not include "/" at the end of this path.
`publisherBicepFile`	Publisher bicep file name; located in the `publisherFolder`
`publisherParametersFile`	Parameters file for the `publisherBicepFile`; located in the `publisherFolder`
`artifactStoreBicepFile`	Artifact store bicep file name; located in the `publisherFolder`
`artifactStoreParametersFile`	Parameters file for the `artifactStoreBicepFile`; located in the `publisherFolder`
`nfdgBicepFile`	NFDG bicep file name; located in the `publisherFolder`
`nfdgParametersFile`	Parameters file for the `nfdgBicepFile`; located in the `publisherFolder`
`nsdgBicepFile`	NSDG bicep file name; located in the `publisherFolder`
`nsdgParametersFile`	Parameters file for the `nsdgBicepFile`; located in the `publisherFolder`
`subscriptionId`	Subscription ID to create publisher resources
`publisherLocation`	Location of Publisher and its children artifacts
`deploymentName`	Name of Azure deployment to be used
`resourceGroup`	Azure Resource Group name where the AOSM resources are created
`publisherName`	Name of the Publisher being used for all AOSM resources
`artifactStoreName`	Name of the Artifact Store under the Publisher
`nsdgName`	Name of Network Service Definition.
`nfdgName`	Name of Network Function Definition.
`cnf`	CNF being deployed. Enter **SMF** or **UPF**.

Variables	Description
`publisherTags.correlation_id`	Correlation ID
`publisherTags.env`	Environment Type – Production or Non-production
`publisherTags.automated_by`	Name of the Automator
`publisherTags.contact_dl`	Contact distribution list
`publisherTags.app`	Name of the application
`publisherTags.prog`	Name of the program
`publisherTags.vendorName`	Name of the vendor
`publisherTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3)
`publisherTags.buildId`	Build ID
`publisherTags.requestedForEmail`	Build Requested Email
`publisherTags.definitionName`	Build Definition Name
`publisherTags.progRelease`	Program release name

Run a Nexus Publisher flow Pipeline

Prior to running a Nexus Publisher flow pipeline, verify that the prerequisite tasks are complete. See Prerequisites.

Running a Nexus Publisher Pipeline

1. Access the **Pipelines** page.
2. If pipeline is not already created,
    a. Click the **New pipeline** button
    b. Select Where is your code? To Azure DevOps
    c. Select SVC-DEV repository
    d. Configure Existing Azure Pipelines YAML file - pipeline/nexus-publisher/nexus-publisher-flow-macro-pipeline.yml
3. On the list of pipelines, select the **[your org] Nexus Publisher** pipeline.
4. Click the **Run pipeline** button.
5. On the **Run pipeline** window:
    a. Verify the correct branch is displayed or select it from the drop-down.

b. Configure the variables

c. Verify the advanced options are configured correctly.

d. (Optional) Select to enable system diagnostics.

e. Click **Run**.

Configure the Designer flow Pipeline

Designer flow pipeline involves Artifact manifest stage, ACR Copy stage and stages to create designer AOSM resources.

Nexus Artifact Manifest pipeline stage

This pipeline stage consumes prd files and creates the artifact manifest for a given artifact store based on the versions of charts and images in the prd files Artifact Manifest pipeline and their corresponding (repository)

> ***Note:*** *When pushing images to the AOSM Artifact Store's ACR using the* `cna_deploy_ inventory` *script, if a permissions issue related to az acr commands occurs, add the related Service Principal as a contributor to the AOSM Artifact Store's ACR.*
>
> *From the Azure portal, navigate to the AOSM Artifact Store's ACR. Select* ***Access Control*** *then* ***+ Add Role Assignment****.*

Nexus ACR Copy Artifacts pipeline stage

The Nexus ACR Copy Artifacts pipeline stage runs on the Agent VM or VMSS itself; not in a container. It cleans up the docker images, helm charts, and other artifacts from either a UnityCloud Artifactory (JFrog Repository) or an Azure Container Registry (inside Artifact Store) and copies them into an Azure Container Registry (inside Artifact Store). Note that because input values are different between the two sources, artifacts may only be copied from one source to the ACR destination during a run of the pipeline.

The pipeline stage consumes PRD files with the source locations of the artifacts being copied to the destination. Once this copy is complete, the pipeline generates new PRD files containing the destination locations of the copied artifacts and pushes them to the storage account. Later, these PRD files can be used as the source files for input to this pipeline.

You need the following software installed on the Agent VM or VMSS to successfully run this pipeline:

- Az cli version 2.5.1 or newer
- Jq version 1.6 or newer
- Docker version 24.0.4 or newer

- oras version 1.1.0 or newer
- Helm version 3.12 or newer

The following files need to be configured to run the Designer flow pipeline.

- pipeline/nexus-designer/nexus-designer-flow-macro-pipeline.yml (SVC-DEV)
- code/nexus_artifact_manifest/nexusArtifactManifest.json (LIB-CODE)
- code/nexus_acr_copy_artifacts/nexusAcrCopyArtifacts.json (LIB-CODE)
- code/nexus_designer/nexusDesigner.json (LIB-CODE)
- code/nexus_designer/aosm/nsd/schemas/siteSpecificCGSchema.json (LIB-CODE)
- code/nexus_designer/aosm/nsd/schemas/secretCGSchema.json (LIB-CODE)
- code/nexus_designer/aosm/nsd/schemas/cnfCGSchema.json (LIB-CODE)
- code/nexus_designer/aosm/nfd/schemas/nfDeployparameters.json (LIB-CODE)
- code/nexus_designer/aosm/nfd/values/nfdvHelmPackageInfo.json (LIB-CODE)
- code/nexus_designer/aosm/nfd/values/values.json (LIB-CODE)

pipeline/nexus-designer/nexus-designer-flow-macro-pipeline.yml (SVC-DEV)

The table below outlines the variables used in the designer macro pipeline file. Some of the variables are configured by default. Make sure to confirm them and update the ‘<CHANGEME>’ placeholders with appropriate values.

Table 44. nexus-designer-macro-pipeline.yml (INFRA-DEV) Variables

Variables	Description
`name`	Name of the pipeline. Replace ‘CHANGEME’ with expected values
`pool.name`	Name of the Agent Pool
`resources.repositories.repository`	Name of the repository used across the pipeline
`resources.repositories.type`	Type of repository
`resources.repositories.name`	Name of the actual ADO repository
`resources.repositories.ref`	Branch of the ADO repository
`resources.containers.container`	Container reference name used across the pipeline

Variables	Description
resources.containers.image	Image name with reference to ACR
resources.containers.endpoint	Service principal name
parameters.azureBranch	LIB-CODE and SVC-DEV Branch
parameters.UCMajorRelease	UC Major Release (Use dash instead of dot; example 4-3)
parameters.UCMinorRelease	UC Minor Release (example 1,2 etc.)
parameters.srcDest	'UC_to_ACR' if the source is UnityCloud Artifactory and the destination is ACR. 'ACR_to_ACR' if the source is ACR and the destination is another ACR.
parameters.sourcePublisherRGAzureRegion	Source Publisher Azure Region (Lowercase Only)
parameters.PublisherRGAzureRegion	Destination Publisher Azure Region (Lowercase Only)
variables.template	Configure two templates. One is the absolute path to the pipeline-template-variables.yml and the other is that to the az-resource-name-template-variables.yml file under config/variables folder in SVC-DEV repository. Configure the files with the prefixes 'smf' or 'upf' according to the type of CNF to be deployed.
variables.UCMajorRelease	UC Major Release
variables.UCMinorRelease	UC Minor Release
variables.PublisherRGAzureRegion	Publisher Resource Group Azure Region
variables.srcDest	Source-Destination configured in parameter
variables.sourcePublisherRGAzureRegion	Source Publisher Azure Region
variables.publisherRGAzureRegionCode	Destination Publisher Resource Group Azure Region code
variables.sourcePublisherRGAzureRegionCode	Source Publisher Resource Group Azure Region code
variables.inputFilenexusArtifactManifest	Input JSON file under `inputSourceFoldernexusArtifactManifest`. Configure the nexusArtifactManifest.json file with the prefixes 'smf' or 'upf' according to the type of CNF to be deployed
variables.inputSourceFoldernexusArtifactManifest	Folder in LIB-CODE containing the inputFile for artifact manifest
variables.deployTimeout	Time allocated for the designer pipeline to run
variables.inputFilenexusAcrCopyArtifacts	Input JSON file for ACR copy artifacts under `inputSourceFoldernexusAcrCopyArtifacts`. Configure the nexusAcrCopyArtifacts.json file with the prefixes 'smf' or 'upf' according to the type of CNF to be deployed

Variables	Description
`variables.inputSourceFoldernexusAcrCopyArtifacts`	Folder in LIB-CODE containing the inputFile for ACR copy artifacts
`variables.loadArtifactsdeployTimeout`	Time allocated for Copy Artifacts to ACR
`variables.cnfArmTemplatePush`	'true' if arm template needs to be pushed, Otherwise 'false'
`variables.chartsPush`	'true' if charts need to be copied. Otherwise 'false'
`variables.imagesPush`	'true' if images need to be copied. Otherwise 'false'
`variables.inputFilenexusDesigner`	Input JSON file for designer under `inputSourceFoldernexusDesigner`. Configure the nexusDesigner.json file with the prefixes 'smf' or 'upf' according to the type of CNF to be deployed
`variables.inputSourceFoldernexusDesigner`	Folder in LIB-CODE containing the inputFile for designer
`stages.template`	Path to micro pipeline with the corresponding ADO repository. Here, Artifact manifest, ACR copy and Designer micro pipelines are configured.

code/nexus_artifact_manifest/nexusArtifactManifest.json (LIB-CODE)

This input file contains variables used as input for Artifact manifest stage of the Nexus Designer Pipeline. By default, there are two copies of nexusArtifactManifest.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_artifact_manifest/affirmed-ds-smf-nexusArtifactManifest.json
- LIB-CODE/code/nexus_artifact_manifest/affirmed-ds-upf-nexusArtifactManifest.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

The table below outlines the variables used in the nexusArtifactManifest.json file. These variables are configured with either default values or placeholders that reference pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 45. nexusArtifactManifest.json (LIB-CODE) Variables

Variables	Description
`artifactManifestFolder`	Path relative to the inputSourceFolder. Do not include "/" at the end of this path.
`artifactManifestBicepFile`	Artifact manifest bicep file name; located in the `artifactManifestFolder`

Variables	Description
`artifactManifestParametersFile`	Parameters file for the `artifactManifestBicepFile`; located in the `artifactManifestFolder`.
`subscriptionId`	Subscription ID to create artifact manifest resources.
`resourceGroup`	Azure Resource Group name where the AOSM resources are created.
`publisherName`	Name of the Publisher being used for all AOSM resources.
`publisherLocation`	Location of Publisher and its children artifacts.
`artifactStoreName`	Name of the Artifact Store under the Publisher.
`artifactManifestName`	Name of the artifact manifest created by running this pipeline
`prdFolder`	Path in LIB-CODE to folder containing PRDs. Default value: LIB-CODE/prd.
`paasPrdFile`	File name of PAAS PRD within prdFolder.
`cnfPrdFile`	File name of CNF(SMF/UPF) PRD within prdFolder.
`armTemplateName`	Name of the CNF ARM template.
`armTemplateVersion`	Version of the ARM template.
`artifactVersionRange`	The version range of the artifacts created as part of the manifest. Edit this to match the UC version currently running.
`artifactManifestTags.correlation_id` **`artifactManifestTags.env`** **`artifactManifestTags.automated_by`** **`artifactManifestTags.contact_dl`** **`artifactManifestTags.app`** **`artifactManifestTags.prog`** **`artifactManifestTags.vendorName`** **`artifactManifestTags.UCMajorReleaseNumber`** **`artifactManifestTags.buildId`** **`artifactManifestTags.requestedForEmail`** **`artifactManifestTags.definitionName`** **`artifactManifestTags.progRelease`**	Correlation ID. Environment Type – Production or Non-production. Name of the Automator. Contact distribution list. Name of the application. Name of the program. Name of the vendor. UC Major Release Number (Use dash instead of dot; example 4-3). Build ID. Build Requested Email. Build Definition Name. Program release name.
`cnf`	CNF being deployed. Enter SMF or UPF.

code/nexus_acr_copy_artifacts/nexusAcrCopyArtifacts.json (LIB-CODE)

This input file contains variables used as input for ACR copy stage of the Nexus Designer Pipeline. By default, there are two copies of nexusAcrCopyArtifacts.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_acr_copy_artifacts/affirmed-ds-smf-nexusAcrCopyArtifacts.json
- LIB-CODE/code/nexus_acr_copy_artifacts/affirmed-ds-upf-nexusAcrCopyArtifacts.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

Place the CNF ARM template file `armTemplate.armTemplateFile` under `armTemplate.armTemplateFolder`. Refer the Sample CNF ARM template section for sample content of CNF ARM template and necessary options to be configured in the template.

Place the cna_deploy_inventory script under `cna_deploy_inventory_path`

The table below outlines the variables used in the nexusAcrCopyArtifacts.json file. These variables are configured with either default values or placeholders that reference pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 46. nexusAcrCopyArtifacts.json (LIB-CODE) Variables

Variables	Description
`sourceRegistry.keyVault.subscription`	Subscription ID of Key-Vault that contains the UnityCloud Artifactory credentials . Applies when srcDest macro variable is set to “UC_to_ACR”.
`sourceRegistry.keyVault.name`	Name of Key-Vault that contains the UnityCloud Artifactory credentials. Applies when srcDest macro variable is set to “UC_to_ACR”.
`sourceRegistry.keyVault.sourceRegistryUsernameSecretName`	Secret name in Key-Vault that contains the username for UnityCloud Artifactory. Applies when srcDest macro variable is set to “UC_to_ACR”.
`sourceRegistry.keyVault.sourceRegistryPasswordSecretName`	Secret name in Key-Vault containing the password for UnityCloud Artifactory. Applies when srcDest macro variable is set to “UC_to_ACR”.
`sourceACR.subscription`	Subscription ID where ACR exists. It contains the images and charts to be copied. Applies when srcDest macro variable is set to “ACR_to_ACR”.
`sourceACR.resourceGroup`	Resource Group where ACR exists. It contains the images and charts to be copied. Applies when srcDest macro variable is set to “ACR_to_ACR”.

Variables	Description
sourceACR.publisher	Publisher under which the Artifact Store that contains the source ACR exists. It contains the images and charts to be copied. Applies when srcDest macro variable is set to “ACR_to_ACR”.
sourceACR.artifactStore	Artifact Store containing source ACR. It contains images and charts to be copied. Applies when `srcDest` macro variable is set to “ACR_to_ACR”.
sourceACR.artifactManifest	Artifact Manifest within Artifact Store containing source ACR. It contains the images and charts to be copied. Applies when `srcDest` macro variable is set to “ACR_to_ACR”.
destinationACR.subscription	Subscription ID of the ACR to which images and charts need to be copied
destinationACR.resourceGroup	Resource Group of the ACR to which images and charts need to be copied.
destinationACR.publisher	Publisher under which the ACR exists indicating to which images and charts need to be copied.
destinationACR.artifactStore	Artifact Store of the ACR to which images and charts need to be copied.
destinationACR.artifactManifest	Artifact Manifest of Artifact Store where the ACR to which images and charts need to be copied exists.
prdFolder	Absolute folder path within `inputSourceFolder` macro variable containing PRD files which have the source images and charts information. Default value: LIB-CODE/prd
acrPrdFolder	Path to the ACR PRD folder, including major and minor release versions. Not in use.
cna_deploy_inventory_path	Relative folder path within `inputSourceFolder` macro variable where *cna_deploy_inventory* script exists. Default value: scripts
storageAccountSubscription	Subscription ID for the storage account.
storageAccountResourceGroup	Resource Group name for the storage account.
storageAccount	Storage account (in the destination Resource Group) to which generated destination location based PRD files are to be copied to.
storageFolder	Storage folder within the storage account to which generated PRD files are copied.
installMsCertScriptLink	Download URL to script which installs the necessary CA certs. Default is set to *https://uc.a4opacketcore.microsoft.com/artifactory/setup/install_ms_certs*.
armTemplate.armTemplateFolder	Relative folder path within the `inputSourceFolder` for placing ARM template file. Default values: armTemplateFolder
armTemplate.armTemplateFile	Name of CNF ARM template file under `armTemplateFolder`. Default values: “smf_nf_template_artifact.json” or “upf_nf_template_artifact.json”

Variables	Description
`armTemplate.armTemplateName`	Name of the artifact in ACR under which the ARM template is stored.
`armTemplate.armTemplateVersion`	Version of the artifact in ACR under which the ARM template is stored
`cnf`	CNF being deployed. Enter SMF or UPF.

Sample CNF ARM template

Following is the sample CNF ARM template to be placed in armTemplateFolder and to be configured as input before running ACR copy artifacts pipeline as part of designer flow.

```
{
  "$schema": "https://schema.management.azure.com/schemas/2019-04-
01/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "metadata": {
    "_generator": {
      "name": "bicep",
      "version": "0.16.1.55165",
      "templateHash": "12927928343793255573"
    }
  },
  "parameters": {
    "nfdv": {
      "type": "object"
    },
    "nfdvId": {
      "type": "string"
    },
    "smfValues": {
      "type": "object"
    },
    "siteSpecificValues": {
      "type": "object"
    },
    "secretValues": {
      "type": "secureObject"
    },
    "nfviIdValue": {
      "type": "string"
    },
    "rollbackEnabled": {
      "type": "bool",
```

```
      "defaultValue": false
    },
    "config": {
      "type": "secureObject",
      "defaultValue": "[union(parameters('smfValues'),
parameters('siteSpecificValues'), parameters('secretValues'))]"
    }
  },
  "variables": {
    "copy": [
      {
        "name": "nfAppOverrides",
        "count": "[length(parameters('nfdv').helmPackageInformation)]",
        "input": {
          "name":
"[parameters('nfdv').helmPackageInformation[copyIndex('nfAppOverrides')].name]"
,
          "deployParametersMappingRuleProfile": {
            "helmMappingRuleProfile": {
              "releaseNamespace":
"[parameters('nfdv').helmPackageInformation[copyIndex('nfAppOverrides')].releas
eNamespace]",
              "releaseName":
"[parameters('nfdv').helmPackageInformation[copyIndex('nfAppOverrides')].releas
e]",
              "helmPackageVersion":
"[parameters('nfdv').helmPackageInformation[copyIndex('nfAppOverrides')].versio
n]",
          "options": {
                "installOptions": {
                  "atomic": false,
                  "wait": true,
                  "timeout": "80",
                  "testOptions": {
                    "enable": false,
                    "timeout": "15",
                    "rollbackOnTestFailure": false
                  }
                },
                "upgradeOptions": {
                  "atomic": true,
                  "wait": true,
                  "timeout": "120",
                  "testOptions": {
```

```
                    "enable": true,
                    "timeout": "15",
                    "rollbackOnTestFailure": true
                  }
                }
              }
            }
          }
        }
      }
    ],
    "nfConfigurationOverrides": {
      "nfConfiguration": {
      "rollbackEnabled": "[parameters('rollbackEnabled')]"
    }},
    "roleOverrides": "[union(variables('nfAppOverrides'),
array(variables('nfConfigurationOverrides')))]"

  },
  "resources": [
    {
      "type": "Microsoft.HybridNetwork/networkFunctions",
      "apiVersion": "2023-09-01",
      "name": "[format('aosm_smf_test{0}', uniqueString(resourceGroup().id))]",
      "location": "[parameters('nfdv').location]",
      "identity": {
        "type": "SystemAssigned"
      },
      "properties": {
        "configurationType": "Secret",
        "copy": [
          {
            "name": "roleOverrideValues",
            "count": "[length(variables('roleOverrides'))]",
            "input":
"[string(variables('roleOverrides')[copyIndex('roleOverrideValues')])]"
          }
        ],
        "networkFunctionDefinitionVersionResourceReference": {
          "id": "[parameters('nfdvId')]",
          "idType": "Open"
        },
        "nfviType": "AzureArcKubernetes",
        "nfviId": "[parameters('nfviIdValue')]",
```

```
        "allowSoftwareUpdate": true,
        "secretDeploymentValues": "[string(parameters('config'))]"
      }
    }
  ]
}
```

Enable Rollback on CNF ISSU Failure

To enable NF level rollback during CNF upgrade failure, NF ARM template should have the changes to enable rollback on failure at both the NF App level and the entire NF level. The above CNF ARM template includes those changes.

Enable Rollback on Failure at NF App Level:

Following are the changes in the above NF ARM template to enable rollback on failure at the NF App level.

```
"options": {
  "installOptions": {
      "atomic": false,
      "wait": true,
      "timeout": "80",
      "testOptions": {
        "enable": false,
        "timeout": "15",
        "rollbackOnTestFailure": false
      }
    },
    "upgradeOptions": {
      "atomic": true,
      "wait": true,
      "timeout": "120",
      "testOptions": {
        "enable": true,
        "timeout": "15",
        "rollbackOnTestFailure": true
      }
    }
}
```

<u>Enable Rollback on Failure at the Whole NF Level:</u>

Following are the changes in the above NF ARM template to enable rollback on failure at the NF level with reconfigurable support.

1. The “rollbackEnabled” parameter is defined in the parameters section with a type of bool and a default value of false:

```
"rollbackEnabled": {
  "type": "bool",
  "defaultValue": false
}
```

2. The “rollbackEnabled” parameter is used in the “nfConfigurationOverrides” variable to create an object that includes the rollback setting:

```
"nfConfigurationOverrides": {
  "nfConfiguration": {
    "rollbackEnabled": "[parameters('rollbackEnabled')]"
  }
}
```

3. The “nfConfigurationOverrides” variable is combined with “nfAppOverrides” to form the “roleOverrides” variable:

```
"roleOverrides": "[union(variables('nfAppOverrides'),
array(variables('nfConfigurationOverrides')))]"
```

4. The “roleOverrides” variable is used in the “copy” loop within the “properties” section of the “Microsoft.HybridNetwork/networkFunctions” resource. This loop iterates over the “roleOverrides” array and includes each item in the “roleOverrideValues”:

```
"copy": [
  {
    "name": "roleOverrideValues",
    "count": "[length(variables('roleOverrides'))]",
    "input":
"[string(variables('roleOverrides')[copyIndex('roleOverrideValues')])]"
  }
]
```

code/nexus_designer/nexusDesigner.json (LIB-CODE)

This input file contains variables used as input for the Nexus Operator Pipeline. By default, there are two copies of nexusDesigner.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_designer/affirmed-ds-smf-nexusDesigner.json
- LIB-CODE/code/nexus_designer/affirmed-ds-upf-nexusDesigner.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

The table below outlines the variables used in the nexusDesigner.json file. These variables are configured with either default values or placeholders that reference pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 47. nexusDesigner.json (LIB-CODE) Variables

Variables	Description
designerFolder	Path relative to the `inputSourceFolder`. Do not include "/" at the end of this path.
nfdvBicepFile	NFDV bicep file name; located in the `designerFolder`.
nfdvParametersFile	Parameters file for the `nfdvBicepFile;` located in the `designerFolder.`
nsdvBicepFile	NSDV bicep file name; located in the `designerFolder`.
nsdvParametersFile	Parameters file for the `nsdvBicepFile`; located in the `designerFolder`.
cgsGlobalBicepFile	CGS Global bicep file name; located in the `designerFolder.`
cgsGlobalParametersFile	Parameters file for the `cgsGlobalBicepFile;` located in the `designerFolder.`
cgsInstanceBicepFile	CGS Instance bicep file name; located in the `designerFolder`.
cgsInstanceParametersFile	Parameters file for the `cgsInstanceBicepFile;` located in the `designerFolder`.
cgsSecretBicepFile	CGS Secret bicep file name; located in the `designerFolder`.
cgsSecretParametersFile	Parameters file for the `cgsSecretBicepFile`; located in the `designerFolder.`
generateHelmInfoBicepFile	Name of the bicep file that generates Helm information required for the NFDv; located in the `designerFolder`. The default name is *aosm_generate_helm_info.bicep*.
generateHelmInfoParametersFile	Parameters file corresponding to the *aosm_generate_ helm_info.bicep* file. The default name is *aosm_ generate_helm_info.parameters.json*.
subscriptionId	Subscription ID to create designer resources.

Variables	Description
resourceGroup	Azure Resource Group name where the AOSM resources are created.
publisherLocation	Location of Publisher and its children artifacts.
publisherName	Name of the Publisher being used for all AOSM resources.
artifactStoreName	Name of the Artifact Store under the Publisher.
cnfCGSchemaName	CNF CG Schema name.
siteSpecificCGSchemaName	Site-specific CG Schema name.
secretCGSchemaName	Secret CG Schema name.
siteSpecificCGSFile	Path to the site-specific CG Schema file; located in the `designerFolder`.
cnfCGSFile	Path to the CNF CG Schema file; located in the `designerFolder`.
secretCGSFile	Path to the secret CG Schema file; located in the `designerFolder.`
nsdgName	Name of Network Service Design Group.
nsdVersion	Version of Network Service Design.
nfdgName	Name of Network Function Definition Group.
nfdVersion	Version of Network Function Definition.
nfTemplateVersion	Version of Network Function template.
nfdvSchemaFile	The relative path from the `designerFolder` to the NFDv schema file.
nfdvHelmPackageInfoFile	The relative path from the `designerFolder` to the helm package information file. It is dynamically filled based on the prd files, so no input is required.
nfdvValuesFile	The relative path from the `designerFolder` to the Values json file containing the values for the CNF.
prdFolder	Path in LIB-CODE to folder containing PRDs for upgrade version. Default value: LIB-CODE/prd
paasPrdFile	File name of PAAS PRD within prdFolder.
cnfPrdFile	File name of CNF(SMF/UPF) PRD within prdFolder
artifactVersionRange	Artifact version range.
cnf	CNF being deployed. Enter SMF or UPF.

Variables	Description
`cgsTags.correlation_id`	Correlation ID.
`cgsTags.env`	Environment Type – Production or Non-production.
`cgsTags.automated_by`	Name of the Automator.
`cgsTags.contact_dl`	Contact distribution list.
`cgsTags.app`	Name of the application.
`cgsTags.prog`	Name of the program.
`cgsTags.vendorName`	Name of the vendor.
`cgsTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`cgsTags.buildId`	Build ID.
`cgsTags.requestedForEmail`	Build Requested Email.
`cgsTags.definitionName`	Build Definition Name.
`cgsTags.progRelease`	Program release name.
`nfdvTags.correlation_id`	Correlation ID.
`nfdvTags.env`	Environment Type – Production or Non-production.
`nfdvTags.automated_by`	Name of the Automator.
`nfdvTags.contact_dl`	Contact distribution list.
`nfdvTags.app`	Name of the application.
`nfdvTags.prog`	Name of the program.
`nfdvTags.vendorName`	Name of the vendor.
`nfdvTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`nfdvTags.buildId`	Build ID.
`nfdvTags.requestedForEmail`	Build Requested Email.
`nfdvTags.definitionName`	Build Definition Name.
`nfdvTags.progRelease`	Program release name.

Variables	Description
`nsdvTags.correlation_id`	Correlation ID.
`nsdvTags.env`	Environment Type – Production or Non-production.
`nsdvTags.automated_by`	Name of the Automator.
`nsdvTags.contact_dl`	Contact distribution list.
`nsdvTags.app`	Name of the application.
`nsdvTags.prog`	Name of the program.
`nsdvTags.vendorName`	Name of the vendor.
`nsdvTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`nsdvTags.buildId`	Build ID.
`nsdvTags.requestedForEmail`	Build Requested Email.
`nsdvTags.definitionName`	Build Definition Name.
`nsdvTags.progRelease`	Program release name.

code/nexus_designer/aosm/nsd/schemas/siteSpecificCGSchema.json (LIB-CODE)

Place the site specific configuration schema JSON for the given CNF in this file.

code/nexus_designer/aosm/nsd/schemas/secretCGSchema.json (LIB-CODE)

Place the secret configuration schema JSON for the given CNF in this file.

The **values.json** file is used to create the NFDv in LIB-CODE.

To update parameters as secrets in the values.json file:

1. Follow the same procedure to add the parameters required as secrets to the **secretCGSchema.json** file as you did with the **siteSpecificCGSchema.json** file.

2. Update the corresponding values of parameters in the **values.json (NFDv)** to `{deployParameters.<parameter-name-fromsecretCGSchema>}`.

3. Add the list of secret parameters to the **code/nexus_designer/aosm/nfd/schemas/nfDeployparameters.json** file. This is the **nfdvSchemaFile**.

4. Add the list of secret parameters and their values to the **secretCGV.json** file.

5. Upload the new ARM template file.

code/nexus_designer/aosm/nsd/schemas/cnfCGSchema.json (LIB-CODE)

Place the CNF specific (SMF or UPF) global configuration schema JSON for the given CNF in this file.

By default, there are two copies of cnfCGSchema.json: one for SMF and another for UPF.

- LIB-CODE/code/nexus_designer/aosm/nsd/schemas/smfCGSchema.json
- LIB-CODE/code/nexus_designer/aosm/nsd/schemas/upfCGSchema.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

code/nexus_designer/aosm/nfd/schemas/nfDeployparameters.json (LIB-CODE)

Place the Configuration group schema JSON for NFDv in this file.

code/nexus_designer/aosm/nfd/values/nfdvHelmPackageInfo.json (LIB-CODE)

This JSON file contains the NFDv helm package information. This is automatically filled at run time by the pipeline; no input is required.

code/nexus_designer/aosm/nfd/values/values.json (LIB-CODE)

Place CNF-specific values JSON in this file.

Run a Nexus Designer flow Pipeline

Prior to running a Nexus Designer flow pipeline, verify that the prerequisite tasks are complete. See Prerequisites.

Running a Nexus Designer Pipeline

1. Access the **Pipelines** page.
2. If pipeline is not already created,

a. Click the **New pipeline** button

b. Select Where is your code? To Azure DevOps

c. Select SVC-DEV repository

d. Configure Existing Azure Pipelines YAML file - pipeline/nexus-designer/nexus-designer-flow-macro-pipeline.yml

3. On the list of pipelines, select the **[your org] Nexus Designer** pipeline.

4. Click the **Run pipeline** button.

5. On the **Run pipeline** window:

   a. Verify the correct branch is displayed or select it from the drop-down.

   b. Configure the variables

   c. Verify the advanced options are configured correctly.

   d. (Optional) Select to enable system diagnostics.

   e. Click **Run**.

Configure the Nexus CNF Operator Pipeline

Using the json key value pairs in the nexusCnfOperator.json file, you can tag specific AOSM resources, including the Site Network Service, CNF CGV, and Site Specific CGV resources, as part of the CNF Operator Pipeline. Refer to the following tables for specific information.

Use the procedure below to complete the required prerequisites to run the pipelines to perform CNF Deploy.

1. Create the site-specific macro variables YAML file under the config folder in the INFRA repository in the format “config/smf/${{ parameters.site_name }}/csmf_ds_${{ parameters.PublisherRGAzureRegion }}_nprd_z${{ parameters.site_name }}csmf${{ parameters.instanceNumber }}_macrovariables.yml”.

   For example, INFRA-DEV/config/smf/site1/csmf_ds_eastus_nprd_zsite1csmf01_macrovariables.yml

2. Configure the file as template under variables section in Operator Deploy Macro pipeline.

3. Create a site-specific result json file adjacent to site-specific macro variables YAML file in the INFRA repository. For example, INFRA-DEV/config/smf/site1/result.json

4. Configure site-specific result json file in the site-specific macro variables YAML file as follows:

- name: sitespecific_json

  value: 'result.json'

5. While populating site-specific result json file, make sure that it has the following set of variables defined.

   a. Variables like,

      "zone": "zsite1csmf01",

      "deploymentName": "smf-site1",

      "resourceGroup": "rg-1",

      "customlocationId": "/subscriptions/subscriptionId/resourceGroups/rg-1/providers/Microsoft.ExtendedLocation/customLocations/cl-1",

      "nfdOfferingLocation": "eastus",

      "paramAlias": "csmf1-site1",

      "cnfCGSchemaName": "csmf-nprd-cnf-global-cgs-01",

      "siteSpecificCGSchemaName": "csmf-nprd-cnf-instance-cgs-01",

      "secretCGSchemaName": "csmf-nprd-cnf-secret-cgs-01",

      "nsdgName": "csmf-nprd-nsdg-01",

      "nsdVersion": "5.1.0-8",

      "nfdgName": "csmf-nprd-nfdg-01",

      "nfdVersion": "5.1.0-8",

      "naksClusterName": "cluster-1",

      "naksClusterNameRG": "cluster-1-rg",

Configure the following files in the Nexus CNF Operator Pipeline and the corresponding repository (LIB-CODE or SVC-DEV):

- pipeline/nexus-cnf_operator/nexus-cnf-operator-macro-pipeline.yml (SVC-DEV)
- code/nexus_cnf_operator/nexusCnfOperator.json (LIB-CODE)
- code/nexus_cnf_operator/aosm/cnf_nsd_values/siteSpecificCGV.json (LIB-CODE)
- code/nexus_cnf_operator/aosm/cnf_nsd_values/cnfCGV.json (LIB-CODE)

- code/nexus_cnf_operator/aosm/cnf_nsd_values/secretCGV.json@LIB-CODE

The following sections show the configurable files in the Nexus CNF Operator Pipeline and their corresponding (repository).

pipeline/nexus-cnf_operator/nexus-cnf-operator-macro-pipeline.yml (SVC-DEV)

The table below outlines the variables used in the CNF operator macro pipeline file. Some of the variables are configured by default. Make sure to confirm them and update the ‘<CHANGEME>’ placeholders with appropriate values.

Table 48. nexus-cnf-operator-macro-pipeline.yml Variables

Variables	Description
`name`	Name of the pipeline. Replace ‘CHANGEME’ with expected values
`pool.name`	Name of the Agent Pool.
`resources.repositories.repository`	Name of the repository used across the pipeline.
`resources.repositories.type`	Type of repository.
`resources.repositories.name`	Name of the actual ADO repository.
`resources.repositories.ref`	Branch of the ADO repository.
`resources.containers.container`	Container reference name used across the pipeline.
`resources.containers.image`	Image name with reference to ACR.
`resources.containers.endpoint`	Service principal name.
`parameters.site_name`	Site Name.
`parameters.instanceNumber`	NF Instance Number (e.g., 01, 02).
`parameters.requestor_id`	The attuid of the person running the Config-It Pipeline run.
`parameters.azureBranch`	LIB-CODE, SVC-DEV and INFRA-DEV Branch.
`parameters.UCMajorRelease`	UC Major Release (Use dash instead of dot; example 4-3).
`parameters.UCMinorRelease`	UC Minor Release (example 1, 2, etc.).
`parameters.NfRGAzureRegion`	NF Azure Region (Lowercase Only).
`parameters.PublisherRGAzureRegion`	Publisher Azure Region (Lowercase Only).

Variables	Description
`variables.template`	Configure three templates. One is the absolute path to the site specific macro variables YAML file under the config folder in the INFRA repository. The other two are the absolute paths to the pipeline-template-variables.yml and az-resourcename-template-variables.yml files under the config/variables folder in the SVC-DEV repository. Configure the files with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed.
`variables.UCMajorRelease`	UC Major Release.
`variables.UCMinorRelease`	UC Minor Release.
`variables.PublisherRGAzureRegion`	Publisher Resource Group Azure Region.
`variables.site_name`	Site Name.
`variables.instanceNumber`	NF Instance Number.
`variables.networkServiceLocation`	NF Azure Region.
`variables.publisherRGAzureRegionCode`	Destination Publisher Resource Group Azure Region code.
`variables.inputFile`	Input JSON file for the CNF Operator. Configure the nexusCnfOperator.json file with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed.
`variables.inputSourceFolder`	Folder in LIB-CODE containing the inputFile for CNF Operator.
`variables.deployTimeout`	Time allocated for the SNS Deploy to run.
`variables.cnfOperation`	CNF Operation (e.g., deploy).
`stages.template`	Path to the micro pipeline with the corresponding ADO repository.

code/nexus_cnf_operator/nexusCnfOperator.json (LIB-CODE)

This input file contains variables used as input for the Nexus Operator Pipeline. By default, there are two copies of nexusCnfOperator.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperator.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperator.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

The table below outlines the variables used in the nexusCnfOperator.json file. These variables are configured with either default values or placeholders that reference pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 49. nexusCnfOperator.json Variables

Variables	Description
`operatorFolder`	Path relative to the inputSourceFolder. Do not include "/" at the end of this path.
`siteBicepFile`	Site bicep file name; located in the `operatorFolder`.
`siteParametersFile`	Parameters file for the `siteBicepFile;` located in the `operatorFolder`.
`snsBicepFile`	SNS bicep file name; located in the `operatorFolder`.
`snsParametersFile`	Parameters file for the `snsBicepFile`; located in the `operatorFolder`.
`cgvGlobalBicepFile`	CGV Global bicep file name; located in the `operatorFolder`.
`cgvGlobalParametersFile`	Parameters file for the `cgvGlobalBicepFile`; located in the `operatorFolder`.
`cgvInstanceBicepFile`	CGV Instance bicep file name; located in the `operatorFolder`.
`cgvInstanceParametersFile`	Parameters file for the `cgvInstanceBicepFile`; located in the `operatorFolder`.
`cgvSecretBicepFile`	CGV Secret bicep file name; located in the `operatorFolder`.
`cgvSecretParametersFile`	Parameters file for the `cgvSecretBicepFile`; located in the `operatorFolder`.
`generateHelmInfoBicepFile`	Name of the bicep file that generates Helm information required for the CNF; located in the `operatorFolder`. The default name is aosm_generate_helm_info.bicep.
`generateHelmInfoParametersFile`	Parameters file corresponding to the aosm_generate_helm_info.bicep file. The default name is aosm_generate_helm_info.parameters.json.
`siteSpecificCGVFile`	Path to the site-specific CGV file; located in the `operatorFolder`.
`secretCGVFile`	Path to the secret CGV file; located in the `operatorFolder`.
`cnfCGVFile`	Path to the CNF CGV file; located in the `operatorFolder`.
`subscriptionId`	Subscription ID to create operator resources.
`resourceGroup`	Azure Resource Group name where the AOSM resources are created.
`publisherName`	Name of the Publisher being used for all AOSM resources.
`publisherLocation`	Location of Publisher and its children artifacts.
`publisherSubscriptionId`	Subscription ID of the Publisher.
`publisherResourceGroup`	Resource Group name of the Publisher.
`networkServiceLocation`	Location of the network service.
`cnfCGSchemaName`	CNF CG Schema name.
`siteSpecificCGSchemaName`	Site-specific CG Schema name.
`secretCGSchemaName`	Secret CG Schema name.

Variables	Description
`nsdgName`	Name of Network Service Design Group.
`nsdVersion`	Version of Network Service Design.
`customlocationId`	Custom location ID of the Arc-enabled Kubernetes cluster on which the CNF is deploy.
`nfdgName`	Name of Network Function Definition Group.
`nfdVersion`	Version of Network Function Definition.
`paramAlias`	Parameter alias.
`prdFolder`	Path in LIB-CODE to folder containing PRDs for upgrade version. Default value: LIB-CODE/prd.
`paasPrdFile`	File name of PAAS PRD within prdFolder.
`cnfPrdFile`	File name of CNF(SMF/UPF) PRD within prdFolder.
`artifactVersionRange`	The version range of helm packages allowed in the artifacts. Edit this to match the UC version currently running.
`naksClusterName`	Name of the NAKS cluster.
`naksClusterNameRG`	Resource Group name of the NAKS cluster.
`snsName`	Name of the SNS.
`instanceCGVName`	Name of the instance CGV.
`secretCGVName`	Name of the secret CGV.
`globalCGVName`	Name of the global CGV.
`siteName`	Name of the site.

`snsTags.mots_id`
`snsTags.env`
`snsTags.created_by`
`snsTags.contact_dl`
`snsTags.NFType`
`snsTags.aonInstance`
`snsTags.nfInstance`
`snsTags.vendor`
`snsTags.swVersion`
`snsTags.publisher`
	Tagged with Site Network Service (SNS).
	Correlation ID.
	Environment Type – Production or Non-production.
	Name of the Automator.
	Contact distribution list.
	Type of Network Function.
	AON Instance name.
	NF Instance name.
	Name of the vendor.
	Software version.
	Name of the Publisher.

Variables	Description
	Tagged with CNF Configuration Group Version:
`cnfCGVTags.mots_id`	Correlation ID.
`cnfCGVTags.env`	Environment Type – Production or Non-production.
`cnfCGVTags.created_by`	Name of the Automator.
`cnfCGVTags.contact_dl`	Contact distribution list.
`cnfCGVTags.NFType`	Type of Network Function.
`cnfCGVTags.aonInstance`	AON Instance name.
`cnfCGVTags.nfInstance`	NF Instance name.
`cnfCGVTags.vendor`	Name of the vendor.
`cnfCGVTags.swVersion`	Software version.
`cnfCGVTags.publisher`	Name of the Publisher.
	Tagged with site specific Configuration Group Value:
`siteSpecificCGVTags.mots_id`	Correlation ID.
`siteSpecificCGVTags.env`	Environment Type – Production or Non-production.
`siteSpecificCGVTags.created_by`	Name of the Automator.
`siteSpecificCGVTags.contact_dl`	Contact distribution list.
`siteSpecificCGVTags.NFType`	Type of Network Function.
`siteSpecificCGVTags.aonInstance`	AON Instance name.
`siteSpecificCGVTags.nfInstance`	NF Instance name.
`siteSpecificCGVTags.vendor`	Name of the vendor.
`siteSpecificCGVTags.swVersion`	Software version.
`siteSpecificCGVTags.publisher`	Name of the Publisher.
	Tagged with site:
`siteTags.mots_id`	Correlation ID.
`siteTags.env`	Environment Type – Production or Non-production.
`siteTags.created_by`	Name of the Automator.
`siteTags.contact_dl`	Contact distribution list.
`siteTags.NFType`	Type of Network Function.
`siteTags.aonInstance`	AON Instance name.
`siteTags.nfInstance`	NF Instance name.
`siteTags.vendor`	Name of the vendor.
`siteTags.swVersion`	Software version.
`siteTags.publisher`	Name of the Publisher.
`keyvaultName`	Name of the Key Vault.
`cnf`	CNF being deployed. Enter SMF or UPF.

code/nexus_cnf_operator/aosm/cnf_nsd_values/siteSpecificCGV.json (LIB-CODE)

Configure the site- or cluster-specific configuration group variables that are defined in the Network Function Definition version (NFDv) in this file.

code/nexus_cnf_operator/aosm/cnf_nsd_values/cnfCGV.json (LIB-CODE)

Configure the CNF-specific configuration group values for the CNF being deployed in this file.

code/nexus_cnf_operator/aosm/cnf_nsd_values/secretCGV.json@LIB-CODE

Configure the Configuration Group Values (CGV) for the secret Configuration Group Schema (CGS) in this file.

> ***Note:*** *While the keys in secretValues represent the parameters made as secrets, the actual key values must be Azure Key-Vault secret names. The Azure Key-Vault values replace the secretValues parameters when you run the CNF Operator pipeline.*

Run a Nexus CNF Operator Pipeline

Prior to running a Nexus Operator Deploy pipeline, verify that the prerequisite tasks are complete. See Prerequisites.

Running a Nexus Operator Pipeline

1. Access the **Pipelines** page.
2. If pipeline is not already created,
    a. Click the **New pipeline** button
    b. Select Where is your code? To Azure DevOps
    c. Select SVC-DEV repository
    d. Configure Existing Azure Pipelines YAML file - pipeline/nexus-operator/nexus-operator-macro-pipeline.yml
3. On the list of pipelines, select the **[your org] Nexus Operator** pipeline.

4. Click the **Run pipeline** button.

5. On the **Run pipeline** window:

    a. Verify the correct branch is displayed or select it from the drop-down.

    b. Configure the variables

    c. Verify the advanced options are configured correctly.

    d. (Optional) Select to enable system diagnostics.

    e. Click **Run**.

Configure the Nexus CNF Delete Pipeline

This pipeline deletes a CNF via deletion of the related Site Network Service (SNS) resource. The following sections show the configurable files in the Nexus CNF Delete pipeline and their corresponding (repository).

- pipeline/gmacro/cnf-operations-delete-macro-pipeline.yml (SVC-DEV)
- code/nexus_cnf_operator/nexusCnfOperatorDelete.json (LIB-CODE)

pipeline/nexus-cnf_operator/nexus-cnf-operator-delete-macro-pipeline.yml (SVC-DEV)

The table below outlines the variables used in the CNF operator macro pipeline file. Some of the variables are configured by default. Make sure to confirm them and update the ‘<CHANGEME>’ placeholders with appropriate values.

Table 50. nexus-cnf-operator-delete-macro-pipeline.yml Variables

Variables	Description
`name`	Name of the pipeline. Replace ‘CHANGEME’ with expected values
`pool.name`	Name of the Agent Pool.
`resources.repositories.repository`	Name of the repository used across the pipeline.
`resources.repositories.type`	Type of repository.
`resources.repositories.name`	Name of the actual ADO repository.
`resources.repositories.ref`	Branch of the ADO repository.
`resources.containers.container`	Container reference name used across the pipeline.

Variables	Description
`resources.containers.image`	Image name with reference to ACR.
`resources.containers.endpoint`	Service principal name.
`parameters.site_name`	Site Name.
`parameters.instanceNumber`	NF Instance Number (e.g., 01, 02).
`parameters.requestor_id`	The attuid of the person running the Config-It Pipeline run.
`parameters.azureBranch`	LIB-CODE, SVC-DEV, and INFRA-DEV Branch.
`parameters.UCMajorRelease`	UC Major Release (Use dash instead of dot; example 4-3).
`parameters.UCMinorRelease`	UC Minor Release (example 1, 2, etc.).
`variables.template`	Configure three templates. One is the absolute path to the site-specific macro variables YAML file under the config folder in the INFRA repository. The other two are the absolute paths to the pipeline-template-variables.yml and az-resourcename-template-variables.yml files under the config/variables folder in the SVC-DEV repository. Configure the files with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed.
`variables.UCMajorRelease`	UC Major Release.
`variables.UCMinorRelease`	UC Minor Release.
`variables.site_name`	Site Name.
`variables.instanceNumber`	NF Instance Number.
`variables.inputFile`	Input JSON file for the CNF Operator Delete. Configure the nexusCnfOperatorDelete.json file with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed.
`variables.inputSourceFolder`	Folder in LIB-CODE containing the inputFile for CNF Operator Delete.
`variables.deployTimeout`	Time allocated for the SNS Delete to run.
`variables.cnfOperation`	CNF Operation (e.g., delete).
`stages.template`	Path to the micro pipeline with the corresponding ADO repository.

code/nexus_cnf_operator/nexusCnfOperatorDelete.json (LIB-CODE)

This input file contains variables used as input for the Nexus Operator Delete Pipeline. By default, there are two copies of nexusCnfOperatorDelete.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperatorDelete.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperatorDelete.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

The table below outlines the variables used in the nexusCnfOperatorDelete.json file. These variables are configured with either default values or placeholders that reference pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 51. nexusCnfOperatorDelete.json (LIB-CODE)

Variables	Description
`resourceGroup`	Azure Resource Group name from which SNS resource will be deleted.
`paramAlias`	Name of the CNF SNS to be deleted. Similar to its use in the Nexus CNF Deploy pipeline, the alias is an identifier with the format `<paramAlias>-smf-sitenetworkservice`
`cnf`	The CNF to be deleted. Enter smf or upf.
`snsName`	Name of the SNS.
`instanceCGVName`	Name of the instance CGV.
`secretCGVName`	Name of the secret CGV.
`globalCGVName`	Name of the global CGV.
`siteName`	Name of the site.

Run a Nexus CNF Operator Delete Pipeline

Prior to running a Nexus Operator Delete pipeline, verify that the prerequisite tasks are complete. See Prerequisites.

Running a Nexus Operator Delete Pipeline

1. Access the **Pipelines** page.
2. If pipeline is not already created,
    a. Click the **New pipeline** button
    b. Select Where is your code? To Azure DevOps
    c. Select SVC-DEV repository
    d. Configure Existing Azure Pipelines YAML file - pipeline/nexus-operator/nexus-operator-delete-macro-pipeline.yml
3. On the list of pipelines, select the **[your org] Nexus Operator Delete** pipeline.
4. Click the **Run pipeline** button.

5. On the **Run pipeline** window:

    a. Verify the correct branch is displayed or select it from the drop-down.

    b. Configure the variables

    c. Verify the advanced options are configured correctly.

    d. (Optional) Select to enable system diagnostics.

    e. Click **Run**.

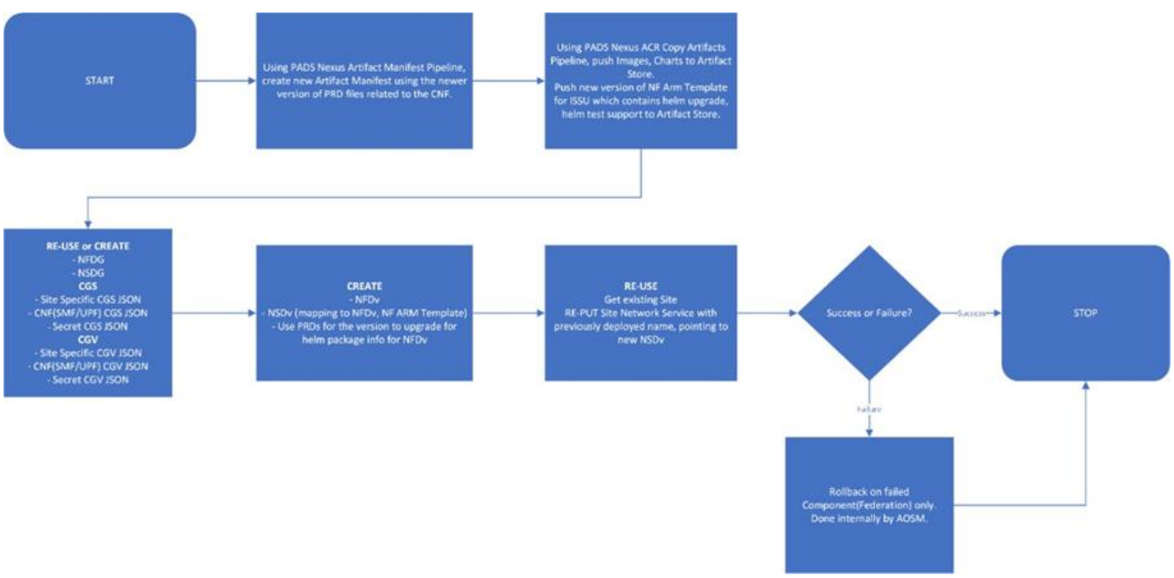
Nexus CNF Operator ISSU Pipeline

This pipeline uses AOSM APIs to upgrade a Site Network Service (CNF) on the Nexus Azure Kubernetes cluster to a newer version using a helm upgrade and helm test . Note that it is only supported if the CNF is capable of an In Service Software Upgrade.

To use this pipeline, an existing Site Network Service (SNS) must have been deployed using the PADS Nexus CNF Operator Pipeline. The pipeline requires the Upgrade NF ARM Template for the CNF to be pushed to the Artifact Store. Additionally, the upgrade version's images and charts must be pushed to the Artifact Store. This can be accomplished using the ACR Copy Artifacts Pipeline.

The following diagram provides a high-level overview of this pipeline:

Figure 10. ISSU Pipeline Overview



For CNF In Service Software Upgrade (ISSU), PADS will either reuse or create the following AOSM Artifacts:

- Site Specific CGS

- CNF (SMF/UPF) Specific CGS
- Secret CGS

PADS will also create new or newer versions of the following AOSM Artifacts:

- NF ARM Template
- NFDv
- NSDv
- Site Specific CGV
- CNF (SMF/UPF) CGV
- Secret CGV

PADS will reuse the existing Site and re-deploy on the Site Network Service, which will point to the new NSDv (NSDv points to NFDv and the new NF ARM Template). Once the AOSM artifacts are created, the CNF should be successfully upgraded or rolled back to the previous version if the upgrade fails.

Use the procedure below to complete the required prerequisites to run the pipelines to perform CNF ISSU.

1. Ensure that CNF is already deployed and running successfully using CNF Operator Deploy or CNF Operator ISSU pipeline

2. Create the site-specific macro variables YAML file under the config folder in the INFRA repository in the format “config/smf/${{ parameters.site_name }}/csmf_ds_${{ parameters.PublisherRGAzureRegion }}_nprd_z${{ parameters.site_name }}csmf${{ parameters.instanceNumber }}_macrovariables.yml”.

   For example, INFRA-DEV/config/smf/site1/csmf_ds_eastus_nprd_zsite1csmf01_macrovariables.yml

3. Configure the file as template under variables section in Operator ISSU Macro pipeline.

4. Create a site-specific result json file adjacent to site-specific macro variables YAML file in the INFRA repository. For example, INFRA-DEV/config/smf/site1/result.json

5. Configure site-specific result json file in the site-specific macro variables YAML file as follows:

   ```
   - name: sitespecific_json
     value: 'result.json'
   ```

6. While populating site-specific result json file, make sure that it has the following set of variables defined.

   a. Variables like,

"zone": "zsite1csmf01",

"deploymentName": "smf-site1",

"resourceGroup": "rg-1",

"customlocationId": "/subscriptions/subscriptionId/resourceGroups/rg-1/providers/Microsoft.ExtendedLocation/customLocations/cl-1",

"nfdOfferingLocation": "eastus",

"paramAlias": "csmf1-site1",

"cnfCGSchemaName": "csmf-nprd-cnf-global-cgs-01",

"siteSpecificCGSchemaName": "csmf-nprd-cnf-instance-cgs-01",

"secretCGSchemaName": "csmf-nprd-cnf-secret-cgs-01",

"nsdgName": "csmf-nprd-nsdg-01",

"nsdVersion": "5.1.0-8",

"nfdgName": "csmf-nprd-nfdg-01",

"nfdVersion": "5.1.0-8",

"naksClusterName": "cluster-1",

"naksClusterNameRG": "cluster-1-rg",

b. In case of using existing AOSM resources,

"nsdgNewOrExisting": "existing",

"cnfCGSnewOrExisting": "existing",

"siteSpecificCGSnewOrExisting": "existing",

"secretCGSnewOrExisting": "existing",

"siteSpecificCGVnewOrExisting": "existing",

"siteSpecificCGVName": "",

"secretCGVnewOrExisting": "existing",

"secretCGVName": "",

"cnfCGVnewOrExisting": "existing",

"cnfCGVName": ""

Here, names should be set empty. The existing AOSM CGV resources names will be taken from Azure resource name template variables file.

c. In case of creating new AOSM resources, (Configure the names as needed)

"nsdgNewOrExisting": "new",

"cnfCGSnewOrExisting": "new",

"siteSpecificCGSnewOrExisting": "new",

"secretCGSnewOrExisting": "new",

"siteSpecificCGVnewOrExisting": "new",

"siteSpecificCGVName": "csmf1-site1-cnf-instance-cgv-01-new",

"secretCGVnewOrExisting": "new",

"secretCGVName": "csmf1-site1-cnf-secret-cgv-01-new",

"cnfCGVnewOrExisting": "new",

"cnfCGVName": "csmf1-site1-cnf-global-cgv-01-new",

7. Configure the Nexus Designer pipeline for newer version

8. To enable NF level rollback during CNF upgrade failure, ensure that NF ARM template has the changes to enable rollback on failure at both the NF App level and the entire NF level. Refer Sample CNF ARM template.

9. Run only following stages of Nexus Designer pipeline

    a. Artifact Manifest

    b. ACR Copy

10. Configure the Nexus CNF Operator ISSU pipeline

11. Run the Nexus CNF Operator ISSU pipeline

The following sections detail the files to modify in order to run this pipeline.

Configure the Nexus CNF Operator ISSU pipeline

The following files need to be configured to run the CNF Operator ISSU pipeline.

- pipeline/nexus-cnf_operator/nexus-cnf-operator-issu-macro-pipeline.yml (SVC-DEV)

- code/nexus_cnf_operator_issu/nexusCnfOperatorIssu.json (LIB-CODE)

pipeline/nexus-cnf_operator/nexus-cnf-operator-issu-macro-pipeline.yml (SVC-DEV)

The following table shows the variables used in the CNF Operator ISSU macro pipeline.

Table 52. cnf-operator-issu-macro-pipeline.yml (SVC-DEV) Variables

Variables	Description
`name`	Name of the pipeline. Replace 'CHANGEME' with expected values
`pool.name`	Name of the Agent Pool.
`resources.repositories.repository`	Name of the repository used across the pipeline.
`resources.repositories.type`	Type of repository.
`resources.repositories.name`	Name of the actual ADO repository.
`resources.repositories.ref`	Branch of the ADO repository.
`resources.containers.container`	Container reference name used across the pipeline.
`resources.containers.image`	Image name with reference to ACR.
`resources.containers.endpoint`	Service principal name.
`parameters.site_name`	Site Name.
`parameters.instanceNumber`	NF Instance Number (e.g., 01, 02).
`parameters.requestor_id`	The attuid of the person running the Config-It Pipeline run.
`parameters.azureBranch`	LIB-CODE, SVC-DEV, and INFRA-DEV Branch.
`parameters.UCMajorRelease`	UC Major Release (Use dash instead of dot; example 4-3).
`parameters.UCMinorRelease`	UC Minor Release (example 1, 2, etc.).
`parameters.NfRGAzureRegion`	NF Azure Region (Lowercase Only).
`parameters.PublisherRGAzureRegion`	Publisher Azure Region (Lowercase Only).
`variables.template`	Configure absolute paths to three templates. 1. site-specific macro variables YAML file under the config folder in the INFRA repository. 2. config/variables/pipeline-template-variables.yml@SVC-DEV 3. config/variables/az-resourcename-template-variables.yml@SVC-DEV Configure the files with the prefixes 'smf' or 'upf' according to the type of CNF to be deployed.
`variables.UCMajorRelease`	UC Major Release.
`variables.UCMinorRelease`	UC Minor Release.
`variables.PublisherRGAzureRegion`	Publisher Resource Group Azure Region.

Variables	Description
`variables.site_name`	Site Name.
`variables.instanceNumber`	NF Instance Number.
`variables.networkServiceLocation`	NF Azure Region.
`variables.publisherRGAzureRegionCode`	Destination Publisher Resource Group Azure Region code.
`variables.inputFile`	Input JSON file for the CNF Operator ISSU. Configure the nexusCnfOperatorIssu.json file with the prefixes ‘smf’ or ‘upf’ according to the type of CNF to be deployed.
`variables.inputSourceFolder`	Folder in LIB-CODE containing the inputFile for CNF Operator ISSU.
`variables.upgradeTimeout`	Time allocated for the SNS ISSU to run.
`variables.cnfOperation`	CNF Operation (e.g., ISSU).
`stages.template`	Path to the micro pipeline with the corresponding ADO repository.

code/nexus_cnf_operator_issu/nexusCnfOperatorIssu.json (LIB-CODE)

This input file contains variables used as input for the Nexus Operator ISSU Pipeline. By default, there are two copies of nexusCnfOperatorIssu.json: one for SMF and another for UPF. The default values in these files are aligned to the CNF type indicated by their prefixes.

- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperatorIssu.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperatorIssu.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

The table below outlines the variables used in the nexusCnfOperatorIssu.json file. These variables are configured with either default values or placeholders that reference site-specific macro variables, site-specific result json params, pipeline template variables, Azure resource name variables, and macro pipeline variables. Modify them as needed.

Table 53. nexusCnfOperatorIssu.json (LIB-CODE) Variables

Variables	Description
`operatorIssuFolder`	Path relative to the `inputSourceFolder`. Do not include "/" at the end of this path. The default is **aosm**.
`nsdgBicepFile`	NSDG bicep file name; located in the `operatorIssuFolder`.
`nsdgParametersFile`	Parameters file for the `nsdgBicepFile`; located in the `operatorIssuFolder`.

Variables	Description
`nfdvBicepFile`	NFDV bicep file name; located in the `operatorIssuFolder`.
`nfdvParametersFile`	Parameters file for the `nfdvBicepFile`; located in the `operatorIssuFolder`.
`nsdvBicepFile`	NSDV bicep file name; located in the `operatorIssuFolder`.
`nsdvParametersFile`	Parameters file for the `nsdvBicepFile`; located in the `operatorIssuFolder`.
`cgsGlobalBicepFile`	CGS Global bicep file name; located in the `operatorIssuFolder`.
`cgsGlobalParametersFile`	Parameters file for the `cgsGlobalBicepFile`; located in the `operatorIssuFolder`.
`cgsInstanceBicepFile`	CGS Instance bicep file name; located in the `operatorIssuFolder`.
`cgsInstanceParametersFile`	Parameters file for the `cgsInstanceBicepFile`; located in the `operatorIssuFolder`.
`cgsSecretBicepFile`	CGS Secret bicep file name; located in the `operatorIssuFolder`.
`cgsSecretParametersFile`	Parameters file for the `cgsSecretBicepFile`; located in the `operatorIssuFolder`.
`cgvGlobalBicepFile`	CGV Global bicep file name; located in the `operatorIssuFolder`.
`cgvGlobalParametersFile`	Parameters file for the `cgvGlobalBicepFile`; located in the `operatorIssuFolder`.
`cgvInstanceBicepFile`	CGV Instance bicep file name; located in the `operatorIssuFolder`.
`cgvInstanceParametersFile`	Parameters file for the `cgvInstanceBicepFile`; located in the `operatorIssuFolder`.
`cgvSecretBicepFile`	CGV Secret bicep file name; located in the `operatorIssuFolder`.
`cgvSecretParametersFile`	Parameters file for the `cgvSecretBicepFile`; located in the `operatorIssuFolder`.
`snsBicepFile`	SNS bicep file name; located in the `operatorIssuFolder`.
`snsParametersFile`	Parameters file for the `snsBicepFile`; located in the operatorIssuFolder.
`generateHelmInfoBicepFile`	Name of the bicep file that generates Helm information required for the CNF; located in the `operatorIssuFolder`. The default name is aosm_generate_helm_info.bicep.
`generateHelmInfoParametersFile`	Parameters file corresponding to the aosm_generate_helm_info.bicep file. The default name is aosm_generate_helm_info.parameters.json.
`siteSpecificCGVFile`	Path to the site-specific CGV file; located in the `operatorIssuFolder`.
`secretCGVFile`	Path to the secret CGV file; located in the `operatorIssuFolder`.
`cnfCGVFile`	Path to the CNF CGV file; located in the `operatorIssuFolder`.
`subscriptionId`	Subscription ID of the existing site network service (SNS).
`resourceGroup`	Resource group name of the site network service (SNS).

Variables	Description
`publisherName`	Name of the publisher being used for all AOSM artifacts related to the SNS.
`publisherLocation`	Location of Publisher and its children artifacts.
`publisherSubscriptionId`	Subscription ID of the Publisher.
`publisherResourceGroup`	Resource Group name of the Publisher.
`networkServiceLocation`	Location of the network service.
`nsdgNewOrExisting`	Define whether to use an **existing** or to create a **new** Network Service Definition Group (NSDG). By default, referenced from site specific result json file.
`nsdgName`	If you are creating a **new** NSDG, enter the name of the NSDG. By default, referenced from site specific result json file.
`nfdgName`	Enter the name of the NFDG. Only existing NFDGs may be used to create a new NFDv. By default, referenced from site specific result json file.
`cnfCGSnewOrExisting`	Define whether to use an **existing** or to create a **new** CNF Configuration Group Schema (CGS). By default, referenced from site specific result json file.
`cnfCGSchemaName`	If you are creating a **new** CNF CGS, enter the name of the CNF CGS. By default, referenced from site specific result json file.
`siteSpecificCGSnewOrExisting`	Define whether to use an **existin**g or to create a **new** Site Specific Configuration Group Schema (CGS). By default, referenced from site specific result json file.
`siteSpecificCGSchemaName`	If you are creating a **new** CGS, enter the name of the site specific CGS. By default, referenced from site specific result json file.
`secretCGSnewOrExisting`	Define whether to use an **existing** or to create a **new** secret Configuration Group Schema (CGS). By default, referenced from site specific result json file.
`secretCGSchemaName`	If you are creating a **new** secret CGS, enter the name of the secret CGS. By default, referenced from site specific result json file.
`siteSpecificCGVnewOrExisting`	Define whether to use an **existing** or to create a **new** Site Specific Configuration Group Value (CGV). By default, referenced from site specific result json file.
`siteSpecificCGVName`	If you are creating a **new** site specific CGV, enter the site specific CGV name. By default, referenced from site specific result json file.
`secretCGVnewOrExisting`	Define whether to use an **existing** or to create a **new** secret Configuration Group Value (CGV). By default, referenced from site specific result json file.
`secretCGVName`	If you are creating a **new** `secretCGVFile`, enter the secret CGV Name. By default, referenced from site specific result json file.

Variables	Description
`cnfCGVnewOrExisting`	Indicates if CNF CGV is **new** or **existing**. By default, referenced from site specific result json file.
`cnfCGVName`	If you are creating a **new** cnf CGV, enter the CNF CGV Name. By default, referenced from site specific result json file.
`prdFolder`	Path in LIB-CODE to folder containing PRDs for upgrade version. Default value: LIB-CODE/prd.
`paasPrdFile`	File name of PAAS PRD within prdFolder.
`cnfPrdFile`	File name of CNF(SMF/UPF) PRD within prdFolder.
`artifactVersionRange`	The version range of helm packages allowed in the artifacts. Edit this to match the UC version currently running.
`nfdVersion`	Version of Network Function Definition.
`nfdvSchemaFile`	The relative path from the `operatorIssuFolder` to the NFDv schema file.
`nfdvHelmPackageInfoFile`	The relative path from the `operatorIssuFolder` to the helm package information file.
`nfdvValuesFile`	The relative path from the `operatorIssuFolder` to the Values json file containing the values for the CNF.
`nsdVersion`	Version of Network Service Design.
`enableRollbackOnISSUFailure`	Flag to indicate whether to enable rollback on CNF ISSU failure or not. By default, referenced from pipeline variables file in SVC-DEV
`paramAlias`	Parameter alias used for CNF Operator Deployment. These entries must be identical. By default, referenced from site specific result json file.
`artifactStoreName`	Name of the Artifact Store under the Publisher.
`nfTemplateVersion`	Version of Network Function template.
`keyvaultName`	Name of the Key Vault.
`customlocationId`	Custom location ID of the Arc-enabled Kubernetes cluster on which the CNF is deployed. By default, referenced from site specific result json file.
`naksClusterName`	Name of the NAKS cluster. By default, referenced from site specific result json file.
`naksClusterNameRG`	Resource Group name of the NAKS cluster. By default, referenced from site specific result json file.
`cnf`	CNF being deployed. Enter SMF or UPF.
`snsName`	Name of the existing SNS. By default, referenced from Operator ISSU macro.
`existingInstanceCGVName`	Name of the existing instance CGV. By default, referenced from Operator ISSU macro.

Variables	Description
existingSecretCGVName	Name of the existing secret CGV. By default, referenced from Operator ISSU macro.
existingGlobalCGVName	Name of the existing global CGV. By default, referenced from Operator ISSU macro.
existingSiteName	Name of the existing site. By default, referenced from Operator ISSU macro.
	Tagged with Site Network Service (SNS).
snsTags.mots_id	Correlation ID.
snsTags.env	Environment Type – Production or Non-production.
snsTags.created_by	Name of the Automator.
snsTags.contact_dl	Contact distribution list.
snsTags.NFType	Type of Network Function.
snsTags.aonInstance	AON Instance name.
snsTags.nfInstance	NF Instance name.
snsTags.vendor	Name of the vendor.
snsTags.swVersion	Software version.
snsTags.publisher	Name of the Publisher.
	Tagged with CNF Configuration Group Version:
cnfCGVTags.mots_id	Correlation ID.
cnfCGVTags.env	Environment Type – Production or Non-production.
cnfCGVTags.created_by	Name of the Automator.
cnfCGVTags.contact_dl	Contact distribution list.
cnfCGVTags.NFType	Type of Network Function.
cnfCGVTags.aonInstance	AON Instance name.
cnfCGVTags.nfInstance	NF Instance name.
cnfCGVTags.vendor	Name of the vendor.
cnfCGVTags.swVersion	Software version.
cnfCGVTags.publisher	Name of the Publisher.

Variables	Description
	Tagged with site specific Configuration Group Value:
`siteSpecificCGVTags.mots_id`	Correlation ID.
`siteSpecificCGVTags.env`	Environment Type – Production or Non-production.
`siteSpecificCGVTags.created_by`	Name of the Automator.
`siteSpecificCGVTags.contact_dl`	Contact distribution list.
`siteSpecificCGVTags.NFType`	Type of Network Function.
`siteSpecificCGVTags.aonInstance`	AON Instance name.
`siteSpecificCGVTags.nfInstance`	NF Instance name.
`siteSpecificCGVTags.vendor`	Name of the vendor.
`siteSpecificCGVTags.swVersion`	Software version.
`siteSpecificCGVTags.publisher`	Name of the Publisher.
	Tagged with Configuration Group Schema (CGS):
`cgsTags.correlation_id`	Correlation ID.
`cgsTags.env`	Environment Type – Production or Non-production.
`cgsTags.automated_by`	Name of the Automator.
`cgsTags.contact_dl`	Contact distribution list.
`cgsTags.app`	Name of the application.
`cgsTags.prog`	Name of the program.
`cgsTags.vendorName`	Name of the vendor.
`cgsTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`cgsTags.buildId`	Build ID.
`cgsTags.requestedForEmail`	Build Requested Email.
`cgsTags.definitionName`	Build Definition Name.
`cgsTags.progRelease`	Program release name.
	Tagged with Network Function Definition Version (NFDV):
`nfdvTags.correlation_id`	Correlation ID.
`nfdvTags.env`	Environment Type – Production or Non-production.
`nfdvTags.automated_by`	Name of the Automator.
`nfdvTags.contact_dl`	Contact distribution list.
`nfdvTags.app`	Name of the application.
`nfdvTags.prog`	Name of the program.
`nfdvTags.vendorName`	Name of the vendor.
`nfdvTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`nfdvTags.buildId`	Build ID.
`nfdvTags.requestedForEmail`	Build Requested Email.
`nfdvTags.definitionName`	Build Definition Name.
`nfdvTags.progRelease`	Program release name.

Variables	Description
	Tagged with Network Service Design Version (NSDV)
`nsdvTags.correlation_id`	Correlation ID.
`nsdvTags.env`	Environment Type – Production or Non-production.
`nsdvTags.automated_by`	Name of the Automator.
`nsdvTags.contact_dl`	Contact distribution list.
`nsdvTags.app`	Name of the application.
`nsdvTags.prog`	Name of the program.
`nsdvTags.vendorName`	Name of the vendor.
`nsdvTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`nsdvTags.buildId`	Build ID.
`nsdvTags.requestedForEmail`	Build Requested Email.
`nsdvTags.definitionName`	Build Definition Name.
`nsdvTags.progRelease`	Program release name.
	Tagged with Network Service Design Group (NSDG)
`nsdgTags.correlation_id`	Correlation ID.
`nsdgTags.env`	Environment Type – Production or Non-production.
`nsdgTags.automated_by`	Name of the Automator.
`nsdgTags.contact_dl`	Contact distribution list.
`nsdgTags.app`	Name of the application.
`nsdgTags.prog`	Name of the program.
`nsdgTags.vendorName`	Name of the vendor.
`nsdgTags.UCMajorReleaseNumber`	UC Major Release Number (Use dash instead of dot; example 4-3).
`nsdgTags.buildId`	Build ID.
`nsdgTags.requestedForEmail`	Build Requested Email.
`nsdgTags.definitionName`	Build Definition Name.
`nsdgTags.progRelease`	Program release name.

code/nexus_cnf_operator_issu/aosm/nsd/schemas/siteSpecificCGSchema.json (LIB-CODE)

Place the site specific configuration schema JSON for the given CNF in this file.

code/nexus_cnf_operator_issu/aosm/nsd/schemas/secretCGSchema.json (LIB-CODE)

Place the secret configuration schema JSON for the given CNF in this file.

The **values.json** file is used to create the NFDv in LIB-CODE.

To update parameters as secrets in the values.json file:

1. Follow the same procedure to add the parameters required as secrets to the **secretCGSchema.json** file as you did with the **siteSpecificCGSchema.json** file.

2. Update the corresponding values of parameters in the **values.json (NFDv)** to `{deployParameters.<parameter-name-fromsecretCGSchema>}`.

3. Add the list of secret parameters to the **code/nexus_cng_operator_issu/aosm/nfd/schemas/nfDeployparameters.json** file. This is the **nfdvSchemaFile**.

4. Add the list of secret parameters and their values to the **secretCGV.json** file.

5. Upload the new ARM template file.

code/nexus_cnf_operator_issu /aosm/nsd/schemas/cnfCGSchema.json (LIB-CODE)

Place the CNF specific (SMF or UPF) global configuration schema JSON for the given CNF in this file.

By default, there are two copies of cnfCGSchema.json: one for SMF and another for UPF.

- LIB-CODE/code/nexus_cnf_operator_issu/aosm/nsd/schemas/smfCGSchema.json
- LIB-CODE/code/nexus_cnf_operator_issu/aosm/nsd/schemas/upfCGSchema.json

Update the corresponding file with the prefix 'smf' or 'upf' depending on the type of CNF to be deployed and configure the path appropriately in the macro pipeline.

code/nexus_cnf_operator_issu/aosm/nfd/schemas/nfDeployparameters.json (LIB-CODE)

Place the Configuration group schema JSON for NFDv in this file.

code/nexus_cnf_operator_issu/aosm/nfd/values/nfdvHelmPackageInfo.json (LIB-CODE)

This JSON file contains the NFDv helm package information. This is automatically filled at run time by the pipeline; no input is required.

code/nexus_cnf_operator_issu/aosm/nfd/values/values.json (LIB-CODE)

Place CNF-specific values JSON in this file.

code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/siteSpecificCGV.json (LIB-CODE)

Configure the site- or cluster-specific configuration group variables that are defined in the Network Function Definition version (NFDv) in this file.

code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/cnfCGV.json (LIB-CODE)

Configure the CNF-specific configuration group values for the CNF being deployed in this file.

code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/secretCGV.json @LIB-CODE

Configure the Configuration Group Values (CGV) for the secret Configuration Group Schema (CGS) in this file.

> ***Note:*** *While the keys in secretValues represent the parameters made as secrets, the actual key values must be Azure Key-Vault secret names. The Azure Key-Vault values replace the secretValues parameters when you run the CNF Operator pipeline.*

Run a Nexus CNF Operator ISSU Pipeline

Prior to running a Nexus Operator ISSU pipeline, verify that the prerequisite tasks are complete. See "Nexus CNF Operator ISSU Pipeline"

Running a Nexus Operator ISSU Pipeline

1. Access the **Pipelines** page.
2. If pipeline is not already created,
    a. Click the **New pipeline** button

b. Select Where is your code? To Azure DevOps

c. Select SVC-DEV repository

d. Configure Existing Azure Pipelines YAML file - pipeline/nexus-operator-issu/nexus-operator-issu-macro-pipeline.yml

3. On the list of pipelines, select the **[your org] Nexus Operator ISSU** pipeline.

4. Click the **Run pipeline** button.

5. On the **Run pipeline** window:

    a. Verify the correct branch is displayed or select it from the drop-down.

    b. Configure the variables

    c. Verify the advanced options are configured correctly.

    d. (Optional) Select to enable system diagnostics.

    e. Click **Run**.

Operations and Maintenance

Some of the ADO packages required to run the ADO tasks in the pipelines are not available by default and you must obtain them from the Visual Studio marketplace. Use the following links to install the specific packages:

- JSON to Variable - task that reads JSON file in your build agent to generate Build/Release variables to be used in other steps
- Replace Tokens - extension that replaces tokens in text files with variable values
- Variable Toolbox - Azure Pipelines tasks for setting and expanding variables

When running the Pipelines for the first time, you must give explicit permission to access other repositories like LIB-CODE.

Figure 11. Release Preview

> ***Note:*** *This is only needed for the initial run. All subsequent runs of the same pipeline will re- use the granted permissions from the first run.*

If any pods or services fail to come up, use the following commands to debug and get logs:

- List all pods and their status
  - → **`kubectl get pods -A`**
- Describe the status of a specific pod in more detail. Usually used to look into a failed pod.
  - → **`kubectl describe pod <pod> -n <namespace>`**
- List all services and their status
  - → **`kubectl get svc -A`**

- Describe the status of a specific service in more detail. Usually used to look into a failed service.
  - → **kubectl describe svc <service> -n <namespace>**

When there is an error in a pipeline with any of the operations (OPM-related) in the following table and there is no error message related to the error, use the following command to examine the logs of the OPM on the OPM VM:

kubectl logs <api-gw-pod> -n vnfmgr

The following table shows the OPM-related operations in which an error may occur without displaying a message related to the error, along with the strings you can search for to identify the reasons for the errors.

Table 54. NB API Strings to Debug

NB API	String to help debug
Attaching pre-existing Cluster (Finalize K8s Cluster Pipeline)	`attachExistingCluster`
Import NF Config Set and Snapshot (CNF Deploy Pipeline)	`importNFConfigSet`
Bulk Installation (CNF Deploy Pipeline)	`installNetworkFunctions`
Get All Federation Status (CNF Deploy Pipeline)	`getAllFedStatus`
Get Single Federation pods running status (CNF Deploy Pipeline)	`retrieveFedPodsStatus`

Install PADS Software

Prerequisites

Verify that the following prerequisite tasks are complete prior to installing the PADS software:

- Extract the **`pads_pipeline-<version>-<build_number>.tgz file.`** Extract the **`pipeline_artifacts.tar.gz`** file inside **`pads_pipeline-<version>-<build_number>/`**.

  → The results should show the two directories –networkcloud/ and nexus/ and third party bom and PRD files.

  → Inside networkcloud and nexus directories, there should be 3 directories LIB-CODE/, INFRA-DEV/ and SVC-DEV/.

- Confirm that the following repositories exist on the ADO project: LIB-CODE, INFRA-DEV, and SVC-DEV.
- Confirm that Service Connections to the following exist on the ADO project: Azure Resource Group and Azure Container Registry.

Steps to Install

Network cloud

LIB-CODE

1. Copy the contents of the networkcloud/*LIB-CODE/* directory extracted from pipeline_artifacts.tar.gz into the LIB-CODE ADO repository.

2. Update the *code/acr_opm_image_copy/acrToOpmCopy.json* file with appropriate values.

3. Update the *code/finalize_cluster/finalizeK8sCluster.json* file with appropriate values.

4. Update the *code/opm_automation/opmAutomation.json* file with appropriate values.

5. Update the *code/nexus_cnf_operator/nexusCnfOperator.json* file with appropriate values.

6. Update the following files with the appropriate files.

   - *code/cnf_operations/cnf-operations-deploy.json*
   - *code/cnf_operations/cnf-operations-delete.json*

- *code/cnf_operations/cnf-operations-upgrade.json*
- *code/cnf_operations/cnf-operations-status.json*

7. Add the *cna_deploy_inventory* script file to a folder in LIB-CODE and refer to it in the appropriate variables.

> ***Note:*** *UnityCloud version 4.1 of the cna_deploy_inventory script must be used.*

8. Add the *snapshotFile* value and *snapshot_specific_values.json* files to LIB-CODE and refer to them in the appropriate variables.

INFRA-DEV

1. Copy the contents of the INFRA-DEV/ directory extracted from pipeline_artifacts.tar.gz into the INFRA-DEV ADO repository.

2. Update the values in the following macro pipeline files: *acr-to-opm-image-copy-macropipeline.yml, finalizeK8sCluster-macro-pipeline.yml, opm-automation-macro-pipeline.yml, pipelineLocking-macro-pipeline.yml, and csoSoftwareInstall-macro-pipeline.yml.*

SVC-DEV

1. Copy the contents of the SVC-DEV/ directory extracted from pipeline_artifacts.tar.gz into the SVC-DEV ADO repository.

2. Update the values in the macro pipeline files: *cnf-operations-delete-macro-pipeline.yml, cnfoperations-deploy-macro-pipeline.yml, cnf-operations-upgrade-macro-pipeline.yml, and cnfoperations-status-macro-pipeline.yml*.

Nexus

LIB-CODE

1. Copy the contents of the nexus/*LIB-CODE/* directory extracted from pipeline_artifacts.tar.gz into the LIB-CODE ADO repository.
2. The following configuration files are already updated with default values. Update them if required.
    - LIB-CODE/code/nexus_acr_copy_artifacts/affirmed-ds-smf-nexusAcrCopyArtifacts.json
    - LIB-CODE/code/nexus_acr_copy_artifacts/affirmed-ds-upf-nexusAcrCopyArtifacts.json

- LIB-CODE/code/nexus_artifact_manifest/affirmed-ds-smf-nexusArtifactManifest.json
- LIB-CODE/code/nexus_artifact_manifest/affirmed-ds-upf-nexusArtifactManifest.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperator.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperatorDelete.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperator.json
- LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperatorDelete.json
- LIB-CODE/code/nexus_cnf_operator_issu/affirmed-ds-smf-nexusCnfOperatorIssu.json
- LIB-CODE/code/nexus_cnf_operator_issu/affirmed-ds-upf-nexusCnfOperatorIssu.json
- LIB-CODE/code/nexus_designer/affirmed-ds-smf-nexusDesigner.json
- LIB-CODE/code/nexus_designer/affirmed-ds-upf-nexusDesigner.json
- LIB-CODE/code/nexus_publisher/affirmed-ds-smf-nexusPublisher.json
- LIB-CODE/code/nexus_publisher/affirmed-ds-upf-nexusPublisher.json

3. Update the following files with appropriate values.
    - code/nexus_designer/aosm/nfd/schemas/nfDeployparameters.json
    - code/nexus_designer/aosm/nfd/values/values.json
    - code/nexus_designer/aosm/nsd/schemas/secretCGSchema.json
    - code/nexus_designer/aosm/nsd/schemas/siteSpecificCGSchema.json
    - code/nexus_designer/aosm/nsd/schemas/smfCGSchema.json
    - code/nexus_designer/aosm/nsd/schemas/upfCGSchema.json
    - code/nexus_cnf_operator/aosm/cnf_nsd_values/cnfCGV.json
    - code/nexus_cnf_operator/aosm/cnf_nsd_values/secretCGV.json
    - code/nexus_cnf_operator/aosm/cnf_nsd_values/siteSpecificCGV.json
    - code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/cnfCGV.json
    - code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/secretCGV.json
    - code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/siteSpecificCGV.json
    - code/nexus_cnf_operator_issu/aosm/nfd/schemas/nfDeployparameters.json

- code/nexus_cnf_operator_issu/aosm/nfd/values/values.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/secretCGSchema.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/siteSpecificCGSchema.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/smfCGSchema.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/upfCGSchema.json

4. Place the PRD files under LIB-CODE/prd folder and configure them in appropriate pipeline-template-variables.yml file under SVC-DEV
5. Place the CNF ARM template file (smf_nf_template_artifact.json or upf_nf_template_artifact.json) under LIB-CODE/code/nexus_acr_copy_artifacts/armTemplateFolder
6. Place the *cna_deploy_inventory* script file under LIB-CODE/code/nexus_acr_copy_artifacts/scripts

INFRA-DEV

1. nexus/INFRA-DEV folder extracted from pipeline_artifacts.tar.gz will be empty.
2. Create the following files and configure them appropriately in operator macro files.
    - Site specific macro variables file
    - Site specific result json configuration file
    - CNF version specific json configuration

SVC-DEV

1. Copy the contents of the nexus/SVC-DEV/ directory extracted from pipeline_artifacts.tar.gz into the SVC-DEV ADO repository.
2. Update the values in the following variables files:
    - SVC-DEV/config/variables/smf-ds-az-resourcename-template-variables.yml
    - SVC-DEV/config/variables/smf-ds-pipeline-template-variables.yml
    - SVC-DEV/config/variables/upf-ds-az-resourcename-template-variables.yml
    - SVC-DEV/config/variables/upf-ds-pipeline-template-variables.yml
3. Update the values in the following macro pipeline files:

- SVC-DEV/pipeline/nexus-publisher/nexus-publisher-flow-macro-pipeline.yml
- SVC-DEV/pipeline/nexus-designer/nexus-designer-flow-macro-pipeline.yml
- SVC-DEV/pipeline/nexus-cnf_operator/nexus-cnf-operator-macro-pipeline.yml
- SVC-DEV/pipeline/nexus-cnf_operator/nexus-cnf-operator-issu-macro-pipeline.yml
- SVC-DEV/pipeline/nexus-cnf_operator/nexus-cnf-operator-delete-macro-pipeline.yml

Upgrade PADS Software

Prerequisites

Verify that the following prerequisite tasks are complete prior to installing the PADS software:

- Extract the **`pads_pipeline-<version>-<build_number>.tgz file.`** Extract the **`pipeline_artifacts.tar.gz`** file inside **`pads_pipeline-<version>-<build_number>/`**.
    - → The results should show the two directories –networkcloud/ and nexus/ and third party bom and PRD files.
    - → Inside networkcloud and nexus directories, there should be 3 directories LIB-CODE/, INFRA-DEV/ and SVC-DEV/.
- Confirm that the following repositories exist on the ADO project: LIB-CODE, INFRA-DEV, and SVC-DEV.
- Confirm that Service Connections to the following exist on the ADO project: Azure Resource Group and Azure Container Registry.

Steps to Upgrade

Complete the following steps in each ADO repository to upgrade.

Network cloud

LIB-CODE

1. Override all files in pipeline/ with the contents of networkcloud/LIB-CODE/pipeline/ directory extracted from pipeline_artifacts.tar.gz.

2. Add new variables to *code/acr_opm_image_copy/acrToOpmCopy.json*.

    - Use LIB-CODE/code/acr_opm_image_copy/acrToOpmCopy.json extracted from pipeline_artifacts.tar.gz for reference.

3. Add new variables to *code/finalize_cluster/finalizeK8sCluster.json*.

    - Use LIB-CODE/code/finalize_cluster/finalizeK8sCluster.json extracted from pipeline_artifacts.tar.gz for reference.

4. Add new variables to *code/opm_automation/opmAutomation.json*.

- Use LIB-CODE/code/opm_automation/opmAutomation.json extracted from pipeline_artifacts.tar.gz for reference.

5. Add new variables to *code/nexus_cnf_operator/nexusCnfOperator.json*.

- Use LIB-CODE/code/nexus_cnf_operator/nexusCnfOperator.json extracted from pipeline_artifacts.tar.gz for reference.

6. Update the following files with appropriate variables from the Affirmed Networks Packaging and Deployment Solutions Guide. Use the corresponding LIB-CODE/ directory files for reference.

- *code/cnf_operations/cnf-operations-deploy.json*
- *code/cnf_operations/cnf-operations-delete.json*
- *code/cnf_operations/cnf-operations-upgrade.json*
- *code/cnf_operations/cnf-operations-status.json*

7. Add the *cna_deploy_inventory* script file to a folder in LIB-CODE and refer to the script file.

> ***Note:*** *UnityCloud version 4.1 of the cna_deploy_inventory script must be used.*

8. Add the PRD files to *code/acr_opm_image_copy* and refer to the PRD files.

INFRA-DEV

Update the values in the following macro pipeline files: *acr-to-opm-image-copy-macropipeline.yml, finalizeK8sCluster-macro-pipeline.yml, opm-automation-macro-pipeline.yml, pipelineLocking-macro-pipeline.yml, and csoSoftwareInstall-macro-pipeline.yml*.

SVC-DEV

Update the values in the macro pipeline files: *cnf-operations-delete-macro-pipeline.yml, cnfoperations-deploy-macro-pipeline.yml, cnf-operations-upgrade-macro-pipeline.yml, and cnfoperations-status-macro-pipeline.yml*.

Nexus

LIB-CODE

1. Take a backup of LIB-CODE ADO repository and update the files based on this backup.
2. Overwrite the contents of the nexus/*LIB-CODE/* directory extracted from pipeline_artifacts.tar.gz into the LIB-CODE ADO repository.
3. The following configuration files are already updated with default values. Update them if required.
    - LIB-CODE/code/nexus_acr_copy_artifacts/affirmed-ds-smf-nexusAcrCopyArtifacts.json
    - LIB-CODE/code/nexus_acr_copy_artifacts/affirmed-ds-upf-nexusAcrCopyArtifacts.json
    - LIB-CODE/code/nexus_artifact_manifest/affirmed-ds-smf-nexusArtifactManifest.json
    - LIB-CODE/code/nexus_artifact_manifest/affirmed-ds-upf-nexusArtifactManifest.json
    - LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperator.json
    - LIB-CODE/code/nexus_cnf_operator/affirmed-ds-smf-nexusCnfOperatorDelete.json
    - LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperator.json
    - LIB-CODE/code/nexus_cnf_operator/affirmed-ds-upf-nexusCnfOperatorDelete.json
    - LIB-CODE/code/nexus_cnf_operator_issu/affirmed-ds-smf-nexusCnfOperatorIssu.json
    - LIB-CODE/code/nexus_cnf_operator_issu/affirmed-ds-upf-nexusCnfOperatorIssu.json
    - LIB-CODE/code/nexus_designer/affirmed-ds-smf-nexusDesigner.json
    - LIB-CODE/code/nexus_designer/affirmed-ds-upf-nexusDesigner.json
    - LIB-CODE/code/nexus_publisher/affirmed-ds-smf-nexusPublisher.json
    - LIB-CODE/code/nexus_publisher/affirmed-ds-upf-nexusPublisher.json
4. Update the following files with appropriate values using backup
    - code/nexus_designer/aosm/nfd/schemas/nfDeployparameters.json
    - code/nexus_designer/aosm/nfd/values/values.json
    - code/nexus_designer/aosm/nsd/schemas/secretCGSchema.json
    - code/nexus_designer/aosm/nsd/schemas/siteSpecificCGSchema.json
    - code/nexus_designer/aosm/nsd/schemas/smfCGSchema.json

- code/nexus_designer/aosm/nsd/schemas/upfCGSchema.json
- code/nexus_cnf_operator/aosm/cnf_nsd_values/cnfCGV.json
- code/nexus_cnf_operator/aosm/cnf_nsd_values/secretCGV.json
- code/nexus_cnf_operator/aosm/cnf_nsd_values/siteSpecificCGV.json
- code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/cnfCGV.json
- code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/secretCGV.json
- code/nexus_cnf_operator_issu/aosm/cnf_nsd_values/siteSpecificCGV.json
- code/nexus_cnf_operator_issu/aosm/nfd/schemas/nfDeployparameters.json
- code/nexus_cnf_operator_issu/aosm/nfd/values/values.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/secretCGSchema.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/siteSpecificCGSchema.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/smfCGSchema.json
- code/nexus_cnf_operator_issu/aosm/nsd/schemas/upfCGSchema.json

5. Place the PRD files under LIB-CODE/prd folder and configure them in appropriate pipeline-template-variables.yml file under SVC-DEV
6. Place the CNF ARM template file (smf_nf_template_artifact.json or upf_nf_template_artifact.json) under LIB-CODE/code/nexus_acr_copy_artifacts/armTemplateFolder
7. Place the *cna_deploy_inventory* script file under LIB-CODE/code/nexus_acr_copy_artifacts/scripts

> ***Note:*** *UnityCloud version 4.1 of the cna_deploy_inventory script must be used.*

INFRA-DEV

1. nexus/INFRA-DEV folder extracted from pipeline_artifacts.tar.gz will be empty
2. Take a backup of INFRA-DEV ADO repository and update the files in next steps based on this backup.
3. Make INFRA-DEV ADO repo empty.
4. Create the following files and configure them appropriately in operator macro files in INFRA-DEV
    - Site specific macro variables file

- Site specific result json configuration file
- CNF version specific json configuration

SVC-DEV

1. Take a backup of INFRA-DEV ADO repository and update the files in next steps based on this backup.
2. Overwrite the contents of the nexus/SVC-DEV/ directory extracted from pipeline_artifacts.tar.gz into the SVC-DEV ADO repository.
3. Update the values in the following variables files.
    - SVC-DEV/config/variables/smf-ds-az-resourcename-template-variables.yml
    - SVC-DEV/config/variables/smf-ds-pipeline-template-variables.yml
    - SVC-DEV/config/variables/upf-ds-az-resourcename-template-variables.yml
    - SVC-DEV/config/variables/upf-ds-pipeline-template-variables.yml
4. Update the values in the following macro pipeline files:
    - SVC-DEV/pipeline/nexus-publisher/nexus-publisher-flow-macro-pipeline.yml
    - SVC-DEV/pipeline/nexus-designer/nexus-designer-flow-macro-pipeline.yml
    - SVC-DEV/pipeline/nexus-cnf_operator/nexus-cnf-operator-macro-pipeline.yml
    - SVC-DEV/pipeline/nexus-cnf_operator/nexus-cnf-operator-issu-macro-pipeline.yml
    - SVC-DEV/pipeline/nexus-cnf_operator/nexus-cnf-operator-delete-macro-pipeline.y

HashiCorp Vault Integration with MNOPM

Overview

The integration of HashiCorp Vault with MNOPM (Multi node OPM) federation is a strategic initiative aimed at enhancing the security and management of sensitive information within the Unity Cloud solution from UC 5.2 release. This integration focuses on migrating sensitive data such as usernames, passwords, connection strings, and certificates from local storage to the HashiCorp Vault, ensuring secure and centralized management of secrets.

Key Features

- **Vault Configuration**: Helm specific value JSON allowing users to enable or disable the HashiCorp Vault, set the server URL, define roles, and specify authentication methods.
- **Sensitive Data Management**: All sensitive data used in the fed-opm_multi (multi-node) deployment, including credentials and certificates, will be stored in the HashiCorp Vault. This eliminates plaintext sensitive data from Helm values and replaces them with vault path references.
- **Automatic Secret Rotation**: OPM will manage the rotation of secrets configured in the Vault server automatically, reducing the need for manual intervention.
- **Exclusions**: Single Node OPM is excluded from this integration due to its lightweight nature and the impracticality of using Kube add-ons and a running K8 cluster.

Goals

- Enhance security by centralizing the management of sensitive information.
- Ensure compliance with security standards by automating secret rotation and management.

This integration is part of the broader effort to improve the security and efficiency of the Unity Cloud solution, providing a robust framework for managing sensitive information across federations.

Prerequisites

To ensure a smooth integration of HashiCorp Vault with OPM, the following pre-requisites must be met:

- **Vault Server Management**: Each customer will manage their own HashiCorp Vault server.
- **Centralized Vault Server**: A centralized (external) Vault server will be used, with multiple Kubernetes clusters interacting with it.

- **Authentication Method**: JWT authentication method with static keys is preferred.
- **Secret Setup**: OPM secret information must be set up in the Vault before deployment.
- **PKI Issuer**: Vault will act as a PKI Issuer, and cert-manager will use secret-less authentication to interact with it.
- **Vault Availability and Configuration**: The availability and configuration of the Vault server will be managed by the customer's HashiCorp Vault team.

Steps to Integrate

Once the prerequisites are met, follow these steps in each repository to integrate the HashiCorp Vault server.

Network cloud Deployment

LIB-CODE

1. Vault configurations can be enabled by keeping the HashiCorp server settings in the helm specific input json file

   LIB-CODE/code/helm_operation/helm-specific/helm_specific_values-template.json

   Ensure the following variables are included with existing variables:

   - "enable-global-vault": "<CHANGE-ME>"
   - "enable-hashicorp-vault": "<CHANGE-ME>"
   - "hashicorp-vault-server-url": "<CHANGE-ME>"

HashiCorp Vault server setting variables

Variables	Description
`enable-global-vault`	To enable the global vault option for the charts. Default is false.
`enable-hashicorp-vault`	To enable the HashiCorp Vault to read the secrets from the vault, by default false
`hashicorp-vault-server-url`	HashiCorp Vault server URL, basically vault server endpoint. For example, http://fed-hc-vault-test-svc-internal.fed-hc-vault-test:8200
`role`	HashiCorp Vault role to access secrets, default is reader role (Optional)
`authMountPath`	HashiCorp Vault Auth mount path, default is jwt (Optional)
`vaultNamespace`	HashiCorp Vault namespace, default is empty (Optional)

2. Ensure these tokens are configured in the respective values YAML of the required federations, from which credentials need to be read from the HashiCorp Vault.

    - **\LIB-CODE\code\helm_operation\Values\fed-opm_multi.yaml**
    - **\LIB-CODE\code\helm_operation\Values\fed-paas_helpers.yaml**
    - **\LIB-CODE\code\helm_operation\Values\fed-kube_addons.yaml**

    Ensure specify the HashiCorp server setting parameters in the values YAML file as outlined below.

```
vault:
  # Global switch to enable / disable vault configuration
  enabled: $enable-global-vault
  # Any one type of vault should be enable at a time
  hashicorpVault:
    # Switch to enable / disable Hashicorp vault
    enabled: $enable-hashicorp-vault
    # Hashicorp Vault server URL
    serverUrl: $hashicorp-vault-server-url
    # Hashicorp Vault role to access secrets
    role: "reader"
    # Hashicorp Vault Auth mount path
    authMountPath: "jwt"
    # Hashicorp Vault namespace
    vaultNamespace: ""
```

> **Note:** Role, authMountPath, and vaultNamespace have default values and are optional. If needed, define these tokens in the Helm-specific input JSON file and mention them in the values YAML files for federations requiring secrets from the HashiCorp Vault.

3. Ensure the below vault secret paths configured in the fed-opm_multi yaml file, under the global section of the chart.

```
secretNames:
  fedopmmulti:
    opmUsernameSecretName: secrets/data/opmUsernameSecretName
    opmPasswordSecretName: secrets/data/opmPasswordSecretName
    mongoUsernameSecretName: secrets/data/mongoUsernameSecretName
    mongoPasswordSecretName: secrets/data/mongoPasswordSecretName
    mongoInitUsernameSecretName: secrets/data/mongoInitUsernameSecretName
    mongoInitPasswordSecretName: secrets/data/mongoInitPasswordSecretName
    mongoConnectionStringSecretName: secrets/data/mongoConnectionStringSecretName
```

> Note: As mentioned in the prerequisite section, the secrets should be placed into the vault at the same secret vault path in the HashiCorp Vault server for MNOPM.

4. Run the helm deploy pipeline for the MNOPM installation and the helm update pipeline for the MNOPM upgrade.

Nexus Deployment

There will be no changes regarding Nexus Deployment as it is outside the scope of the HashiCorp Vault Integration with MNOPM

Verify the HashiCorp Vault Integration

Below are the steps to verify whether the secrets are being read from the HashiCorp Vault after the successful deployment / upgrade of MNOPM federation. Execute the following command on the master node using the command line interface (CLI).

kc get secrets -n vnfmgr

The TYPE of the MNOPM secrets (opm-auth, mongo-auth, and mongo-init-auth) should be "**Opaque**".

```
NAME                                      TYPE                    DATA   AGE
kubeconfig-def765-0123456789abcdef        Opaque                  1      12d
m2m-private-keys                          Opaque                  0      12d
mongo-auth                                Opaque                  3      12d
mongo-bootstrap-auth-data                 Opaque                  1      12d
mongo-init-auth                           Opaque                  3      12d
opm-auth                                  Opaque                  2      12d
opm-unitycloud-com-ingress-cert-manager   kubernetes.io/tls       3      12d
sh.helm.release.v1.fed-opm-multi.v1       helm.sh/release.v1      1      12d
users                                     Opaque                  1      12d
```